

NANYANG TECHNOLOGICAL UNIVERSITY



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

**INTERPRETABLE FUZZY DEEP TRANSFORMER NEURAL
NETWORK AND MULTI-AGENT REINFORCEMENT
LEARNING PORTFOLIO OPTIMIZATION**

PRIYA REKAH D/O M RAVI KUMAR

U2223260H

DEGREE IN COMPUTER ENGINEERING

SUPERVISOR: A/P QUEK HIOK CHAI

COLLEGE OF COMPUTING AND DATA SCIENCE

2025

Abstract

As artificial intelligence continues to shape the financial world, machine learning and reinforcement learning have become powerful tools for stock trading and portfolio management. These models can learn from market behavior and adapt to changing trends, helping investors make better, faster decisions. But in a market where no single model works best for every stock, flexibility is key [1].

The Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN) marks a significant leap forward in explainable AI for financial markets. By integrating deep transformer architectures with fuzzy logic, IFDTNN generates transparent, rule-based insights that bridge the gap between complex machine predictions and human understanding. This hybrid model not only maintains high predictive performance but also empowers traders to trust and interpret the rationale behind each trading signal, addressing one of the key challenges in modern AI-driven finance [2].

To harness the diversity of financial market behavior, this project introduces a multi-agent reinforcement learning framework that trains and evaluates six distinct agents. These include policy gradient methods [3], actor-critic models [4], deep evolution strategies [5] and more techniques. Each agent learns from technical indicators such as the Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD) [6], and smoothed moving averages, while also incorporating key fundamental metrics like return on equity and earnings growth. This variety allows each model to specialize and adapt to different market conditions [7].

Once trained, the highest-performing agent for each stock is selected to form a smart, diversified portfolio that leverages the strengths of all seven approaches. This dynamic allocation strategy improves both profitability and robustness, enabling real-time adaptation to market shifts. With the added layer of interpretability from IFDTNN, this system offers not just performance, but also clarity giving traders and investors a powerful, transparent, and future-ready toolkit for navigating the financial markets.

Acknowledgements

I want to sincerely thank Professor Quek Hiok Chai, who oversaw my final year project, for all of his help, support, and inspiration during this endeavor. His unwavering support allowed me to make tremendous progress in my job by offering valuable insights into the project's development. I have benefited from his sharings of his technical expertise and life teachings, which will motivate me long after this project is finished.

I would like to thank Chai Wei Heng Alwin for his study on the "Interpretable Fuzzy Deep Transformer Neural Network, IFDTNN," which I used in my project. I also want to thank my fellow mates for their invaluable guidance anytime I've been having problems. They have improved the project's enjoyment and inspired me to investigate fresh approaches and strategies to enhance executions.

Lastly, I want to express my sincere gratitude to my family and friends for the emotional support and encouragement throughout the course of the project.

Table of Contents

Abstract.....	1
Acknowledgements.....	2
Table of Contents.....	3
List of Tables.....	7
List of Figures.....	8
Chapter 1.....	10
Introduction.....	10
1.1 Background.....	11
1.2 Motivations.....	13
1.3 Objectives.....	13
1.4 Organisation of Report.....	14
Chapter 2.....	16
Literature Review.....	16
2.1 Clustering.....	16
2.1.2 K-Means Algorithm.....	18
2.2 Feature Selection.....	19
2.2.1 Monte Carlo evaluative selection (MCES).....	20
2.3 Neural Networks.....	20
2.3.1 Multi-Layer Perceptron (MLP).....	23
2.3.2 Long short-term memory (LSTM).....	24
2.3.2.1 Architecture.....	25
2.3.3 Bidirectional Long short-term memory (BI-LSTM).....	26
2.3.3.1 Architecture.....	26
2.3.4 Transformer.....	27
2.3.4.1 Architecture.....	27
2.3.4.2 Encoder and Decoder.....	28
2.3.4.3 Encoder-Only Transformers.....	29
2.3.4.4 Positional Encoding.....	29
2.3.4.5 Attention Mechanism.....	30
2.3.4.6 Pointwise Feedforward Networks.....	30
2.4 Optimization.....	31
2.4.1 Hyperparameters.....	34
2.4.2 Genetic Algorithms.....	36
2.5 Evaluation Metrics.....	37
2.5.1 Coefficient of Determination (R^2).....	37
2.5.2 Root Mean Squared Error (RMSE).....	38
2.6 Fuzzy Inference Systems.....	39

2.6.1 Fuzzy Logic.....	39
2.6.2 Fuzzy Set Theory.....	39
2.6.3 Fuzzy Rules.....	40
2.6.4 Types of Fuzzy Systems.....	40
2.6.5 Fuzzy Inference Process.....	41
2.7 Reinforcement Learning Algorithms.....	43
2.7.1 Deep Evolution Strategy Agent.....	46
2.7.2 Actor-Critic Agent.....	46
2.7.3 Recurrent Curiosity Q-Learning Agent.....	48
2.7.4 Proximal Policy Gradient Agent.....	49
2.7.5 Evolution Strategy Agent.....	49
2.7.6 Double Dueling Recurrent Q-Learning Agent.....	50
2.7.7 Deep Deterministic Policy Gradient Agent.....	52
Chapter 3.....	54
The Stock Market and Algorithmic Finance.....	54
3.1 Stock Market.....	54
3.1.1 Overview of the Singapore Exchange (SGX).....	54
3.1.2 Market Trends in the Singapore Stock Market.....	55
3.1.3 Financial Portfolio in Singapore Context.....	55
3.1.4 Company Fundamentals.....	56
3.2 Trend Indicators.....	57
Chapter 4.....	61
Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN).....	61
4.1 IFDTNN Architecture.....	61
4.1.1 Input Layer.....	63
4.1.2 Fuzzification Layer.....	63
4.1.3 Neural Network/ Rule Base Layer.....	64
4.1.4 Consequent Layer.....	65
4.1.5 Output Layer.....	66
4.2 Hebbian Learning Mechanism.....	66
4.3 Rule Firing Strength and Updating.....	66
4.3.1 Pseudo-weights of Fuzzy Rules (with Hebbian Learning).....	67
4.3.2 Identifying Dominant Clusters.....	67
4.3.3 Pseudo-weight Calculation (Firing Strength).....	68
Hebbian Update Rule.....	68
4.4 Memory Decay.....	69
4.5 Tagging Mechanism for Interpretability.....	69
Chapter 5.....	70
Experimental Results and Analysis.....	70
5.1 Dataset Segmentation and Usage.....	70
5.2 Objectives.....	70

5.3 Model Approach.....	71
5.4 Portfolio Selection.....	71
5.4.1 Diversified Portfolio Strategy Using SGX-listed Stocks.....	71
5.4.1.1 Fundamental Analysis.....	72
5.4.1.2 Technical Pattern Analysis.....	73
5.4.1.3 Reinforcement Learning-Based Weighting.....	76
5.4.1.4 Correlation-Based Filtering.....	77
5.5 IFDTNN Implementation.....	77
5.5.1 Data Preprocessing.....	78
5.5.1.1 Feature Engineering.....	78
5.5.1.2 Handling Data Shifts and Drifts: Sliding Window Normalization.....	81
5.5.2 K-Means Clustering for Fuzzification.....	82
5.5.2.1 Defining the Number of Clusters.....	82
5.5.2.2 Centroid Initialization.....	82
5.5.2.3 Cluster Merging for Generalization.....	83
5.5.2.4 Fuzzification Outcome.....	84
5.5.3 Feature Selection Using Monte Carlo Evaluative Selection (MCES).....	84
5.5.4 Hyperparameter Tuning.....	85
5.5.4.1 Parameter.....	85
5.5.4.2 Results and Insights.....	86
5.5.5 Evaluation IFDTNN Prediction.....	86
5.5.6 Benchmarking the IFDTNN Architecture.....	87
5.5.6.1 Crisp Feature Training as a Baseline.....	87
Chapter 6.....	88
Experimental Results and Benchmarking.....	88
6.1 MCES Feature Selection Results.....	88
6.2 Evaluation of K-Means Clustering-Based Defuzzified Predictions.....	91
6.3.2 Hyperparameter Optimization.....	96
6.3.3 Fuzzy Rule Base.....	97
6.3.4 IFDTNN Architecture Benchmark Results.....	98
6.3.4.1 Benchmarking IFDTNN Architecture on Crisp Feature Training.....	98
Chapter 7.....	100
Trend Reversals and Algorithmic Finance.....	100
7.1 Buy and Hold Strategy: Baseline Benchmark.....	100
7.2 Technical Indicators and Trend Analysis.....	101
7.2.1 Simple Moving Average(SMA).....	101
7.2.2 Volatility Penalty.....	102
7.2.3 MACD.....	102
7.2.1 Experiment 1.....	106
7.2.1 Experiment 2.....	106
7.3 Genetic Algorithm-Based Threshold Optimization.....	107

7.3.1 Experiment 3.....	108
7.4 Comparison of the Results.....	110
Comparative Performance Summary (Initial Investment: \$10,000 each, 2500 Days).....	110
Chapter 8.....	111
Reinforcement Learning and Multi-Agents for Trading.....	111
8.1 Portfolio Optimisation.....	111
8.1.1 Targeted Parameter Optimization.....	112
8.1.2 Second-Order RMA: Noise Reduction Layer.....	112
8.1.3 Adaptive Multi-Indicator Signal Blending.....	112
8.2 Baseline A2C Architecture and Initialization.....	113
8.2.1 Action Sequence Analysis.....	113
8.2.2 Parameters:.....	113
8.2.3 Trading Logic:.....	115
8.3 Experiment 4.....	115
8.4 Experiment 5.....	119
8.5 Cross- Agent Performance Benchmark.....	120
8.6 Multi-Agent Performance.....	122
8.7 IFDTNN Multi-Agent Performance During High Volatility Period (COVID-19).....	124
Chapter 9.....	126
Conclusion and Future Works.....	126
9.1 Conclusion.....	126
9.2 Future Works.....	127
9.2.1 Incorporating a More Robust Ensemble strategy model for Agent Models.....	127
9.2.2 Real-Time Online Trading System.....	127
9.2.3 Multi-Asset and Multi-Market Expansion.....	127
9.2.4 Inclusion of High-Volatility or Sentiment-Driven Stocks.....	128
9.2.5 Automated Stock Screening System.....	128
9.2.6 Enhanced Explainability and Regulatory Readiness.....	128
9.3 Final Remarks.....	128
References.....	129
Appendix.....	139

List of Tables

Table 2.1 Displays a taxonomy of clustering algorithms.....	18
Table 2.3 Lists down the activation functions.....	23
Table 2.4 Transformer-Specific Hyperparameters.....	34
Table 2.5 Fuzzy Rules.....	41
Table 2.6 Defuzzification Process.....	42
Table 2.7 Displays Taxonomy of Agents.....	45
Table 3.1 Key Financial Indicators.....	57
Table 3.2 List of Trend Indicators.....	59
Table 6.1 Top features selected Using MCES for C38U.SI Look-Ahead Models.....	89
Table 6.2 Cluster-based Prediction Performance (Test Dataset).....	92
Table 6.3 R2 model performance of all stocks.....	93
Table 6.5 C38U.SI (t+13) Hyperparameter Tuning Outcome.....	96
Table 6.6 Top Fuzzy Rules of t+13 Models.....	98
Table 6.7 IFDTNN Benchmark on C38U.SI Crisp Feature Training Performance Comparison.....	99
Table 7.1 Simple Moving Average.....	102
Table 7.2 ROI of Buy and Hold Strategy without commission fees.....	106
Table 7.3 ROI of Buy and Hold Strategy with commission fees.....	107
Table 7.4 Genetic Algorithm (GA) Hyperparameters.....	108
Table 7.5 ROI of Buy and Hold Strategy with commission fees with GA.....	109
Table 7.6 Comparisons of ROI of Buy and Hold Strategy.....	110
Table 8.1 Features of Reinforcement learning.....	111
Table 8.2 Parameters used to optimise Agents.....	114
Table 8.3 ROI of optimised Actor Critic Agent without commission fees.....	116
Table 8.4 ROI of optimised Actor Critic Agent with commission fees.....	119
Table 8.5: ROI Comparison Across Agents.....	121
Table 8.6 Unpredicted Multi-agent performance of overall portfolio returns based on ROI.. 123	
Table 8.7 Comparison of Multi-agent performance of Portfolio A and B returns based on ROI.....	124

List of Figures

Figure 2.1 Deep Neural Network.....	21
Figure 2.2 Multi layer Perceptron.....	24
Figure 2.3 Recurrent Neural Network Pipeline.....	25
Figure 2.4 Long Short Term Memory Pipeline.....	25
Figure 2.6 The model design of the benchmarking LSTM and BI-LSTM.....	27
Figure 2.7 Transformer Architecture.....	28
Figure 2.8 Attention Mechanism.....	30
Figure 2.9 Grid Search.....	31
Figure 2.10 Random Search.....	32
Figure 2.11 Genetic Algorithm.....	32
Figure 2.13 Gradient-Based.....	33
Figure 2.14 Genetic Algorithm Pipeline.....	36
Figure 2.15 Fuzzy inference process in a Mamdani-type system.....	42
Figure 2.16 Reinforcement learning framework.....	43
Figure 2.17 Multi-Agent Reinforcement learning framework [50].....	44
Figure 2.18 Actor Critic Network.....	47
Figure 3.1 Comprehensive Technical Analysis of Capital and Integrated Commercial Trust (C38U.SI).....	60
Figure 4.1 IFDTNN Architecture.....	62
Figure 5.1 Selected six Singapore Stocks.....	73
Figure 5.2 Dynamic Price Trend of Cyclic Portfolio of Q0F.SI.....	73
Figure 5.3 Dynamic Price Trend of Cyclic Portfolio of AJBU.SI.....	74
Figure 5.4 Dynamic Price Trend of Cyclic Portfolio of C38U.SI.....	74
Figure 5.5 Dynamic Price Trend of Cyclic Portfolio of S63.SI.....	75
Figure 5.6 Dynamic Price Trend of Cyclic Portfolio of S68.SI.....	75
Figure 5.7 Dynamic Price Trend of Cyclic Portfolio of U11.SI.....	76
Figure 5.8 RL weights of the Selected Stocks.....	76
Figure 5.9 Correlation matrix of selected stocks.....	77
Figure 5.10 Feature Engineering.....	80
Figure 6.1 MCES Feature Importance across (t+1 to t+13) of C38U.SI.....	90
Figure 6.2 Frequency Statistics of the features of MCES.....	91
Figure 6.3 Trends of IFDTNN Prediction Performance.....	94
Figure 6.4 Consolidation of stocks Prediction vs Actual Trace T+5 and T+10.....	95
Figure 7.1 Trend and Crossover Signals of CapitaLand Integrated Commercial Trust (C38.SI).....	103

Figure 7.2 Trend and Crossover Signals of Keppel DC REIT (AJBU.SI).....	103
Figure 7.3 Trend and Crossover Signals of IHH Healthcare (Q0F.SI).....	104
Figure 7.4 Trend and Crossover Signals of ST Engineering (S63.SI).....	104
Figure 7.5 Trend and Crossover Signals of SGX (S68.SI).....	105
Figure 7.6 Trend and Crossover Signals of UOB Bank (U11.SI).....	105
Figure 8.1 Buy and Sell indicators of U11.SI.....	116
Figure 8.2 Buy and Sell indicators of C38U.SI.....	117
Figure 8.3 Buy and Sell indicators of Q0F.SI.....	117
Figure 8.4 Buy and Sell indicators of S68.SI.....	117
Figure 8.5 Buy and Sell indicators of S63.SI.....	118
Figure 8.6 Buy and Sell indicators of AJBU.SI.....	118
Figure 8.7 Overall ROI of individual stocks.....	120
Figure A.1 Frequency Statistics of the features of MCES of AJBU.SI.....	139
Figure A.2 MCES Feature Importance across (t+1 to t+13) of AJBU.SI.....	140
Figure A.3 Frequency Statistics of the features of MCES of Q0F.SI.....	141
Figure A.4 MCES Feature Importance across (t+1 to t+13) of Q0F.SI.....	141
Figure A.5 Frequency Statistics of the features of MCES of S63.SI.....	141
Figure A.6 MCES Feature Importance across (t+1 to t+13) of S63.SI.....	142
Figure A.7 Frequency Statistics of the features of MCES of S68.SI.....	142
Figure A.8 MCES Feature Importance across (t+1 to t+13) of S68.SI.....	143
Figure A.9 Frequency Statistics of the features of MCES of U11.SI.....	144
Figure A.10 MCES Feature Importance across (t+1 to t+13) of U11.SI.....	144

Chapter 1

Chapter 1 introduces the motivation behind applying artificial intelligence, deep reinforcement learning, to stock trading. It outlines the research problem, objectives, and significance of using a multi-agent system to maximize returns through adaptive, data-driven decision-making.

Introduction

Financial stock prices are vital indicators of economic performance and are at the core of decision-making in trading, investment, and risk management. Accurately forecasting stock prices has long been a challenge due to the volatile, non-linear, and noisy nature of financial markets. Traditionally, this task has relied on statistical models such as autoregressive integrated moving average (ARIMA) and time series analysis techniques [8]. While these models offer mathematical clarity, they often struggle to capture the intricate patterns and sudden shifts inherent in real-world financial data.

The advent of deep learning has significantly improved our ability to uncover complex, hidden relationships within large datasets. Techniques such as recurrent neural networks (RNNs), long short-term memory (LSTM), and convolutional architectures have outperformed traditional models in many forecasting tasks, especially in capturing temporal dependencies and nonlinear dynamics [9]. As a result, deep learning has become increasingly popular in computational finance for predicting price movements, identifying market regimes, and making data-driven decisions.

However, despite their predictive power, deep learning models are often criticized for their lack of transparency. These "black box" systems provide limited insight into how decisions are made, raising concerns in domains like finance where explainability, trust, and regulatory compliance are critical. Financial institutions and individual investors alike require models that not only perform well but also offer clear, interpretable reasoning behind their predictions [10].

To address this, fuzzy logic systems have emerged as a complementary approach. Unlike deep learning, fuzzy systems can encode expert knowledge using human-readable rules and linguistic labels. Although they may not match the predictive accuracy of deep networks on their own, their

interpretability makes them valuable in high-stakes environments. Combining these strengths has led to the development of neuro-fuzzy systems hybrid architectures where fuzzy inference is embedded within a neural network framework to enable both learning and reasoning. Mamdani-type systems tend to favor interpretability, while Takagi-Sugeno-Kang (TSK) models focus more on precision [11]. Still, a trade-off between accuracy and interpretability persists.

This project explores a novel direction by integrating the interpretability of fuzzy logic with the decision-making efficiency of reinforcement learning agents in a portfolio optimization context. Specifically, the system leverages seven different reinforcement learning agents, policy gradient, actor-critic, curiosity-driven learning, deep evolution strategy, recurrent Q-learning, and double-dueling deep Q-learning to individually learn trading policies for selected stocks. Each agent uses a combination of technical indicators such as the Relative Strength Index (Wilder, 1978), Moving Average Convergence Divergence (Appel, 2005), and second-order smoothed moving averages to interpret market signals and guide decisions.

To optimize performance, the framework selects the best-performing agent for each stock based on return on investment and integrates them into a composite portfolio. This multi-agent architecture ensures that each stock is traded by the strategy best suited to its specific characteristics, allowing for specialization and improved outcomes. The resulting hybrid portfolio demonstrates how combining diverse learning paradigms with fuzzy-enhanced interpretability and robust selection logic can lead to high-performing, adaptive, and explainable financial systems.

Through this work, we aim to push the boundaries of explainable artificial intelligence in finance by merging reinforcement learning, fuzzy logic, and deep generative techniques into a comprehensive system for stock trading and portfolio optimization.

1.1 Background

In recent years, the rapid advancement of artificial intelligence and deep learning has transformed many sectors, including finance. Deep learning architectures, such as multi-layer perceptrons (MLPs), recurrent neural networks (RNNs), Transformers and long short-term memory networks (LSTMs), have been widely adopted in financial forecasting tasks. Their ability to automatically extract features from vast amounts of sequential data makes them a natural fit for stock price

prediction, where the relevance of indicators like volume, price, or momentum can vary over time [12].

However, while deep learning models can deliver strong predictive performance, they often behave as “black boxes,” providing little insight into the reasoning behind their decisions. This lack of transparency is particularly concerning in finance, where interpretability, accountability, and trust are paramount[13]. To address this, hybrid models that integrate fuzzy logic and rule-based reasoning have been developed to improve interpretability without sacrificing accuracy.

At the same time, another promising approach in financial modeling is reinforcement learning (RL), which allows agents to learn optimal trading strategies through trial and error. Unlike supervised learning, RL algorithms make decisions sequentially and adaptively based on rewards and penalties, mimicking the real-time dynamics of market behavior. Still, no single agent is universally effective across all types of stocks, sectors, or market conditions. Each stock may exhibit unique patterns that align better with certain learning styles or model assumptions.

To overcome this limitation, this project proposes a multi-agent reinforcement learning framework, where seven different agents are trained and evaluated individually for each stock. These include actor-critic agents, curiosity-driven explorers, policy gradient learners, deep evolution strategies, recurrent dueling Q-networks, and deterministic deep policy gradient (DDPG) agents. Each agent learns to trade based on price patterns and technical indicators such as the Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), and second-order smoothed moving averages tools widely used to detect trends and reversals in the market [14].

After training, the best-performing agent for each stock measured by return on investment (ROI) is selected. This strategy enables each model to specialize in a specific stock where it performs best. The individual stock-level agents are then integrated into a dynamically optimized portfolio, creating a system that adapts to the unique behavior of each asset. This form of portfolio allocation reflects a data-driven, modular approach to financial decision-making, where AI is used not just to predict but also to tailor and optimize performance across multiple assets.

By combining deep reinforcement learning with modular agent specialization and "Interpretable Fuzzy Deep Transformer Neural Network, IFDTNN, this project aims to develop a robust, adaptive,

and interpretable framework for stock trading. It bridges the gap between predictive accuracy and human-understandable strategies, contributing to the growing field of explainable AI in finance.

1.2 Motivations

In financial markets, different stocks exhibit unique behaviors influenced by factors like volatility, trends, and liquidity, making it unrealistic to expect a single model to consistently outperform across all assets. This project is driven by the insight that leveraging the strengths of multiple reinforcement learning agents each with different architectures and learning strategies can result in more adaptive and effective stock-specific strategies. By selecting the agent that delivers the highest return for each individual stock and combining their outputs into a dynamic portfolio, the system maximizes overall returns while reducing risk through diversification. This multi-agent approach mirrors the logic of specialized asset managers, each focused on optimizing a particular stock, thereby enhancing the overall financial outcome. By combining multiple reinforcement learning agents and selecting the best-performing model for each stock, alongside using interpretable models like the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN), this research aims to build an adaptive, explainable, and high-performing portfolio optimization strategy.

1.3 Objectives

The objective of this project is to design a Interpretable Deep Fuzzy Transformer Neural Network (IFDTNN) and a dynamic, multi-agent reinforcement learning system that maximizes long-term portfolio returns by leveraging the strengths of diverse agent architectures and stock-specific behavior. This project understands how important it is for models to be interpretable not just for better decision-making, but also to build trust with stakeholders, especially when the outcomes carry high stakes. Recognizing that no single learning model can generalize optimally across all market conditions, this system identifies the best-performing agent for each stock and integrates them into an adaptive portfolio allocation strategy. Through technical indicators, price trends, and reinforcement signals, the system aims to deliver a robust, explainable, and high-performing financial model.

The specific objectives are as follows:

1. Engineering meaningful features from raw financial market data
2. Applying clustering techniques to fuzzify features for better interpretability
3. Building predictive models using deep neural network architectures
4. Generating human-readable fuzzy rules from learned model behavior
5. Evaluating the predictive accuracy of the transformer component in IFDTNN
6. To preprocess and engineer technical indicators from real-world stock price data.
7. To implement and train multiple reinforcement learning agents (e.g., policy gradient, actor-critic, curiosity-based, evolution strategy, etc.) on historical market data.
8. To evaluate each agent's performance on a per-stock basis using return on investment (ROI) metrics.
9. To select the best-performing agent per stock and construct an optimized hybrid portfolio.

1.4 Organisation of Report

Chapter 2 presents a comprehensive literature review covering the key technical concepts and algorithms that underpin the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN) and reinforcement learning methodologies. And describes the development of seven different reinforcement learning agents used for trading. It includes agent architectures, exploration strategies, and how each was trained on market data.

Chapter 3 outlines the fundamentals of stock market dynamics, trading principles, and key financial indicators. It sets the foundation for understanding how algorithmic strategies interact with real-world financial data.

Chapter 4 introduces the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN), detailing its architecture, fuzzy rule integration, and how it enhances both prediction performance and interpretability.

Chapter 5 presents the key parameters related to the IFDTNN architectures used in the project. All models and experiments were implemented using the Python programming language.

Chapter 6 presents the experimental setup, including dataset preparation, training pipelines and feature engineering.

Chapter 7 analyzes and compares the performance of buy and hold strategy with Genetic Algorithms

Chapter 8 dives into the development of seven unique reinforcement learning (RL) agents designed specifically for stock trading. It covers the architecture of each agent, their exploration strategies, and how they were trained using historical market data.

Chapter 9 concludes the report by summarizing key insights and provides suggestions of future work.

Chapter 2

Literature Review

Chapter 2 presents a comprehensive literature review covering the key technical concepts and algorithms that underpin the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN) and reinforcement learning methodologies. And describes the development of seven different reinforcement learning agents used for trading. It includes agent architectures, exploration strategies, and how each was trained on market data.

2.1 Clustering

Clustering is an essential unsupervised learning method used to discover inherent groupings within data based on similarity or proximity without prior labels. The objective is to partition the data into distinct subsets (clusters), such that data points within the same cluster are more similar to each other than to those in other clusters. Various clustering techniques have been developed, each with different assumptions, strengths, and use cases. These techniques are broadly categorized as follows:

Clustering Algorithms	Description	Advantages	Limitations
K-Means [15]	A partitioning algorithm that divides data into k clusters by minimizing the within-cluster sum of squares. Each cluster is represented by its centroid.	- Simple and scalable - Efficient for large datasets - Works well on numerical stock indicators	- Requires predefined k - Assumes spherical clusters - Sensitive to outliers and noise

Agglomerative [16]	A bottom-up hierarchical method that starts with individual points and merges the closest clusters iteratively to form a dendrogram.	- Reveals nested cluster relationships - No need to predefine k	- High computational cost - Difficult to scale for large stock datasets
Density-Based Spatial Clustering of applications with Noise (DBSCAN)[17]	Groups data points that are closely packed together while marking points in low-density regions as noise. Ideal for discovering non-linear patterns.	- Detects clusters of arbitrary shapes - Handles outliers well	- Poor performance with varying densities - Parameter tuning (ϵ, minPts) can be tricky
Gaussian Mixture Model (GMM) [18]	A model-based probabilistic clustering that assumes data is generated from a mixture of Gaussian distributions. Each point has a probability of belonging to each cluster.	- Probabilistic (soft) assignments - Handles elliptical clusters - Useful in financial modeling	- Sensitive to initialization - May overfit on small datasets - Assumes Gaussian distribution
Mean Shift [19]	A non-parametric clustering method that updates centroids by shifting to the mean of data points in the neighborhood.	- No need to specify number of clusters - Identifies cluster modes - Works well with financial indicator densities	- Computationally expensive - Not ideal for very large stock datasets

Table 2.1 Displays a taxonomy of clustering algorithms

2.1.2 K-Means Algorithm

The K-Means algorithm is one of the most widely used clustering techniques in unsupervised machine learning. It works by grouping data points into k distinct clusters based on similarity, helping to uncover hidden patterns or structures within datasets without using labeled outcomes.

At its core, K-Means attempts to partition the dataset into k clusters such that the data points within each cluster are as close as possible to the cluster's centroid (the average position of all points in that group). The measure of "closeness" is typically the Euclidean distance [20].

Euclidean Distance Formula:

The distance between two points a and b in multi-dimensional space is calculated using the following formula:

$$d(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

where:

- a and b are vectors (data points) in a feature space,
- a_i and b_i represent the individual components (features) of those vectors.

This distance quantifies how similar or dissimilar two data points are—the smaller the distance, the more similar they are.

1. Initialization:

Select k initial centroids either randomly or using advanced techniques such as K-Means++ to improve stability and convergence.

2. Cluster Assignment:

Each data point is assigned to the nearest centroid based on the Euclidean distance.

3. Centroid Update:

After assigning all data points, the centroid of each cluster is updated by taking the mean of all points within that cluster. This is given by Equation (2.2):

$$(\bar{x}_0, \dots, \bar{x}_K) = ((1/L) \sum_{i=1}^n x_{0i}, \dots, (1/L) \sum_{i=1}^n x_{Ki})$$

where,

x_n = the n coordinate value of a data point in euclidean n -space

x_{ni} = the n coordinate value of data point i in Euclidean n -space

L = number of data points in the dataset

The algorithm iterates until the cluster assignments stabilize either when the centroids no longer change between iterations or when a predefined maximum number of iterations is reached [21].

In this project, the K-Means algorithm is applied in a one-dimensional feature space. The objective is to cluster data points along this single dimension, allowing the use of cluster statistics to describe and segment the dataset. The algorithm's reliance on Euclidean distance makes it especially suitable for such low-dimensional applications, ensuring both speed and stability in the convergence process.

Furthermore, the flexibility of centroid initialization—whether through random selection or methods like K-Means++ offers additional control over convergence behavior and cluster quality. This helps in producing more meaningful and representative clusters that effectively capture the underlying structure of the data.

2.2 Feature Selection

Feature selection is a vital step in data preprocessing, especially for deep learning models, which are prone to the curse of dimensionality. In high-dimensional spaces, data becomes increasingly sparse, requiring more complex models and raising the risk of overfitting due to limited generalization capabilities [22]. By selecting only the most relevant features and removing redundant ones, feature selection reduces input dimensionality, ultimately enhancing model performance [23]. For deep

neural networks, this process also boosts training efficiency, as a reduced feature space leads to simpler models with fewer parameters such as weights and biases to learn and update.

2.2.1 Monte Carlo evaluative selection (MCES)

Monte Carlo Evaluative Selection (MCES) is a data-driven, domain-agnostic feature selection algorithm that is well-suited for both regression and classification tasks. Unlike traditional methods that depend heavily on specific learning models or prior knowledge of the problem space, MCES evaluates the importance of each feature based on simulated, model-agnostic evaluations. This makes it highly flexible and broadly applicable in data-driven environments where underlying relationships are unknown or complex.

The core idea behind MCES is to assess how changes in feature availability affect model performance through random simulations. It leverages the Weak Law of Large Numbers, which guarantees that the output of the simulations will converge as the number of iterations increases. By repeatedly altering feature inputs and measuring the impact on prediction error, MCES can identify features that contribute meaningfully to the model's performance.

In this project, although only 24 features are present initially, the fuzzification process significantly expands the input space. If each feature is represented by six fuzzy clusters, the resulting input dimensionality could easily exceed 100. Processing all these features without filtering would introduce unnecessary complexity and increase the risk of overfitting. Therefore, MCES is applied prior to fuzzification to prune out irrelevant or weakly contributing features. This not only reduces computational overhead but also improves the interpretability and generalization of the final model.

Since MCES does not rely on any specific model assumptions or domain knowledge, and works purely through randomized evaluations, it is highly compatible with the project's data-driven approach. It offers a robust method for identifying the most informative features, thereby enhancing model performance while maintaining simplicity.

2.3 Neural Networks

A neural network is a machine learning model inspired by the way the human brain processes information. It consists of layers of interconnected nodes, or "neurons", that are designed to

recognize complex patterns within data. Each neuron receives input, processes it using mathematical functions, and passes the result to the next layer in the network. By adjusting the strengths of these connections known as weights during training, the network learns to make increasingly accurate predictions.

In the context of this project, neural networks serve as the foundation for building models that can learn from financial data, adapt to patterns, and assist in forecasting or decision-making tasks with improved accuracy[24].

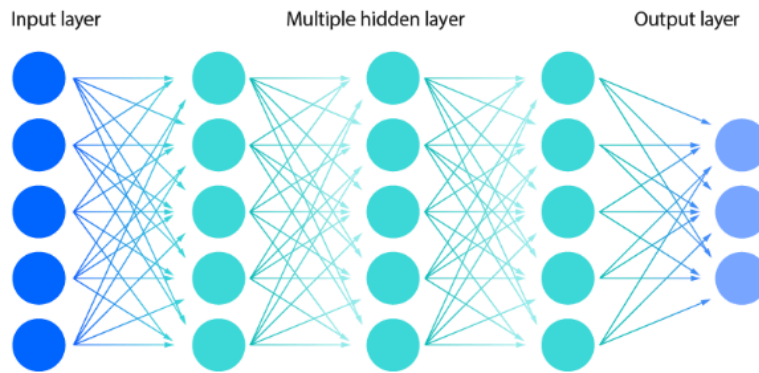


Figure 2.1 Deep Neural Network

Artificial Neuron

An artificial neuron serves as the computational core of neural networks. It processes multiple inputs, applies weighted summation along with a bias term, and passes the result through an activation function to produce a final output. The input aggregation (or synaptic input) is mathematically expressed as:

$$u = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

where:

u = Total synaptic input to the neuron

x_i = Feature/ input i

w_i = Weight associated with input x_i

b = Bias term

Once the synaptic input is calculated, it is passed through an activation function which introduces non-linearity and decides whether the neuron should be activated [24].

Activation Functions

Sigmoid [25]	$f(u) = \frac{1}{1+e^{-bu}}$	Output range = (0,1)	Compresses output between 0 and 1; interpretable as probability. But suffers from vanishing gradients in deeper layers.
Rectified Linear Unit (ReLU) [25]	$f(u) = \max(0, u)$	[0,∞)	It is efficient and widely used. and helps speed up convergence, but inactive (zero) for negative inputs.
GeLu (Gaussian Error Linear Unit) [25]	$\text{GELU}(x) = x \cdot \Phi(x)$	$(-\infty, \infty)$	It combines properties of ReLU and sigmoid/tanh, applying a probabilistic gating mechanism based on the input's position on the standard normal curve.
Tanh (Hyperbolic Tangent) [25]	$f(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$	(-1,1)	It has zero-centered output. It offers stronger gradients than sigmoid but still prone to saturation issues.
Linear [25]	$f(u) = u$	$(-\infty, \infty)$	There is no nonlinearity

			introduced. It is used in simple regression models. Not ideal for complex patterns due to constant gradient.
--	--	--	--

Table 2.3 Lists down the activation functions

where

e = Euler's number (approx. 2.718)

u = represents the synaptic input to the neuron

b = in the sigmoid is a slope parameter controlling steepness

Forward and Back Propagations

Forward propagation takes the input, passes it through the layers of the neural network using the current weights and biases, and produces an output (such as a prediction or score). Along the way, it stores intermediate results (like hidden activations) needed for learning. Backward propagation starts from the output error (the difference between the predicted output and the actual target), and works backwards through the network to compute the gradients of the loss with respect to each parameter (weights and biases). These gradients are the output of backpropagation and are used to update the model's parameters using an optimization algorithm like SGD. This loop runs during training until the network learns to minimize the error [26].

2.3.1 Multi-Layer Perceptron (MLP)

A Multilayer Perceptron (MLP) is a type of feedforward neural network that consists of multiple layers of neurons: an input layer, one or more hidden layers, and an output layer. Each neuron in a layer is connected to every neuron in the subsequent layer through weighted connections[27]. During

training, the MLP learns to adjust these weights to minimize the error in its predictions, enabling it to model complex relationships in data[28].

The MLP model as the predictive model trained for the in feature selection model Monte Carlo Evaluation Selection (MCES).

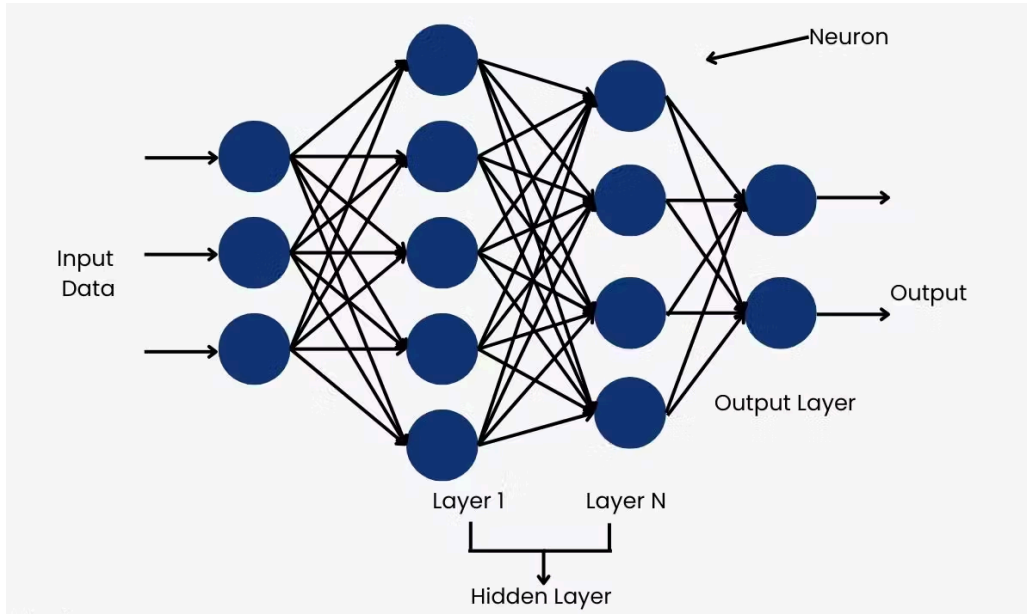


Figure 2.2 Multi layer Perceptron

2.3.2 Long short-term memory (LSTM)

Long Short Term Memory (LSTM) networks are specialized type of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data. Unlike traditional RNNs, which tend to struggle with learning from distant past inputs due to the vanishing gradient problem, LTSMs maintain a more stable learning process over long sequences. This is made possible through their internal gating mechanism consisting of the input data, forget gate, and output gate which collectively regulate the flow of information through the network. These gates determine which parts of the previous memory to retain, what new information to accept, and what to output at each time step[29].

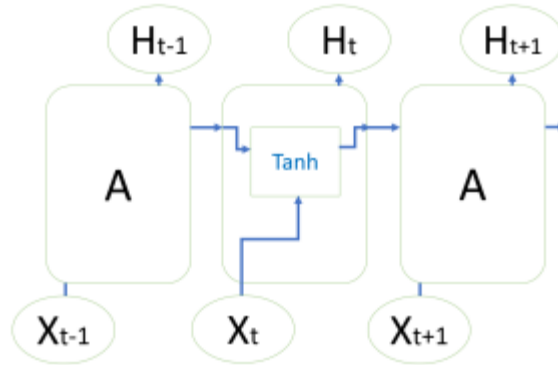


Figure 2.3 Recurrent Neural Network Pipeline

2.3.2.1 Architecture

In time series forecasting, LSTMs are especially useful because of their ability to model temporal dependencies across days, weeks, or even longer periods. Their architecture allows them to learn the underlying patterns and relationships in stock prices, where past values can have a significant influence on future trends. LSTM models also tend to handle noise relatively well and are more resistant to unstable training compared to vanilla RNNs.

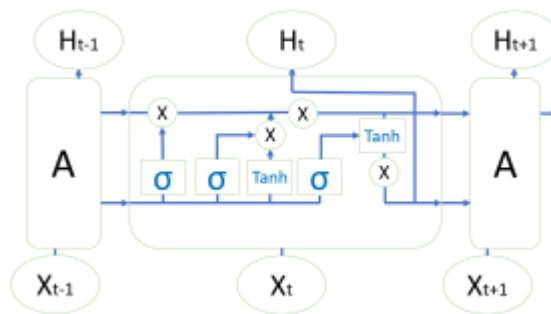


Figure 2.4 Long Short Term Memory Pipeline

LSTM Pipeline:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \text{ (Forget Gate)}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \text{ (Input Gate)}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \text{ (Output Gate)}$$

However, LSTMs come with certain limitations. They are computationally more expensive due to the additional parameters and complexity introduced by their gate structure[30]. As a result, training can be slower, especially on large datasets or with longer sequence lengths. Additionally, LSTM models are prone to overfitting if not carefully regularized, and may still struggle with very long sequences if the architecture or hyperparameters are not well-tuned.

2.3.3 Bidirectional Long short-term memory (BI-LSTM)

Bidirectional LSTM (BiLSTM) enhances the standard LSTM model by adding a second LSTM layer that reads the input sequence in the reverse direction. This means that for each time step, the model has access not only to the preceding inputs (as in a traditional LSTM), but also to the future inputs within the same sequence. By combining both forward and backward contextual information, BiLSTM captures temporal dependencies more comprehensively, making it particularly effective for tasks where understanding the full sequence context is essential—such as sentiment analysis, speech recognition, and stock price forecasting using historical data. In the case of stock prediction, BiLSTM can detect subtle relationships in price movements by analyzing how future and past data jointly influence a prediction point. Technically, this is achieved by concatenating or merging the outputs from both directions before passing them to subsequent layers[29].

2.3.3.1 Architecture

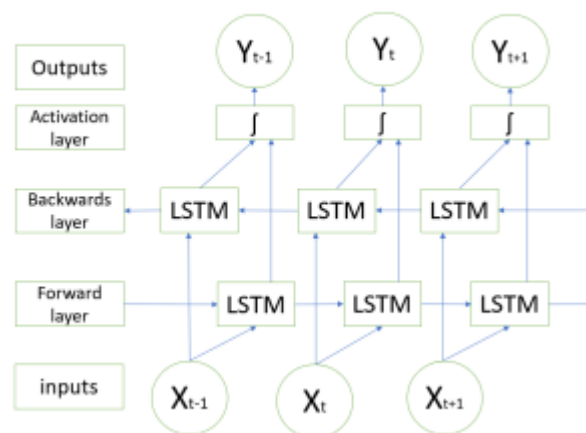


Figure 2.5 Long Short Term Memory Pipeline

Despite its strengths, BiLSTM is best suited for offline analysis or backtesting scenarios where the complete data sequence is available in advance, as it cannot operate in real-time forecasting where

future values are not yet known. It also introduces additional computational overhead and longer training time due to the doubled network complexity. Nevertheless, with proper hyperparameter tuning and architecture design, BiLSTM often outperforms unidirectional LSTM in accuracy and robustness, particularly in complex, nonlinear time series like stock markets.

In this project, the BiLSTM and LSTM model is used as a benchmark to evaluate the performance of our proposed forecasting model. By processing time series data in both forward and backward directions, the models provide high accuracy, allowing us to fairly assess the effectiveness and improvements of our model against a well-established deep learning architecture.

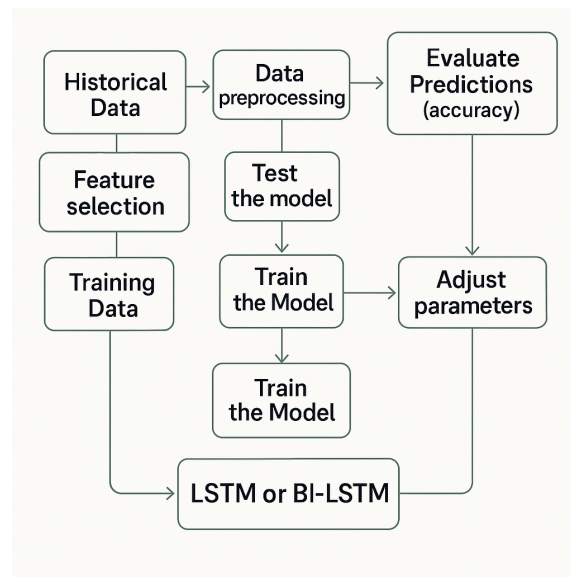


Figure 2.6 The model design of the benchmarking LSTM and BI-LSTM

2.3.4 Transformer

The Transformer is a deep learning architecture introduced by Vaswani et al. (2017), which replaced recurrence with self-attention mechanisms. Unlike RNNs, transformers allow for parallel computation and long-range dependency modeling, making them the foundation for modern large language models like GPT, BERT, and T5.

2.3.4.1 Architecture:

The Transformer architecture is built from a stack of identical layers, each consisting of two main subcomponents: a multi-head self-attention mechanism and a position-wise feedforward network, both wrapped with residual connections and layer normalization. The model processes input sequences in parallel by first embedding tokens and adding positional encodings to capture word order. In encoder-decoder models, the encoder processes the entire input sequence to produce contextual representations, which the decoder then uses, along with masked self-attention, to generate the output sequence token by token[31]. In decoder-only transformers (like GPT), the architecture consists solely of stacked decoder blocks, where each token attends only to previous tokens (causal attention). The uniform dimensionality of inputs and outputs across all layers allows for deep stacking, enabling transformers to model complex, long-range dependencies efficiently[32].

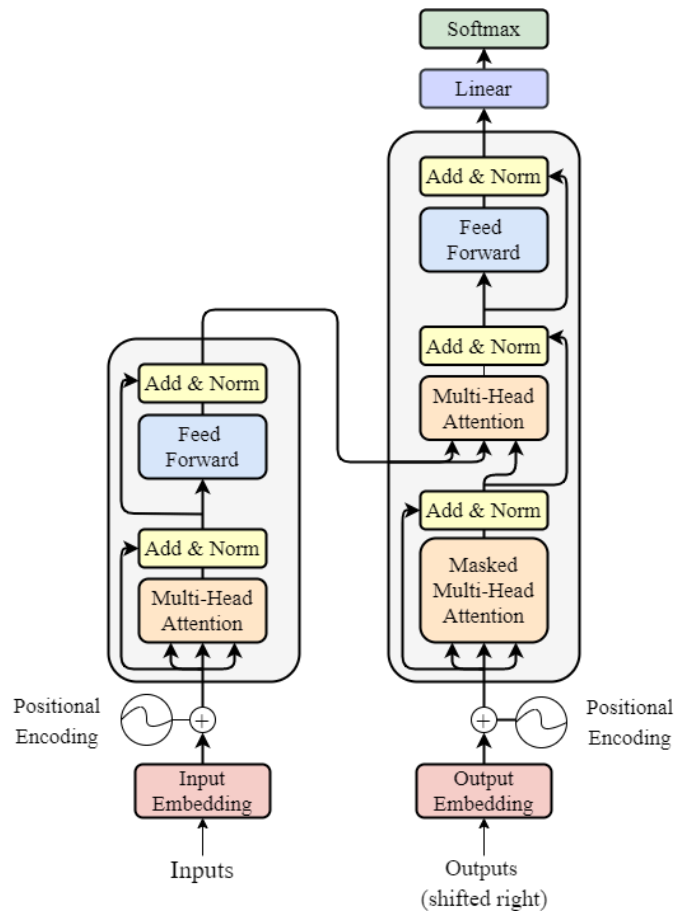


Figure 2.7 Transformer Architecture

2.3.4.2 Encoder and Decoder

The original Transformer consists of two main components:

Encoder: Takes the input sequence, processes it through stacked transformer blocks (each with self-attention and feedforward layers), and produces contextualized representations.

Decoder: Generates output sequences, attending to both its past outputs and encoder outputs using masked self-attention and encoder-decoder attention[33].

2.3.4.3 Encoder-Only Transformers

The encoder-only Transformer architecture is designed for understanding tasks like text classification, sentiment analysis, and named entity recognition. It consists of a stack of identical Transformer encoder blocks, where each block includes a multi-head self-attention layer followed by a position-wise feedforward network, both wrapped with residual connections and layer normalization. Input tokens are first converted into embeddings and combined with positional encodings to retain information about word order. The self-attention mechanism allows each token to attend to all other tokens in the input sequence, capturing contextual relationships regardless of their distance. The output is a sequence of contextualized embeddings, one for each input token, which can be further processed depending on the task. Models like BERT are based on this encoder-only architecture, focusing on understanding and representing input text rather than generating new sequences.

2.3.4.4 Positional Encoding

Since transformers lack recurrence, they use positional encodings to inject information about the order of tokens. These can be learned vectors or fixed sinusoidal patterns, and are added to token embeddings before passing through the network[34].

Equation:

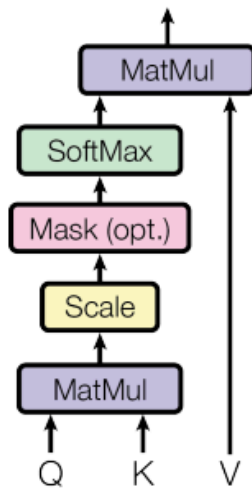
$$PE_{p,i} = \begin{cases} \sin\left(\left(\frac{p}{10000}\right)^{i/d}\right) & i \text{ even} \\ \cos\left(\left(\frac{p}{10000}\right)^{(i-1)/d}\right) & i \text{ odd} \end{cases}$$

where p indicates the position of the token in the sentence, i is the i th component of the vector, d is the model embedding dimension.

2.3.4.5 Attention Mechanism:

At the heart of the transformer is the self-attention mechanism, which allows each token to attend to other tokens in the input. It computes weighted combinations of token representations using query (Q), key (K), and value (V) matrices, enabling the model to focus on contextually relevant words. Multi-head attention enhances this by learning diverse relationships in parallel[32].

Scaled Dot-Product Attention



Multi-Head Attention

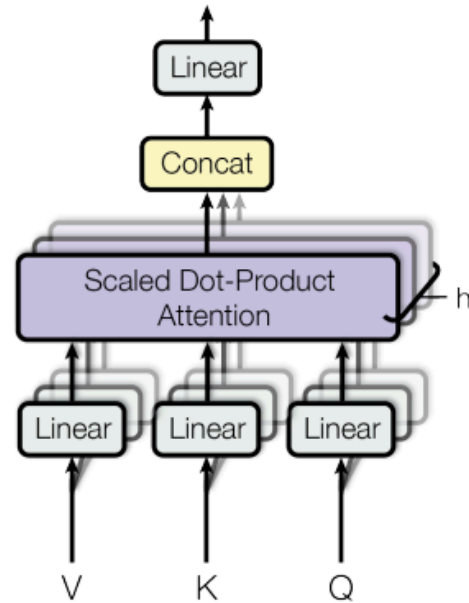


Figure 2.8 Attention Mechanism

2.3.4.6 Pointwise Feedforward Networks

After the attention layer, each token passes through a feedforward neural network applied independently at each position. This consists of two linear layers with a ReLU activation in between and is used to further transform the attention outputs.

2.4 Optimization

During the training phase of the transformer model, it's essential to tune its hyperparameters to enhance its predictive performance. This process aims to strike a balance between learning patterns from the training data and generalising well to unseen data, thereby avoiding problems like overfitting where model memories training data too closely or underfitting (where it fails to learn meaningful patterns).

Grid Search:

This method involves exhaustively searching through a manually specified subset of hyperparameters. While systematic, it becomes computationally expensive as the number of hyperparameters increases, making it impractical for large models. It can be computationally expensive, especially with many parameters or large datasets[36].

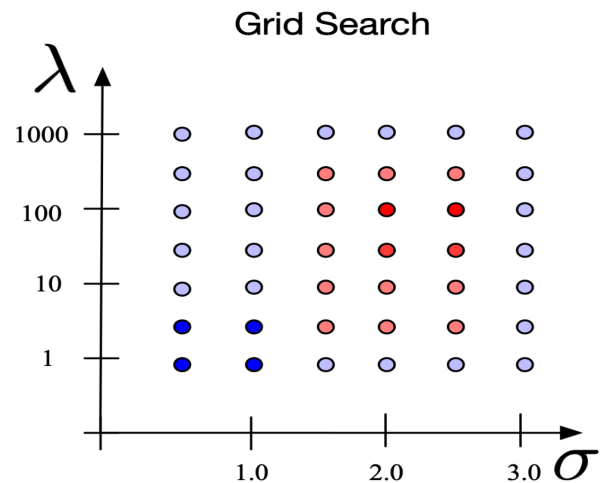


Figure 2.9 Grid Search

Random Search:

Instead of evaluating all possible combinations, random search selects random combinations of hyperparameters to train the model. This approach can be more efficient than grid search but may still require substantial computational resources without guaranteeing optimal results. It misses optimal values; still inefficient for high-dimensional search spaces[36].

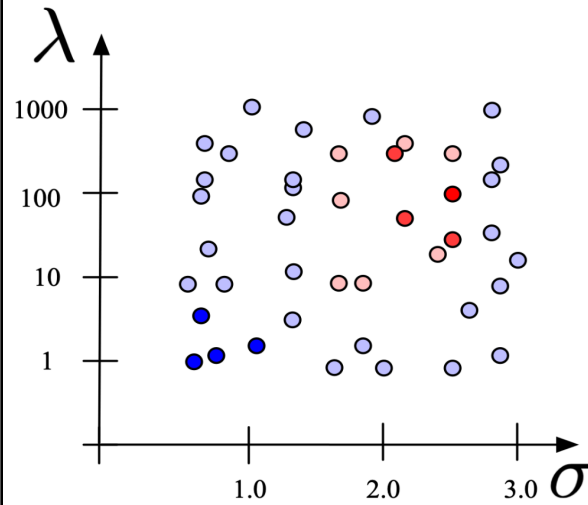


Figure 2.10 Random Search

Genetic Algorithms:

These algorithms use mechanisms inspired by biological evolution, such as mutation and selection, to evolve hyperparameter configurations over successive generations. They can effectively explore large search spaces but may converge slowly and require careful tuning of their own parameters[36].

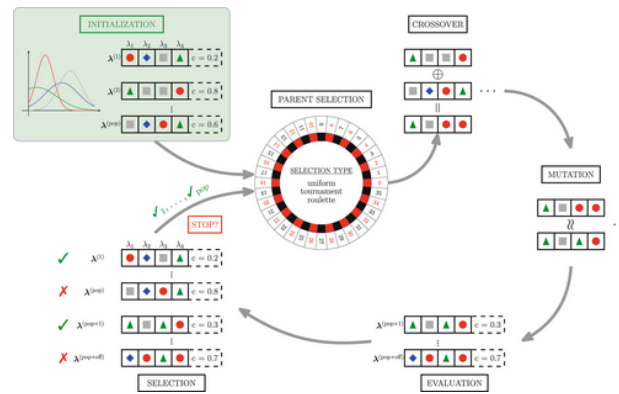


Figure 2.11 Genetic Algorithm

Bayesian Optimization:

This probabilistic model-based approach builds a surrogate model to approximate the objective function and uses it to select promising hyperparameter configurations. While more sample-efficient, it can be complex to implement and may struggle with high-dimensional spaces[36].

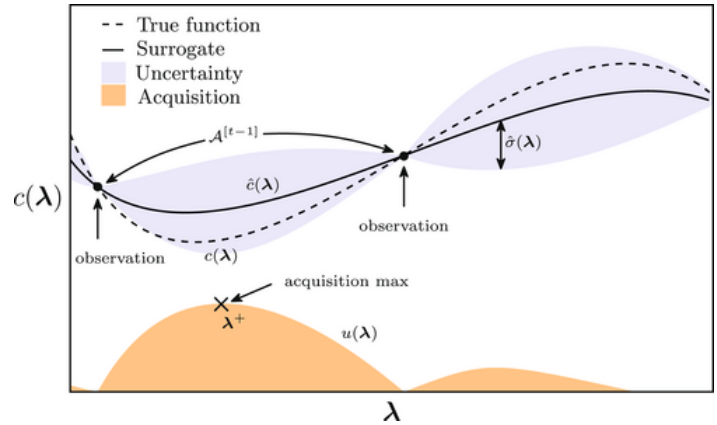


Figure 2.12 Bayesian Optimization

Gradient-based

Methods such as Hypergradient Descent, use backpropagation to compute gradients of validation loss with respect to hyperparameters and adjust them accordingly, offering faster convergence in differentiable settings [37].

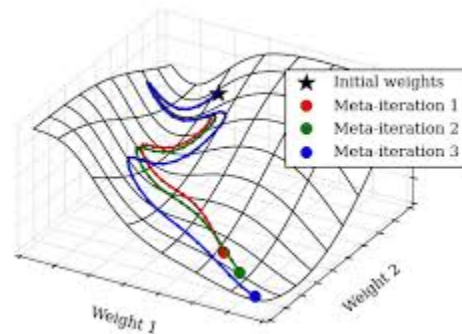


Figure 2.13 Gradient-Based

2.4.1 Hyperparameters

Hyperparameters play a crucial role in determining how well a neural network learns and generalizes from data. They directly influence the model's complexity, learning pace, and computational efficiency. In this section, we describe key hyperparameters that were considered and tuned in this project, categorized into transformer-specific and general network settings.

Hyperparameter	Description
Number of Attention Heads	Determines how many parallel attention computations are performed. Each head processes different projections of the input to learn distinct contextual relationships. This allows the model to focus on various parts of the input simultaneously.
Number of Encoder Layers	Specifies how many encoder blocks are stacked in the transformer. More layers increase model depth and capacity but also add to training time and memory usage.
Dimension of encoder Layers	Defines the size of the hidden representations in each encoder layer. Larger dimensions lead to richer feature learning but increase the number of trainable parameters and computational cost[38].

Table 2.4 Transformer-Specific Hyperparameters

General Hyperparameters

Beyond the transformer structure, there are several general hyperparameters that influence training behavior and performance.

The optimizer type dictates how the model updates its parameters. In this project, two common optimizers were explored:

- Adam (Adaptive Moment Estimation) combines the benefits of momentum and adaptive learning rates. It tracks the average of both the gradients and their squares, allowing it to adapt the learning rate for each parameter individually. Adam is well-suited for problems with noisy gradients and sparse features and is computationally efficient due to its low memory requirements [39].
- Stochastic Gradient Descent (SGD) updates model parameters based on small subsets (batches) of the data, introducing randomness into the learning process. This often helps the model escape local minima and is particularly efficient when dealing with large datasets [40].

The learning rate controls how quickly the model updates its weights in response to the error signal. A higher learning rate speeds up learning but may overshoot optimal solutions, while a lower rate results in slower, more stable convergence.

Batch size determines how many training examples are used in one update step. Smaller batches introduce more noise during training, which can improve generalization but often require longer training times.

Additionally, the number and size of linear layers in the network influence its expressive power. More layers and larger dimensions increase the model's ability to capture complex patterns but also raise the risk of overfitting and require more computational resources.

2.4.2 Genetic Algorithms

The genetic algorithm begins with the initialization of a random population, representing a diverse set of candidate solutions. Each individual in this population is defined by a set of genes, with each gene assigned a chromosome value. Collectively, these genes and their possible chromosome values determine the search space that the genetic algorithm explores in pursuit of an optimal solution.

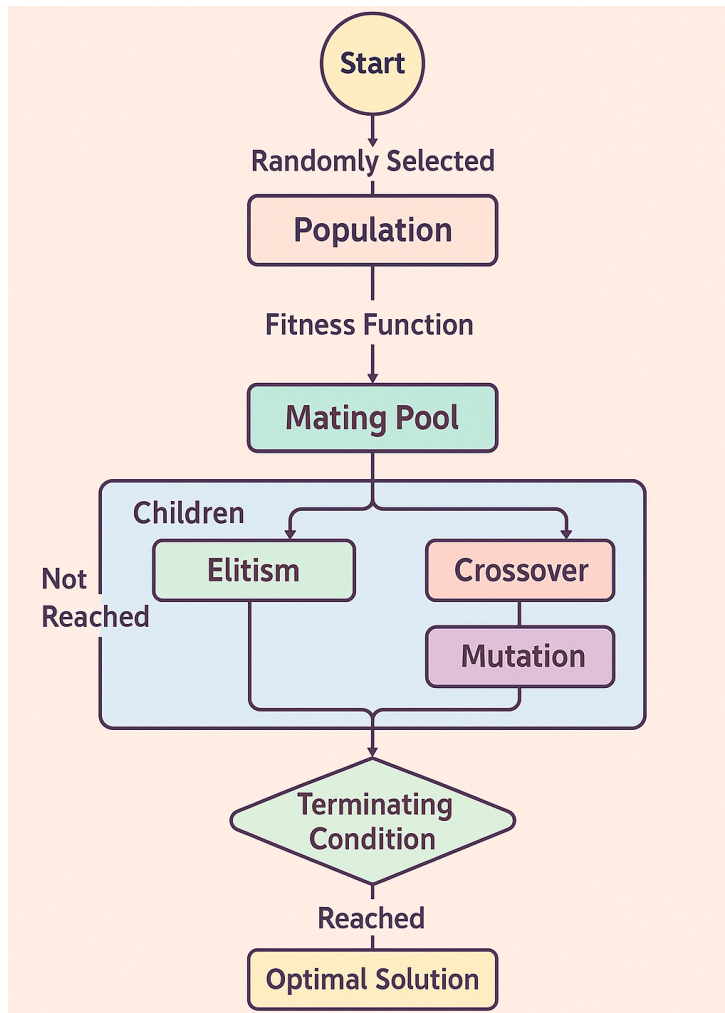


Figure 2.14 Genetic Algorithm Pipeline

Elitism

To preserve the best-performing individuals across generations, the genetic algorithm incorporates elitism. In each iteration, a subset of individuals with the highest fitness scores is carried over directly

into the next generation without undergoing crossover or mutation. This ensures that optimal or near-optimal solutions already discovered are not lost. These elite individuals can also participate in reproduction, acting as parents in the crossover process.

Crossover

Crossover is a genetic operation that simulates biological mating. During this process, two parent individuals exchange segments of their genetic information at one or more crossover points, resulting in child individuals that are a combination of both parents. This introduces variation and allows the algorithm to explore new areas of the search space [30]. The crossover point can be fixed or determined randomly.

Mutation

After crossover, the resulting child individuals undergo mutation, where one or more genes are randomly altered. This randomness introduces additional diversity into the population and helps prevent the algorithm from getting stuck in local optima. Mutation ensures that the search continues to explore new possibilities by slightly modifying existing solutions.

Through repeated cycles of selection, crossover, mutation, and elitism, the genetic algorithm progressively refines the population toward optimal solutions, within the defined problem constraints[41].

2.5 Evaluation Metrics

To evaluate the predictive performance of the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN) architecture, the following metrics were used:

2.5.1 Coefficient of Determination (R^2)

The coefficient of determination, or R^2 , is a statistical metric used in regression to measure the proportion of variance in the dependent variable that can be explained by the independent variables. An R^2 value of 1 indicates perfect prediction, while a value of 0 suggests the model fails to capture any variability. Values can also be negative, indicating worse-than-baseline performance [31].

The formula for R^2 is:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - f(X_i))^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

where:

- y_i : actual value
- $f(X_i)$: predicted value
- $\bar{y}$: mean of actual values
- N : number of samples
- RSS: residual sum of squares
- TSS: total sum of squares

R^2 is easy to interpret and useful for comparing models, but it does not always reflect predictive performance [32].

2.5.2 Root Mean Squared Error (RMSE)

The Root Mean Squared Error (RMSE) is a commonly used metric that quantifies the difference between predicted and actual values. A lower RMSE indicates better predictive accuracy, as it reflects how closely predictions align with actual outcomes[41].

The formula for RMSE is:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (f(X_i) - y_i)^2}$$

where:

- y_i : actual value
- $f(X_i)$: predicted value
- N : number of data points

2.6 Fuzzy Inference Systems

A Fuzzy Inference System is a decision-making framework based on fuzzy set theory, used to map inputs (features) to outputs (predictions) through a structured set of logical rules.[42]. Unlike traditional models that assume precision and deterministic relationships, FIS handles imprecision and uncertainty, which are common in real-world systems. This makes it especially valuable for complex environments like financial markets where perfect data is rarely available and patterns can be ambiguous[43].

In conventional models, crisp inputs and outputs are expected to represent real-world conditions exactly. However, this is often unrealistic due to natural variability and incomplete observations. A FIS addresses the issue by using fuzzy sets, where each input is associated with a degree of membership, allowing the model to reason in a more human-like, interpretable way [44]. In this project, the IFDTNN incorporates fuzzy memberships to generate interpretable rules that explain model predictions.

2.6.1 Fuzzy Logic

Fuzzy logic extends classical binary logic by allowing truth values to range continuously between 0 and 1. This enables the system to handle ambiguous or partially true statements in a natural way[45]. For example, instead of saying “the stock is volatile” (true or false), fuzzy logic allows us to say “the stock is somewhat volatile” with a degree of truth, enhancing the model’s flexibility in capturing real-world nuances.

2.6.2 Fuzzy Set Theory

Fuzzy sets generalize classical sets by allowing partial membership. A fuzzy set A in universe X is defined as

$$A = \{(x, \mu_A(x)) \mid x \in X\}$$

where

- x : an element in the universe
- $\mu_A(x)$: the membership function, representing the degree to which x belongs to set A

In this project, the Gaussian membership function is used:

$$\mu_A(x) = a \cdot e^{-\frac{(x-c)^2}{2\sigma^2}}$$

where

- a : peak height (set to 1)
- c : mean of the fuzzy set
- σ : standard deviation

This formulation captures the smooth transitions between membership levels making it suitable for modeling stock market variability.

2.6.3 Fuzzy Rules

Fuzzy rules are expressed using IF_THEN statements that connect fuzzy input conditions to fuzzy outputs. Each rule has:

1. An antecedent (e.g., “IF volatility is high”), and
2. A consequent (e.g., “THEN risk is high”).

Rule are evaluated using fuzzy logic operators:

- AND: $\min(\mu_A(x), \mu_B(x))$
- OR: $\max(\mu_A(x), \mu_B(x))$
- NOT: $1 - \mu_A(x)$

These rules are evaluated in parallel, and their outputs are combined through aggregation before final output generation.

2.6.4 Types of Fuzzy Systems

Two main types of Fuzzy Inference System (FIS) are commonly used:

- Mamdani-Type FIS: Outputs are fuzzy sets that must be defuzzified to yield crisp values. This type is more interpretable and preferred when human-readable rules sets are important[46].

- Takagi-Sugeno-Kang (TSK) FIS: Outputs are crisp values, generated directly using functions like $z = ax + by + c$. TSK is more computationally efficient and suitable for mathematical analysis. [39]

In this project, the Mamdani-type FIS is adopted to explain predictions made by the black-box neural network. This enhances transparency and trust, especially in financial modeling where stakeholders need to understand the rationale behind decisions.

2.6.5 Fuzzy Inference Process

The Mamdani-type fuzzy inference process consists of five key steps[46]:

1) Fuzzification

Eac crisp input is converted into a degree of membership using the defined membership functions.

2) Apply Fuzzy Operators

The antecedents of the fuzzy rules are evaluated using operators like AND, OR, and NOT (see Table 2.5).

Operation	Effect
Subset	$\mu A(x) \leq \mu B(x)$
Complement	$1 - \mu A(x)$
Disjunction (OR)	$\max(\mu A(x), \mu B(x))$
Conjunction (AND)	$\min(\mu A(x), \mu B(x))$

Table 2.5 Fuzzy Rules

3) Implication

Each rule's truth degree is used to shape the output fuzzy set, determining how strongly it contributes to the outcome[47].

4) Aggregation

Outputs from all rules are combined into a single fuzzy set, typically using the max operator.

5) Defuzzification

The aggregated fuzzy output is converted into a crisp value.

Method	Description
Maximum-Membership	Takes the value with the highest membership [48]
Center of Gravity	Computes the centroid of the area under the membership function
Weighted Average	Averages fuzzy set centroids weighted by membership values
Mean of Max Membership	Averages the range of values with maximum membership

Table 2.6 Defuzzification Process

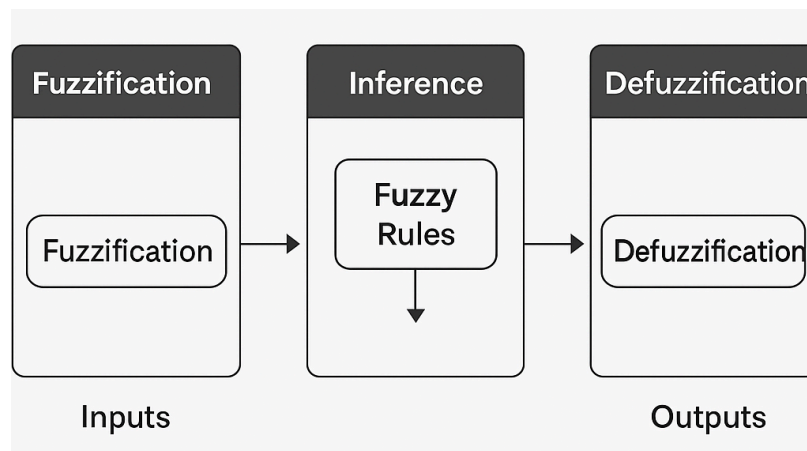


Figure 2.15 Fuzzy inference process in a Mamdani-type system.

In this project, the Weighted Average Method is used for defuzzification, as it better captures volatile and dynamic changes in stock prices, providing more accurate and interpretable results.

2.7 Reinforcement Learning Algorithms

Reinforcement Learning (RL) is a branch of machine learning that focuses on training an agent to make a series of decisions by interacting with an environment to achieve optimal outcomes over time. Unlike supervised learning, where the model is trained on labeled data, RL agents learn through trial and error, guided by rewards or penalties based on the actions they take. The primary goal of reinforcement learning is to discover a policy that maximizes the cumulative reward in the long run [49]

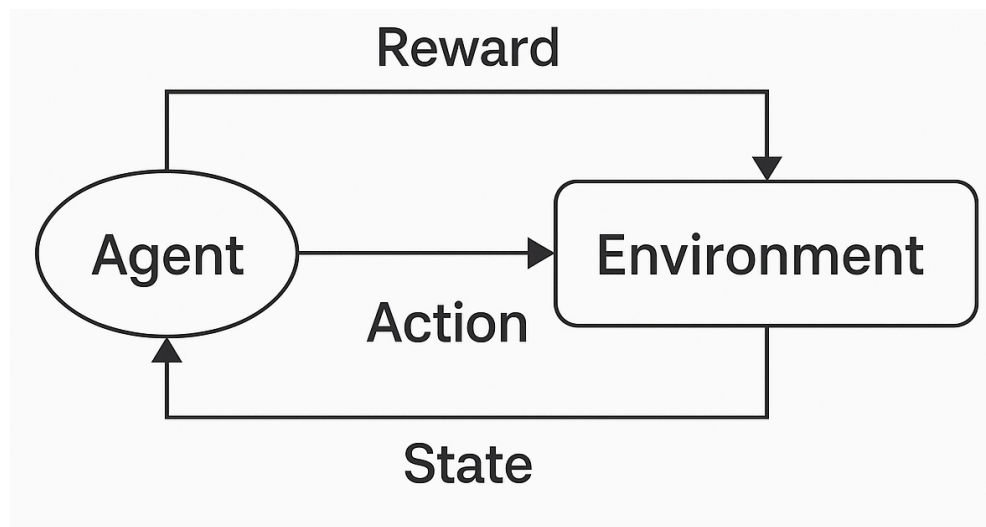


Figure 2.16 Reinforcement learning framework

This approach is inspired by how humans and animals learn from experience by exploring their surroundings, taking actions, and receiving feedback. The agent continuously updates its behavior based on this feedback loop, gradually improving its decision-making strategy [50].

In this project, seven agents are evaluated, and the one achieving the highest returns is selected.

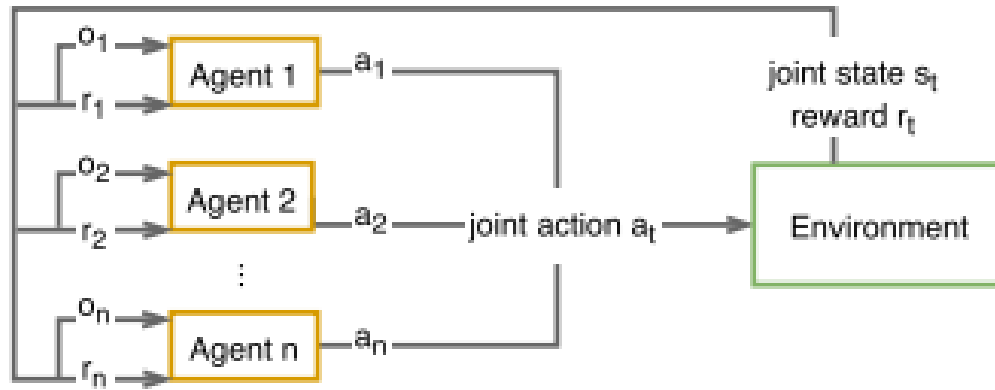


Figure 2.17 Multi-Agent Reinforcement learning framework [50]

Agent	Description	Advantages	Methods/Architectures	Limitations	Works best with
Deep Evolution Strategy [51]	Random mutation and selection of parameter vectors.	Gradient-free, highly parallelizable	Population-based search	Sample inefficient; requires large populations	Long Term investing with smooth trends
Actor-Critic [52]	Simulations policy and value learning with recurrent layers.	Captures time dependencies, more stable updates.	LSTM, Temporal difference (TD)-learning	Reward-sensitive; slower convergence	High volatility or cyclic stocks
Curiosity-Driven Q-Learning [53]	Uses intrinsic rewards to	Encourages exploration in	Intrinsic Curiosity	Hard to tune intrinsic	Newly listed or emerging

	explore unseen state-action pairs	sparse-reward environments	Module	reward; may over-explore	stocks
Proximal Policy Gradient [54]	Trust-region-like update for stable policy learning	Sample efficient, stable policy gradients	Clipped surrogate objective	Sensitive to hyperparameters	Mid-frequency trading
Evolution Strategy	Uses fitness-based evaluation without explicit rewards	Simple gradient-free	Mutation + Selection	Low learning speed; noisy evaluation	Low-frequency stocks with consistent movement
Double-Dueling Recurrent Q-Learning [55]	Combines double Q-learning and dueling architecture with memory	Accurate value estimation, better stability	LSTM, dueling heads, experience replay	Computationally heavy	Blue-chip stocks ETFs
Deep Deterministic Policy Gradient [56]	Continuous action actor-critic method using target networks and replay	Suitable for fine-grained control	Actor-Critic + Ornstein-Uhlenbeck Noise	Complex to tune; sensitive to noise	Stocks with small price moves and tight spreads

Table 2.7 Displays Taxonomy of Agents

2.7.1 Deep Evolution Strategy Agent

The Deep Evolution Strategy (DES) agent is a population-based optimization algorithm inspired by natural selection. It does not rely on gradients, which makes it well-suited for noisy, non-differentiable, or complex search spaces like stock trading environments.

The key idea behind DES is to maintain a population of model parameters(weights), perturb them with noise sampled from Gaussian distribution, evaluate their performance(reward), and then update the base model parameters based on the weighted average of those perturbations. The update rule is as follows:

Update Equation:

$$\theta_{\text{new}} = \theta + (\alpha / (n * \sigma)) \sum R_i * \epsilon_i$$

where:

$\epsilon_i \sim N(0, I)$: noise sampled from a standard normal distribution

R_i : reward achieved with perturbed parameters $\theta + \sigma \epsilon_i$

α : learning rate

σ : noise standard deviation

n : number of samples in the population

By evaluating many perturbations in parallel, the DES agent explores the parameter space and avoids getting stuck in local minima. The stochastic, global search mechanism is powerful in financial environments where rewards signals can be sparse or deceptive.

2.7.2 Actor-Critic Agent

The Actor-Critic (A2C) architecture blends the strengths of value-based and policy-based reinforcement learning. It uses two neural networks:

- **Actor**: Proposes actions given the current state, based on a policy. Parameterized policy $\pi(a|s; \theta)$
- **Critic**: Evaluates the value of being in a state, using a value function. Value function $V(s; w)$

This agent incorporates a LSTM network and Temporal difference (TD)- learning to model the sequential dependencies inherent in time series data like stock prices[57].

Temporal Difference Learning:

$$V(s_t) \leftarrow V(s_t) + \alpha (r + \gamma V(s_{t+1}) - V(s_t))$$

Loss Functions:

- Actor Loss: $L_{actor} = -\log \pi(a_t | s_t; \theta) A_t$
- Critic Loss: $L_{critic} = (r + \gamma V(s'; w) - V(s; w))^2$

where

$$A_t = r + \gamma V(s_{t+1}) - V(s_t)$$

Update Rules:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} L_{actor}$$

$$w \leftarrow w - \beta \nabla_w L_{critic}$$

This dual-structure enables the agent to continuously improve its decision-making and evaluation mechanisms, enhancing trading performance over time[58].

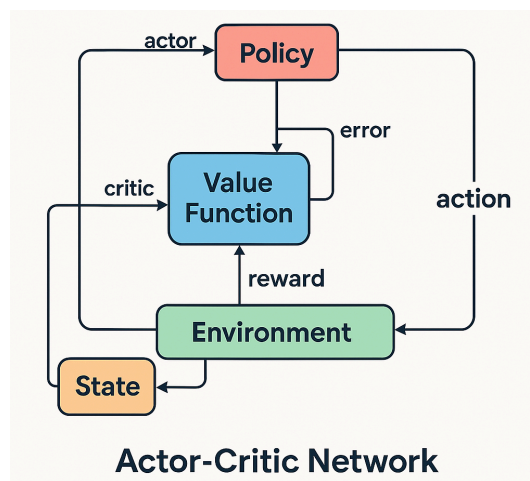


Figure 2.18 Actor Critic Network

2.7.3 Recurrent Curiosity Q-Learning Agent

This agent extends the classic Q-Learning framework by introducing intrinsic motivation through curiosity. It encourages exploration in complex environments by rewarding the agent not only for external success but also for encountering surprising or novel states.

Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s,a)] \quad [59]$$

Curiosity Reward Function:

The intrinsic reward at time t can also be written as:

$$R_t^{\text{int}} = \left\| f(s_{t+1}) - \hat{f}(s_{t+1}) \right\|^2$$

where

$f(s_{t+1})$:actual features of the next state

$\hat{f}(s_{t+1})$:predicted features by the forward dynamics model

Total Reward Function:

The agent's total reward function combines extrinsic and intrinsic components:

$$R_t = R_t^{\text{ext}} + \eta R_t^{\text{int}}$$

where

R_t^{ext} : extrinsic reward from the environment

R_t^{int} : intrinsic curiosity-based reward

η : scaling factor to balance intrinsic vs. extrinsic motivation

The agent uses a forward dynamics model to predict future state embeddings. The curiosity reward is high when the prediction error is large indicating novel or unexplored dynamics. This promotes robust learning in noisy or sparse-reward domains like financial markets[60].

2.7.4 Proximal Policy Gradient Agent

This agent directly optimizes the policy to maximize expected cumulative reward using the policy gradient theorem.

Objective Function:

The goal is to maximize the expected return under a parameterized policy π_θ :

$$J(\theta) = \mathbb{E}_{\pi_\theta}[R]$$

where:

θ : parameters of the policy network

R : cumulative reward (return)

Policy Gradient Theorem :

The gradient of the expected return with respect to the policy parameters is given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) \cdot R]$$

This method bypasses the need for value estimation, directly adjusting the policy parameters θ through stochastic gradient ascent. Although this approach can suffer from high variance, it is simple and effective for learning directly from experience[61].

2.7.5 Evolution Strategy Agent

The Evolution strategy utilizes black-box optimization where the agent's parameters are evolved over time. Random noise is added to weights, and each perturbed model is evaluated on its ability to maximize reward.

Gradient Estimate:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{n\sigma} \sum_{i=1}^n R_i \cdot \epsilon_i$$

where:

$\theta \in R^d$: the current policy parameters

$\epsilon_i \sim N(0, I)$: Gaussian noise for perturbation

σ : standard deviation of the noise

n : number of perturbations

The parameters are updated as:

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$$

where:

α : learning rate

The weights are updated based on a weighted combination of successful perturbations. This method is robust to non-stationary, noisy environments and does not require gradient backpropagation[62].

2.7.6 Double Dueling Recurrent Q-Learning Agent

This agent architecture integrates Double Q-Learning, Dueling Network Architecture, and a Recurrent LSTM Module to enhance stability, reduce overestimation, and improve performance on sequential data such as financial time series.

Double Q-Learning: Double Q-Learning addresses the overestimation bias of standard Q-learning by decoupling action selection from action evaluation. Two networks are maintained:

$$Q(s, a) = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta), \theta')$$

where:

$Q(s', a'; \theta)$: selection of the best action $\arg \max_{a'}$

$Q(s', a'; \theta')$: evaluating its value

Dueling Network Architecture:

Decomposes Q-value into state value and advantage:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$$

where:

$V(s)$: State value

$A(s, a)$: Advantage function

This allows the network to better estimate the value of being in a state, regardless of the action[63].

Recurrent LSTM Module:

An LSTM layer is integrated into the Q-network to capture temporal dependencies and patterns across time steps, which is crucial for sequential environments like stock trading or robotic control[64].

The recurrent structure enables the agent to learn from:

- Past states and actions
- Long-term trends
- Delayed rewards

The network predicts the value of the current state and the advantage of each action, allowing more accurate and stable Q-value estimates. The addition of LSTM further empowers the agent to remember patterns in past price movements.

2.7.7 Deep Deterministic Policy Gradient Agent

The Deep Deterministic Policy Gradient (DDPG) algorithm is a powerful actor-critic, model-free, off-policy method designed to handle continuous action spaces. This makes it especially suitable for applications like financial trading, where actions (e.g., trade volume or proportion of capital allocation) are continuous rather than discrete[65].

Architecture:

1) Deterministic Policy:

The actor network $\mu(s; \theta_\mu)$ maps each state s to a specific action a deterministically (no action distribution like in stochastic policy gradient methods).

2) Critic Network:

The critic network $Q(s, a; \theta_Q)$ evaluates how good the action a is in state s .

3) Target Networks:

Two additional target networks μ' and Q' , with delayed updates, are used to stabilize learning.

4) Experience Replay:

Past experiences (s, a, r, s') are stored in a buffer and sampled randomly during training to break temporal correlations and improve data efficiency.

5) Exploration Noise:

Since the policy is deterministic, exploration noise (e.g., from the Ornstein-Uhlenbeck process) is added to the actor's actions during training to encourage exploration.

Critic Update (Q-value estimation):

Minimize the Mean Squared Bellman Error (MSBE):

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - Q(s_i, a_i | \theta^Q))^2$$

where:

$y_i = r_i + \gamma Q'(s_i + 1, \mu'(s_i + 1) | \theta^Q)$: the target Q-value

$\mu'(s_i+1)$: action predicted by the target actor

$Q'(s_i+1, \cdot)$: Q-value from the target critic

Actor Update:

Update actor parameters using the policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_a Q(s, a | \theta^Q) \big|_{a=\mu(s)} \nabla_{\theta^\mu} \mu(s | \theta^\mu)$$

where:

N : number of samples in the mini-batch

$Q(s, a | \theta^Q)$: critic network

$\mu(s | \theta^\mu)$: actor network

$\nabla_a Q$: how much the Q-value changes with respect to action

$\nabla_{\theta^\mu} \mu$: how much the action changes with respect to actor parameters

This chain rule allows the actor to learn how to adjust that lead to higher predicted rewards from the critic.

Chapter 3

The Stock Market and Algorithmic Finance

Chapter 3 outlines the fundamentals of stock market dynamics, trading principles, and key financial indicators. It sets the foundation for understanding how algorithmic strategies interact with real-world financial data.

3.1 Stock Market

The stock market serves as a dynamic ecosystem where investors, institutions, and traders exchange ownership in companies through shares. It functions not only as a platform for capital raising but also as a barometer of economic performance, investor sentiment, and corporate health.

In this project, the focus is on Singapore-listed stocks, specifically those on the Singapore Exchange (SGX) a well-regarded and highly liquid market in Southeast Asia. Understanding the structure and behavior of the SGX is essential before applying algorithmic models to analyze and optimize portfolio strategies[66].

In financial modeling, several key metrics are commonly used to evaluate risk and return. Alpha (α) [49] reflects how well an investment strategy performs relative to the market, serving as an indicator of excess return. Conversely, Beta (β) measures systematic risk, representing how sensitive an asset is to market movements. Momentum captures the likelihood that a price trend will persist, while systematic risk refers to market-wide, unavoidable risk. On the other hand, unsystematic risk is specific to individual companies or sectors and can be minimized through diversification. Finally, volatility quantifies the degree of variation in asset returns and is often used to assess risk[67].

3.1.1 Overview of the Singapore Exchange (SGX)

The Singapore Exchange (SGX) is the main securities exchange in Singapore, offering a broad range of equities, bonds, REITs, ETFs, and derivatives. It is known for its:

Market Stability: SGX operates in a highly regulated and transparent financial environment.

Sector Diversity: Listed companies span across finance (e.g., UOB, DBS), real estate (e.g.,

CapitaLand, Ascendas REIT), infrastructure, healthcare (e.g., IHH Healthcare), industrials (e.g., Keppel DC REIT), and technology.

Investor Accessibility: SGX is open to both institutional and retail investors, with growing adoption of algorithmic trading tools.

3.1.2 Market Trends in the Singapore Stock Market

A market trend refers to the general direction of an asset's price over a period, often analyzed through trendlines, price action, and technical indicators such as Moving Average Convergence Divergence (MACD) or Relative Strength Index (RSI) [68]. In the Singapore Exchange (SGX), these tools are widely used by retail and institutional investors to assess stock movements.

An uptrend occurs when price movements form a series of higher highs and higher lows, typically reflecting bullish sentiment among investors. Conversely, a downtrend is identified by lower highs and lower lows, indicating bearish sentiment. Recognizing these trends is essential for developing momentum trading strategies that aim to capitalize on continued price movements [69].

While technical indicators are effective in identifying current trends, they are inherently lagging because they rely on historical price data [70]. Thus, they may not account for forward-looking events or real-time information asymmetries.

In this project, we enhance lagging indicators by integrating forecasted price values derived from machine learning models. This hybrid approach seeks to detect trend reversals early, improving the timing of buy/sell signals in a momentum-based trading strategy.

3.1.3 Financial Portfolio in Singapore Context

In Singapore, portfolios often include government instruments like Singapore Savings Bonds (SSBs) or Singapore Government Securities (SGS), along with equities listed on SGX such as DBS, OCBC, or CapitaLand.

The primary purpose of a portfolio is diversification to reduce unsystematic risk, which is specific to individual companies or sectors and can be mitigated through varied holdings. The principle of "not putting all your eggs in one basket" underpins modern portfolio construction, even in CPF Investment Schemes.

Rebalancing is a key practice in managing risk and maintaining target allocations. Whether done periodically or tactically based on market conditions, rebalancing helps align a portfolio with the investor's risk profile and objectives [72]. This project only evaluates cyclic portfolios.

3.1.4 Company Fundamentals

Indicator	Definition & Formula	Uses
Earnings per Share (EPS) [73]	A company's net profit divided by the number of common shares it has outstanding; it indicates how much money a company makes for each share of its stock and is a widely used metric for estimating corporate value. $\text{EPS} = \frac{\text{Net Income} - \text{Preferred Dividends}}{\text{End-of-Period Common Shares Outstanding}}$	High P/E may signal growth expectations or overvaluation. Useful when comparing firms in the same industry.
Price-to-Earnings (P/E) Ratio [74]	A stock valuation metric that compares a company's share price to its earnings per share (EPS). $\text{P/E Ratio} = \frac{\text{Market value per share}}{\text{Earnings per share}}$	Useful when comparing firms in the same industry.
Return on Equity (ROE) [75]	A measure of financial performance calculated by dividing net income by shareholders' equity.	Compared to industry averages to detect performance issues.

	$ROE = \frac{\text{Net Income}}{\text{Shareholder's Equity}}$	
Debt-to-equity [76]	A ratio that is used to evaluate a company's financial leverage and is calculated by dividing a company's total liabilities by its shareholder equity. $\text{Debt/Equity} = \frac{\text{Total Liabilities}}{\text{Total Shareholders' Equity}}$	Assesses financial risk — a higher D/E ratio suggests greater reliance on borrowed funds, which may increase default risk.
Market Capitalization [77]	Market capitalisation, or market cap, represents a company's total market value, calculated by multiplying its share price by the number of outstanding shares. It reflects the company's size and overall valuation. $\text{Market Capitalisation} = \text{Price per share} * \text{Number of of shares outstanding}$	Firms are categorized into large-cap, mid-cap, and small-cap, each with different risk-return profiles.

Table 3.1 Key Financial Indicators

3.2 Trend Indicators

For stocks, it is necessary to judge the stability of the company through its fundamentals to assess the financial status of firms. This is especially true for stocks in dividend portfolios given that the objective is to receive dividend handouts rather than to leverage on the changes of the price in stock.

Indicator	Type	Formula
Simple Moving Average (SMA)	Trend (Lagging)	Smooths price data to identify trend direction. Calculates average price over a fixed period to filter out short-term fluctuations. $SMA_t = \frac{1}{n} \sum_{i=0}^{n-1} Close_{t-i}$
Exponential Moving Average (EMA)[78]	Trend (Lagging, weighted)	Identifies recent price movements with more sensitivity than SMA. Gives more weight to recent prices to react faster to trend changes. $EMA_t = \alpha \cdot Close_t + (1 - \alpha) \cdot EMA_{t-1}, \text{ where } \alpha = \frac{2}{n+1}$
Relative Moving Average (RMA)	Trend	Also known as Wilder's smoothing. Used in RSI. Smoother than EMA for stable signals. $RMA_t = \frac{(RMA_{t-1} \cdot (n-1) + Value_t)}{n}$
Moving Average Convergence Divergence (MACD)	Momentum & Trend	Measures the relationship between two EMAs to identify trend changes. Signal line crossovers and divergence are used for buy/sell signals. $MACD_t = EMA_{12} - EMA_{26},$ $Signal_t = EMA_9(MACD_t)$

Bollinger Bands	Volatility	Measures price volatility relative to a moving average. Expands/contracts based on market volatility; price crossing bands can signal overbought/oversold status. $\text{Mid}_t = \text{SMA}_{20},$ $\text{Upper}_t = \text{Mid}_t + 2 \cdot \sigma,$ $\text{Lower}_t = \text{Mid}_t - 2 \cdot \sigma$
Relative Strength Index (RSI) [79]	Momentum (Oscillator)	Identifies trend strength and overbought/oversold conditions. $\text{RSI} > 70$ = overbought, $\text{RSI} < 30$ = oversold; also used for divergence detection. $\text{RS}_t = \frac{\text{Average Gain}_n}{\text{Average Loss}_n},$ $\text{RSI}_t = 100 - \frac{100}{1 + \text{RS}_t}$
Standard Deviation	Volatility	Measures market volatility by assessing price dispersion. Higher values = more volatility; used alongside other indicators like Bollinger Bands.

Table 3.2 List of Trend Indicators

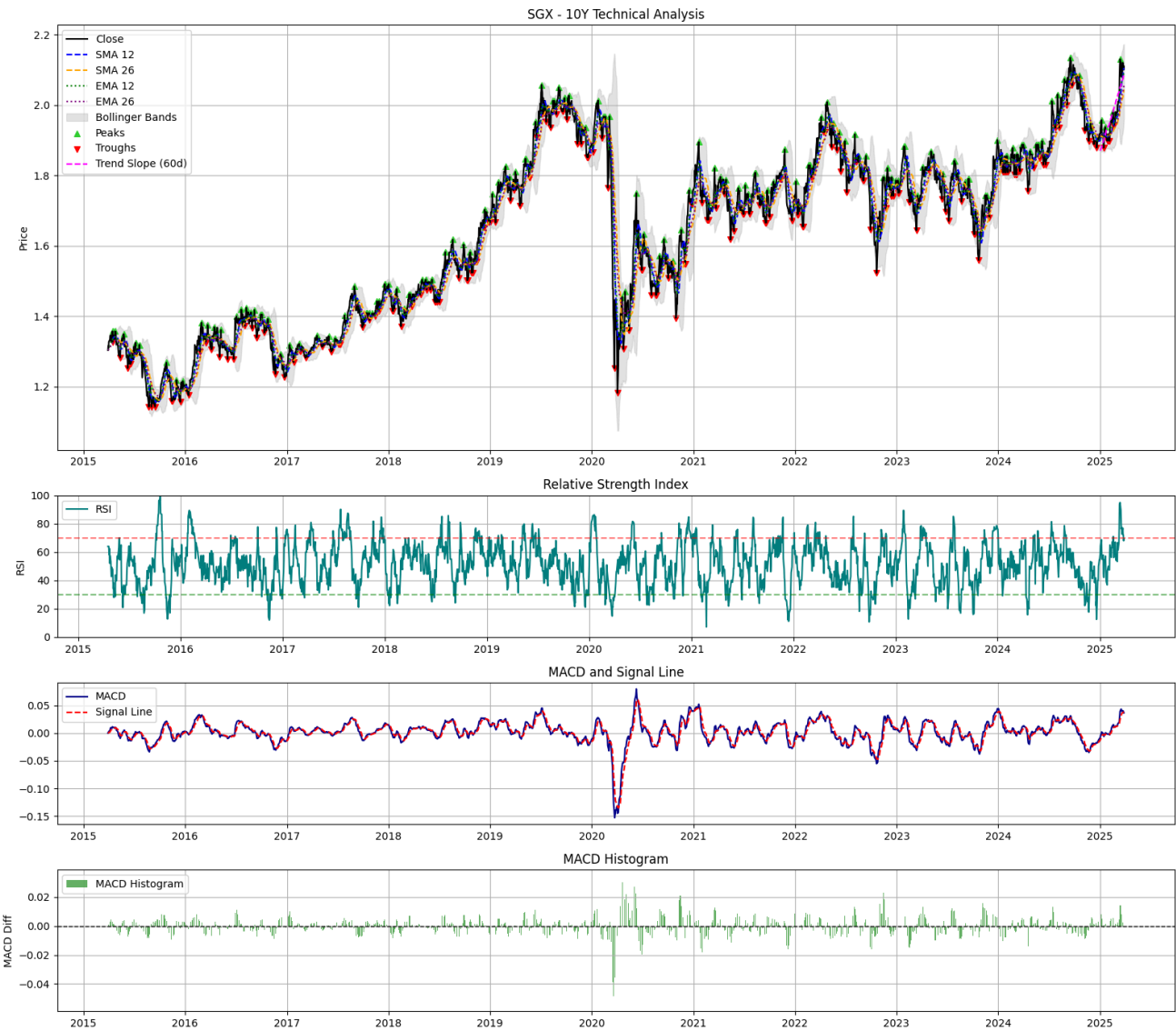


Figure 3.1 Comprehensive Technical Analysis of Capital and Integrated Commercial Trust (C38U.SI)

Chapter 4

Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN)

Chapter 4 introduces the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN), detailing its architecture, fuzzy rule integration, and how it enhances both prediction performance and interpretability.

4.1 IFDTNN Architecture

The Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN) architecture, as shown in *Figure 4.1*, consists of five layers. It begins by accepting crisp input values denoted as $X=[x_1, x_2, \dots, x_i]$. These inputs undergo fuzzification, converting them into fuzzy representations that are then fed into the transformer neural network and the fuzzy rule base layer.

The fuzzy inputs provided to the inference layer are denoted as c , where c_i , f_i represents the fuzzy input derived from the f_i -th cluster of input feature i . The fuzzy rule base, represented as $C=[c_1, c_2, \dots, c_l]$, maps these fuzzy inputs to corresponding fuzzy outputs in the consequent layer, denoted as h . Specifically, $h_{t,gt}$ indicates the fuzzy output associated with the g_t -th cluster of the output feature t . For this project, $t = 1$ since there is only a single output feature the closing price.

The fuzzy outputs are then defuzzified to produce final crisp outputs, represented as $Y=[y_1, y_2, \dots, y_l]$. IFDTNN uniquely combines a fuzzy inference system with a transformer-based deep learning model, aiming to enhance both prediction accuracy and model interpretability. This hybrid design promotes transparency, enabling each prediction to be traced through a set of fuzzy rules operating in parallel with the neural network layers.

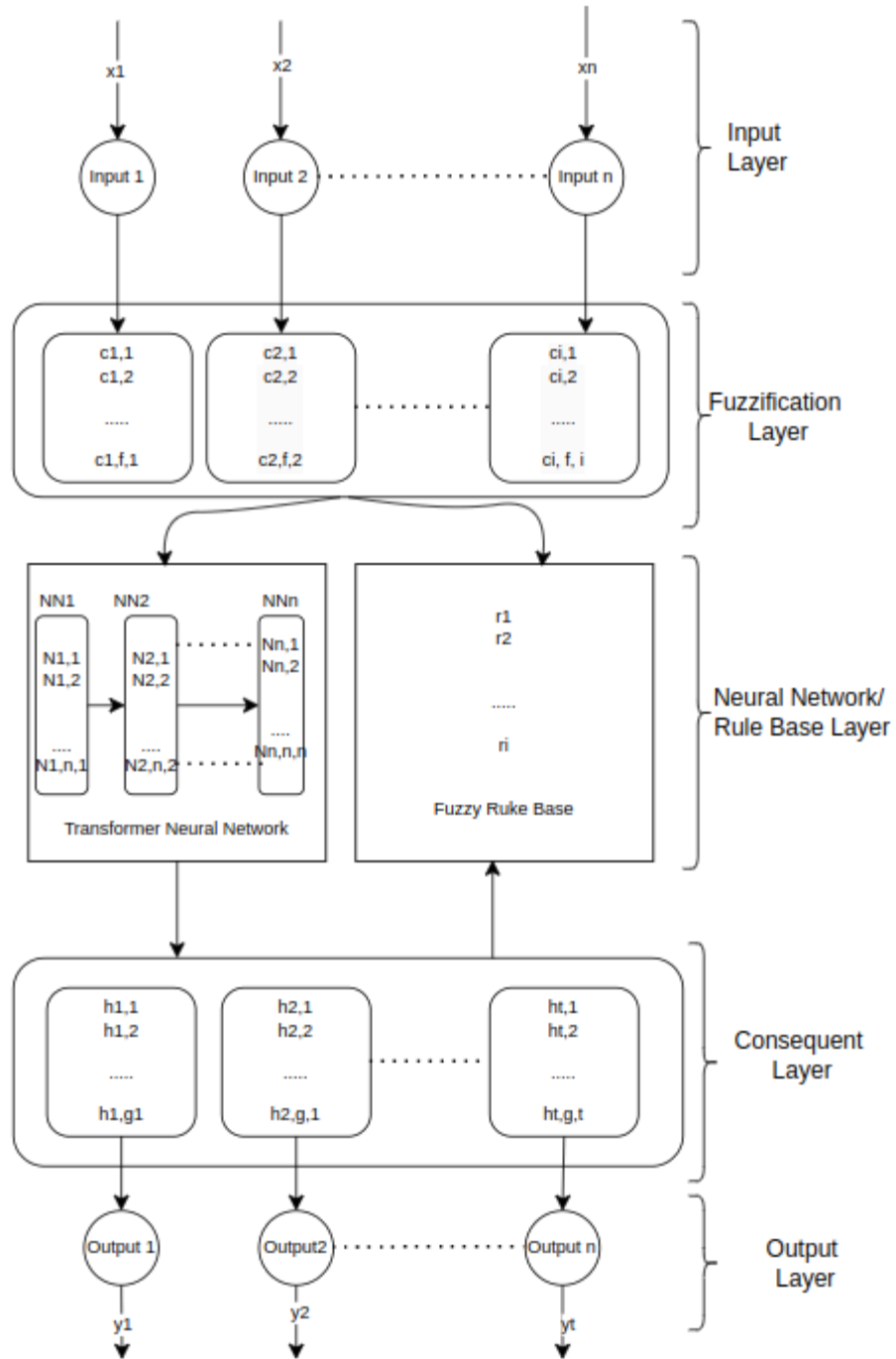


Figure 4.1 IFDTNN Architecture

4.1.1 Input Layer

The input layer receives the raw numerical values (crisp values) for each feature. Each input feature x_i is passed directly into the system without transformation. The relationship is represented as:

$$I_in(i) = x_i$$

$$I_out(i) = I_in(i)$$

where:

- $I_in(i)$ = crisp input value of feature x_i to the input layer
- $I_out(i)$ = crisp output value of feature x_i from the input layer
- x_i = original crisp input value of feature i

This layer simply forwards the input features to the next stage of processing.

4.1.2 Fuzzification Layer

The fuzzification layer transforms each crisp input value into a vector of fuzzy membership values. This is achieved using a Gaussian membership function based on the cluster statistics of each feature. For each feature, the fuzzified output indicates the degree to which the input belongs to a cluster:

$$C_i^{in} = I_i^{out}$$

$$c_{i,f} = \exp(-((C_i^{in} - \mu_{i,f})^2) / (2 * \sigma_{i,f}^2))$$

$$C_i^{out} = [c_{i,1}, c_{i,2}, \dots, c_{i,f-1}]$$

where:

- C_i^{in} = input to the fuzzification layer for feature i
- I_i^{out} = crisp output of the input layer for feature i
- $c_{i,f}$ = fuzzy membership value of feature i to cluster f

- $\mu_{i,f}$ = mean of cluster f for feature i
- $\sigma_{i,f}$ = standard deviation (variance) of cluster f for feature i
- $\exp$ = Euler's number ($e \approx 2.718$)
- C_i^{out} = vector of fuzzified outputs for feature i

where $\mu_{i,f}$ and $\sigma_{i,f}$ are the mean and standard deviation of the cluster f for feature i.

4.1.3 Neural Network/ Rule Base Layer

This layer consists of two parallel subcomponents:

1) Transformer Neural Network Path (N):

This path uses a deep transformer network to learn temporal patterns from the fuzzified input features. It receives the fuzzy membership values from Layer 2 and processes them through transformer encoders and fully connected layers. The output is a vector of predicted membership values for the output clusters.

$$N_in = [c_{1,1}, c_{1,2}, \dots, c_{i,f_i}]$$

$$N_out = [\hat{h}_{1,1}, \hat{h}_{1,2}, \dots, \hat{h}_{t,g_t}]$$

where:

- c_{i,f_i} = Fuzzified input value of feature i for cluster f_i
- $\hat{h}_{t,g_t}$ = Predicted fuzzy membership value for output feature t to cluster g_t

2) Fuzzy Rule Path (R):

This path builds interpretability by computing Hebbian weights based on the relationship between the fuzzified inputs (antecedents) and the predicted fuzzy outputs (consequent). The Mamdani-type fuzzy rules are derived to explain how inputs map to outputs using symbolic logic.

IF x_1 is $c_{1,a}$ AND x_2 is $c_{2,\beta}$ AND ... AND x_k is $c_{k,z}$

THEN h_1 is $h_{1,p}$ AND ... AND h_k is $h_{k,q}$

where:

- k = Rule number (k-th fuzzy rule)
- x_i = Input feature i
- $c_{i,p}$ = Winning cluster of input feature i in rule k
- $h_{i,q}$ = Predicted fuzzy output (consequent) for feature i in rule k
- p, q, a, b, z = Indexes of the winning clusters for input and output features

The neural network outputs predicted fuzzy membership values for output features. Concurrently, fuzzy rules map input clusters to output clusters based on co-activation patterns. The combined operation supports both data-driven learning and rule-based reasoning.

4.1.4 Consequent Layer

This layer receives the fuzzy predictions from the transformer network. These fuzzy values represent degrees of membership to possible output clusters and are passed on to the defuzzification layer.

$$H_{in} = N_{out}$$

$$H_{out} = H_{in}$$

where:

- H_{in} = input to the consequent layer (from transformer output)

- H_out = output from the consequent layer (passed to defuzzification)

4.1.5 Output Layer

The output layer converts fuzzy membership values into crisp outputs using a weighted average approach:

$$O_{in} = H_out$$

$$O_{out} = \text{sum}(\mu_i * \hat{h}_i) / \text{sum}(\hat{h}_i) \quad (\text{Equation 4.6})$$

where:

- O_{in} = input to the output layer for feature t
- O_{out} = final crisp output for feature t
- μ_i = mean (crisp) value of output feature i in cluster j
- $\hat{h}_i$ = fuzzy membership prediction of output feature i to cluster j

where $\hat{h}_i$ is the predicted fuzzy membership and μ_i is the corresponding cluster center.

4.2 Hebbian Learning Mechanism

Hebbian learning is used to establish and strengthen fuzzy rules based on co-activation of input and output clusters. The weight update is defined as:

$$w_i(t+1) = w_i(t) + x_i * y$$

This mechanism reinforces rules that are repeatedly activated during training, providing an interpretable explanation for the prediction logic.

4.3 Rule Firing Strength and Updating

For each instance, the dominant input and output clusters are identified. The rule that maps these clusters is updated or created with a firing strength based on the product of membership values. If the rule exists, its weight is increased; otherwise, it is added with the current strength.

4.3.1 Pseudo-weights of Fuzzy Rules (with Hebbian Learning)

In fuzzy neural systems like IFDTNN, simply having antecedents, consequents, and fuzzy rules is not sufficient. What we lack is an understanding of how strong each rule is. To capture this, we compute pseudo-weights that represent the strength or influence of each fuzzy rule. This is done using Hebbian Learning, a biologically inspired approach originally introduced in neuroscience.

"When two neurons fire together repeatedly, their connection becomes stronger."

This principle has been adapted to neural networks and fuzzy systems for enhancing interpretability and learning patterns over time. In the context of IFDTNN, we adopt a method inspired by the POPFNN model [20].

4.3.2 Identifying Dominant Clusters

For each data instance, the winning cluster for both input and output features is identified. The winning cluster is the one with the highest membership value.

Let the set of winning clusters for the input features be:

$$C = \{C_{1,a}, C_{2,b}, \dots, C_{n,z}\}$$

where:

$C_{i,j}$ = dominant cluster j for input feature i

Since we consider only one output feature in this case, the dominant cluster for the output is denoted as:

D = output winning cluster

Each fuzzy rule is defined in the form:

$$\{C \rightarrow D\}$$

where C is the antecedent (input cluster set) and D is the consequent (output cluster).

4.3.3 Pseudo-weight Calculation (Firing Strength)

To compute the strength (or weight) of a fuzzy rule, we calculate both the forward and backward firing strengths:

$$ffw = \min(\mu_{C_i}(x_i)) \quad \text{-- Forward firing strength (min of all input memberships)}$$

$$fbw = \mu_{D(y)} \quad \text{-- Backward firing strength (membership value of output)}$$

where:

$\mu_{C_i}(x_i)$ = membership value of input feature i in cluster C_i

$\mu_{D(y)}$ = membership value of output feature in cluster D

The final pseudo-weight (f_rule) for the rule is:

$$f_rule = ffw * fbw \quad \text{(Equation 4.7)}$$

Hebbian Update Rule

The Hebbian update for a rule is applied whenever a rule is used:

$$w_{i,j}(t+1) = w_{i,j}(t) + x_i * y_j \quad \text{(Equation 4.8)}$$

where:

$w_{i,j}(t)$ = Hebbian weight between input node x_i and output node y_j at time t

x_i = value of input node i

y_j = value of output node j

If the rule $\{C \rightarrow D\}$ already exists in the fuzzy rule base, its weight is updated using this equation. If it does not exist, a new rule is created and its initial weight is set using the calculated f_rule . By combining winning clusters, fuzzy memberships, and Hebbian updates, IFDTNN enhances rule interpretability and allows the neural network to learn which fuzzy rules are most important. These pseudo-weights form the foundation for building an explainable, evolving fuzzy rule base that complements the deep transformer network in decision-making.

[1] Ong, P., & Lim, C. (2021). POPFNN: A Neural-Fuzzy System with Pseudo Hebbian Learning for Rule Evolution. *IEEE Transactions on Fuzzy Systems*.

4.4 Memory Decay

To prevent weight explosion, a decay factor ($\lambda = 0.98$) is introduced. Rules that are not activated over time will have their weights reduced as follows:

$$w_{ij}(t+1) = w_{ij}(t) * \lambda$$

This emphasizes recent patterns and maintains system stability.

where,

w_{ij} = hebbian weight between input node x and output node y at a particular time instance

λ = decaying factor

t = current time instance of iteration

4.5 Tagging Mechanism for Interpretability

The tagging mechanism links fuzzy rules with specific transformer neurons. Each neuron's activation is tracked and associated with the rules it supports. This enables traceability from input features to fuzzy rules to neural outputs, offering valuable insight into how predictions are generated.

$$activated = \begin{cases} 1 & \text{if } \frac{\sum_{d=1}^D a_d^{NN_i}}{D} \geq threshold \\ 0 & \text{otherwise} \end{cases}$$

The tagging process is carried out at each layer of the transformer network up to the final output prediction layer. By identifying the activated nodes within the transformer and utilizing the rule base that maps inputs to outputs, a pair of paths—one through the neural network and one through the rule base—can be traced. This mapping connects fuzzy inputs to fuzzy outputs, offering interpretability and insights into the decision-making process of the otherwise black-box model.

Chapter 5

Experimental Results and Analysis

Chapter 5 presents the key parameters related to the IFDTNN architectures used in the project. All models and experiments were implemented using the Python programming language.

5.1 Dataset Segmentation and Usage

For model development and experimentation, daily market data was used, specifically the adjusted closing prices (to account for corporate actions like stock splits) and trading volume. The entire dataset is divided into three distinct sets for model training: 1) train dataset 2) validation dataset 3) test dataset.

Train Set (Initial Public Offering (IPO) listing date to 31 December 2018):

Used to train the transformer-based model to learn historical price patterns and forecast future values over a 13-day sliding window.

Validation Set(From 1 January 2018 to 31 December 2018)

Used for hyperparameter tuning and early stopping to enhance model generalization and prevent overfitting.

Test Set (From 1 January 2019 to 1 January 2024)

A 5-year window used to evaluate the model's predictive performance and apply the model in trend reversal detection and portfolio optimization tasks.

5.2 Objectives

The transformer-based model in this project is designed to anticipate stock price fluctuations over a 13-day forward-looking horizon. By generating future price projections, the model enhances the responsiveness of traditional technical indicators, which typically rely on historical data and suffer from inherent lag. This forward-looking capability supports more timely identification of potential trend reversals.

Incorporating these predicted values into technical indicator calculations enables earlier detection of market shifts. As a result, investors and analysts can respond proactively, making informed decisions and reallocating assets within a reasonable window before major price movements occur.

5.3 Model Approach

In this project, the multiple model approach was adopted to forecast each individual day within the 13-day look-ahead period. n separate models are used to predict the price value of each individual day $T+i$ within the n -day look-ahead period. This method was chosen as it allows each model to specialize in forecasting a specific time step, enabling more precise predictions. Feature selection and training are tailored for each i -th day. By isolating the prediction task for each day, we are able to tailor the feature selection and model training process accordingly, reducing prediction noise and avoiding crosstalk that may arise from a shared model attempting to predict multiple future values simultaneously.

5.4 Portfolio Selection

To minimize currency risks, this project considers the Singapore Dollar (SGD) as the base currency, and the portfolios constructed consist of securities listed on the Singapore Exchange(SGX).

5.4.1 Diversified Portfolio Strategy Using SGX-listed Stocks

The objective of this portfolio is to create a balanced investment strategy by combining cyclical stocks, which tend to outperform during economic expansions, with defensive stocks that offer stability during downturns. By leveraging market cycles, the portfolio aims to generate value through momentum investing, identifying equities showing early signs of upward trend reversals while gradually reallocating away from those signaling potential declines. This approach aligns with a market-timing philosophy, allowing investors to capture returns across different phases of the economic and market cycle while maintaining a diversified risk profile.

5.4.1.1 Fundamental Analysis

1. Market Capitalization

Range spans from large-cap (e.g., UOB Bank – \$63.65B) to mid-/small-cap (e.g., Keppel DC REIT – \$4.87B). It is diversified by size, giving a mix of stability and growth potential.

2. Valuation – P/E Ratio

UOB Bank and Keppel DC REIT have relatively lower P/E ratios (<15), indicating value-driven picks. And ST Engineering and IHH Healthcare have higher P/E, suggesting growth expectations are priced in. The portfolio balances value and growth.

3. Beta (Risk)

All selected stocks have low beta values (<1.0), indicating lower volatility relative to the market. SGX and IHH Healthcare in particular show very low beta, making them suitable for defensive positioning. This suggests the portfolio leans toward stability and capital preservation despite aiming for momentum.

4. Earnings Growth

Keppel DC REIT stands out with extremely high reported growth (3.73), though this may be due to recent base effects. SGX and ST Engineering also show strong forward momentum. IHH Healthcare and CapitaLand have low growth, but their inclusion may be driven by other factors like consistent ROE or defensive sector roles.

5. Return on Equity (ROE)

SGX and ST Engineering have very high ROE, indicating efficient capital usage. And CapitaLand Integrated Commercial Trust shows weak ROE, suggesting less efficient returns.

	Company	MarketCap (Billion SGD)	P/E Ratio	Beta (Risk)	Earnings Growth	Return on Equity
	UOB Bank	63.650000	10.760000	0.700000	0.110000	0.130000
	IHH Healthcare	18.530000	23.220000	0.190000	0.010000	0.100000
	ST Engineering	21.170000	30.820000	0.530000	0.200000	0.260000
	SGX	14.270000	21.850000	0.120000	0.240000	0.350000
	CapitaLand Integrated Commercial Trust	15.360000	15.000000	0.750000	0.130000	0.060000
	Keppel DC REIT	4.870000	12.710000	0.590000	3.730000	0.110000

Figure 5.1 Selected six Singapore Stocks

5.4.1.2 Technical Pattern Analysis

To complement the fundamentals, a time-series analysis of price dynamics was conducted:

Peak and Trough Detection: A 10-year historical analysis was performed to identify cyclical behavior in price movements.

Cycle Speed: The average duration between peaks and troughs was computed for each stock. Stocks exhibiting shorter cycle intervals were deemed more responsive to market movements, making them suitable candidates for momentum-based strategies.

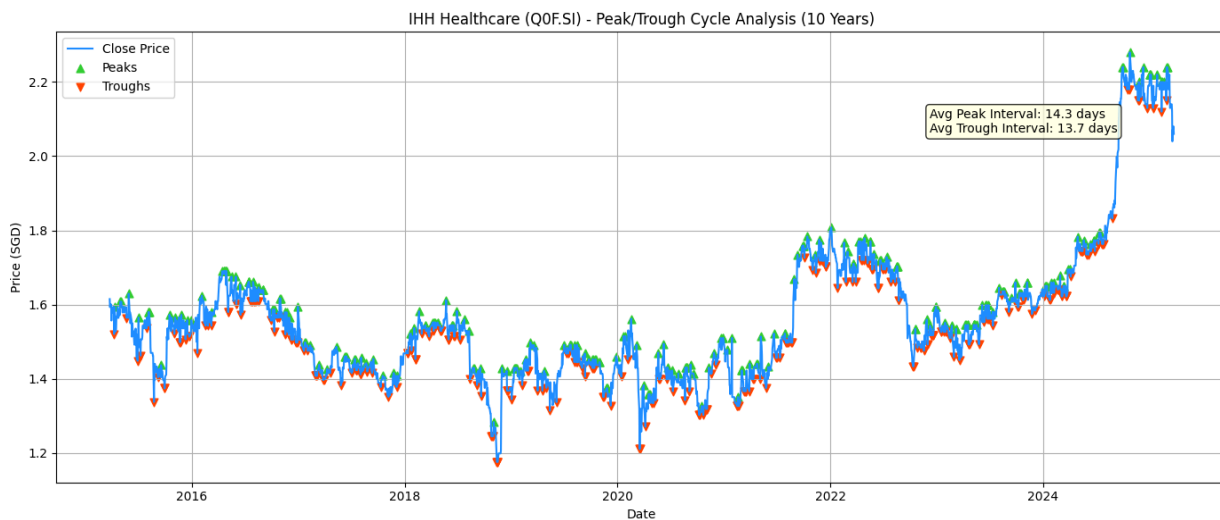


Figure 5.2 Dynamic Price Trend of Cyclic Portfolio of Q0F.SI

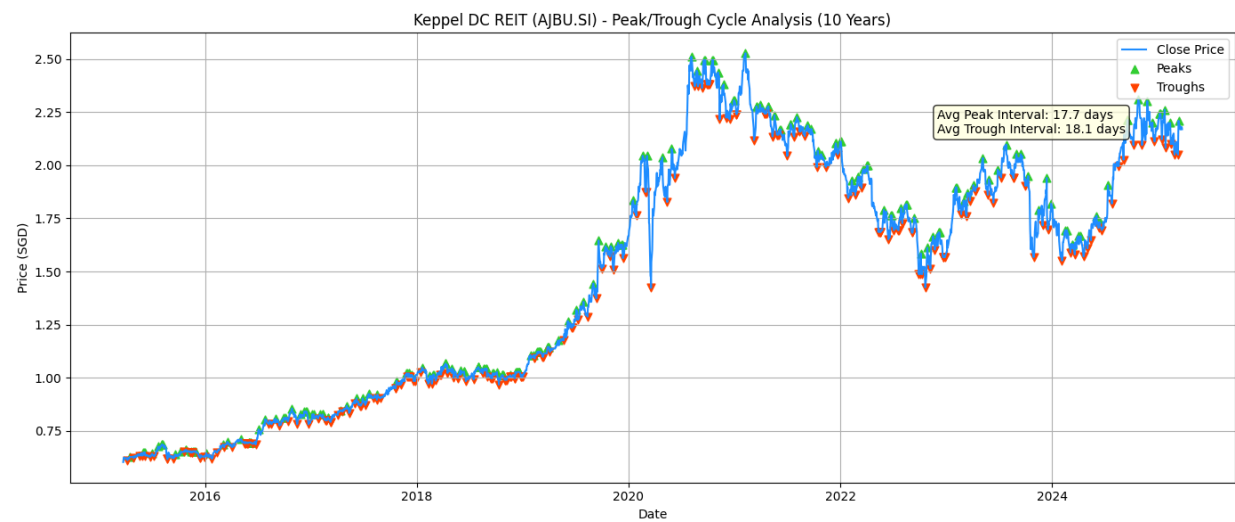


Figure 5.3 Dynamic Price Trend of Cyclic Portfolio of AJBU.SI

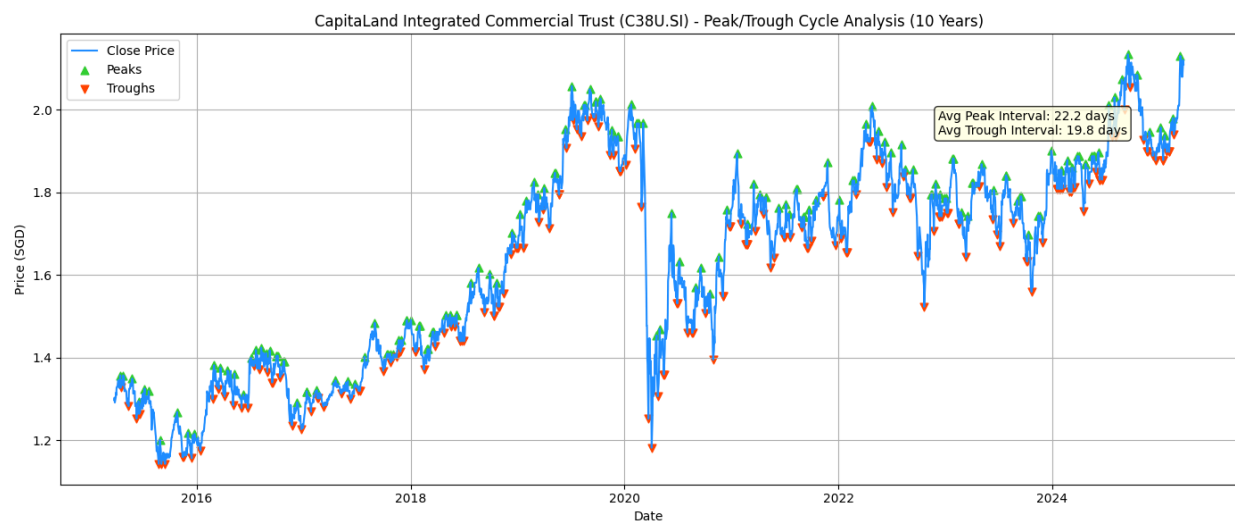


Figure 5.4 Dynamic Price Trend of Cyclic Portfolio of C38U.SI

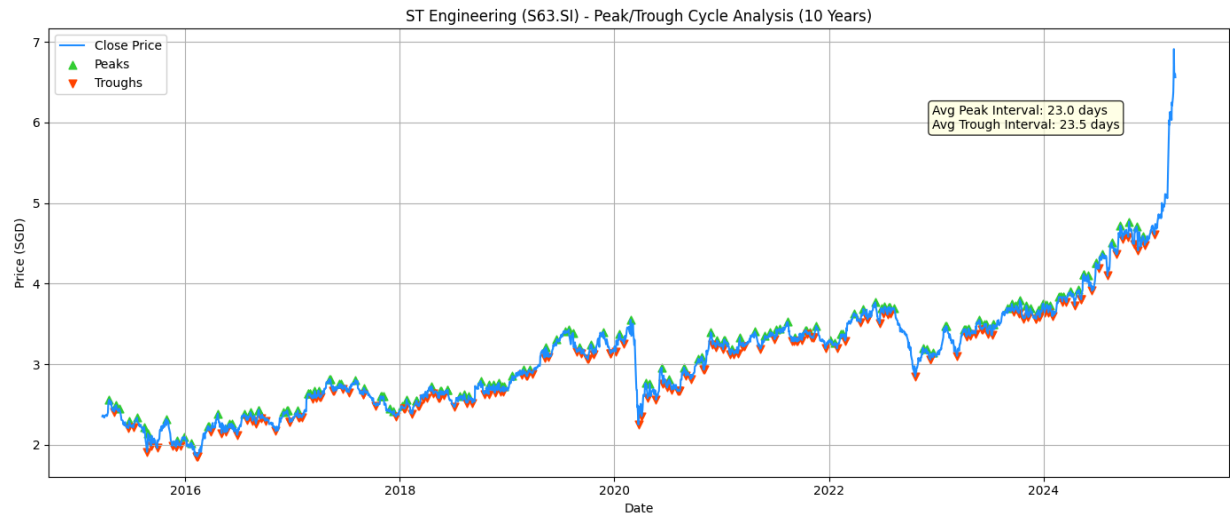


Figure 5.5 Dynamic Price Trend of Cyclic Portfolio of S63.SI

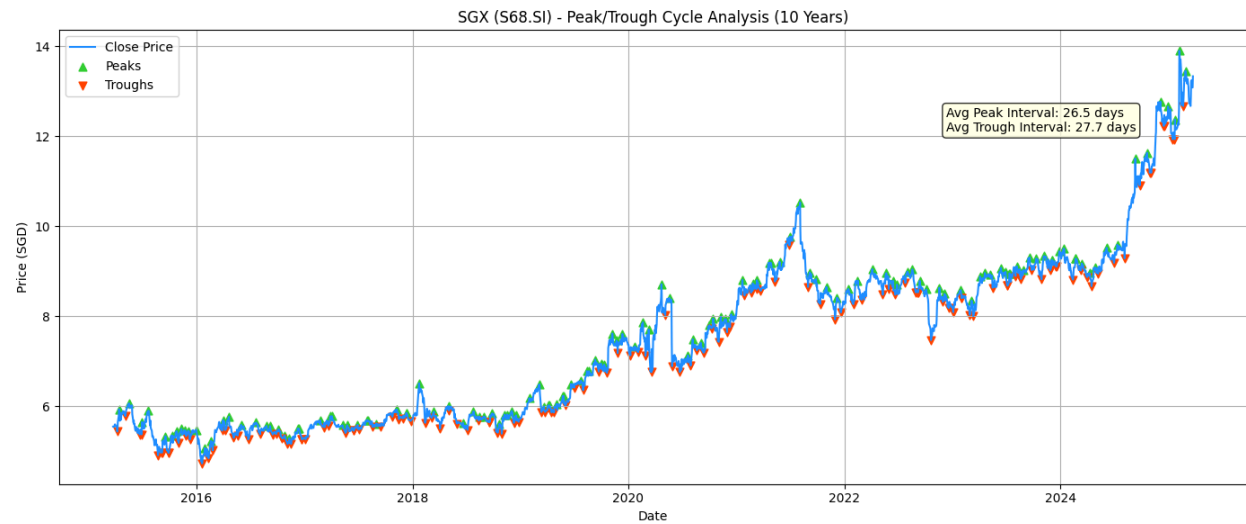


Figure 5.6 Dynamic Price Trend of Cyclic Portfolio of S68.SI

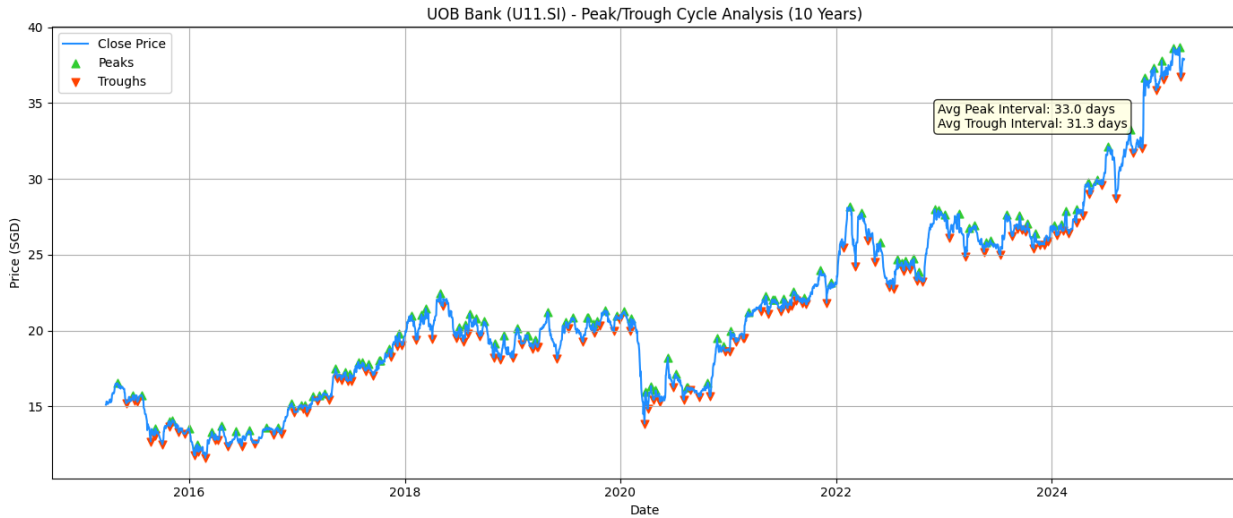


Figure 5.7 Dynamic Price Trend of Cyclic Portfolio of U11.SI

5.4.1.3 Reinforcement Learning-Based Weighting

A shallow neural network model was employed to assign a reinforcement learning (RL) weight to each stock based on its cycle dynamics. The model was trained using the computed peak and trough speeds as input features. It learned to associate faster cyclical behavior with higher potential trading opportunities. Stocks with higher RL-assigned weights were prioritized for portfolio inclusion, as they demonstrated a greater propensity for price action within shorter timeframes.

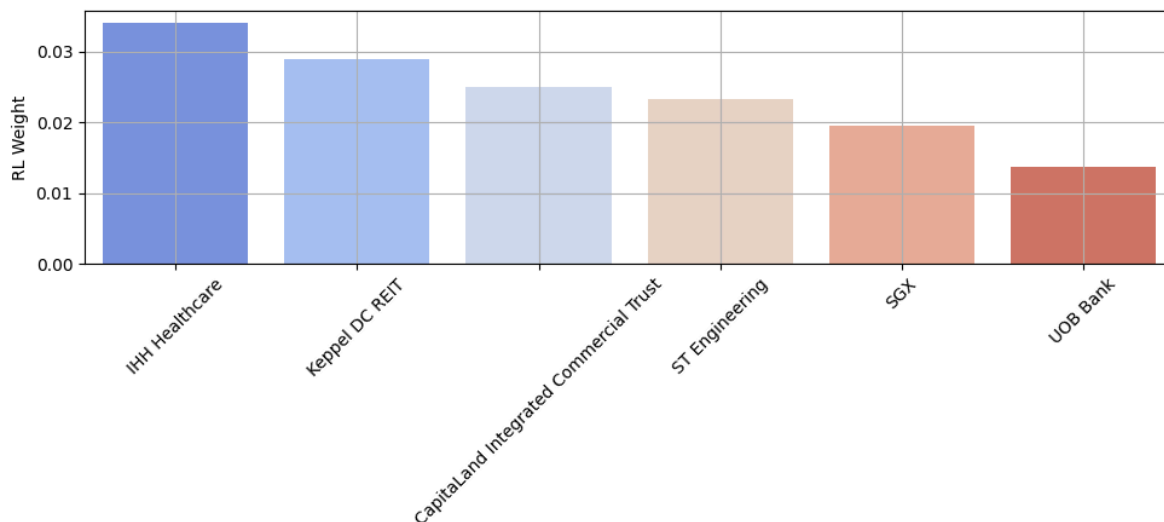


Figure 5.8 RL weights of the Selected Stocks

5.4.1.4 Correlation-Based Filtering

To enhance diversification and reduce unsystematic risk, correlation analysis was applied. The correlation matrix of the selected stock's 3-year returns confirms a diverse composition, with all pairwise correlations maintained below the 0.7 threshold. This supports the hypothesis that the portfolio avoids concentration risk and achieves better risk dispersion.

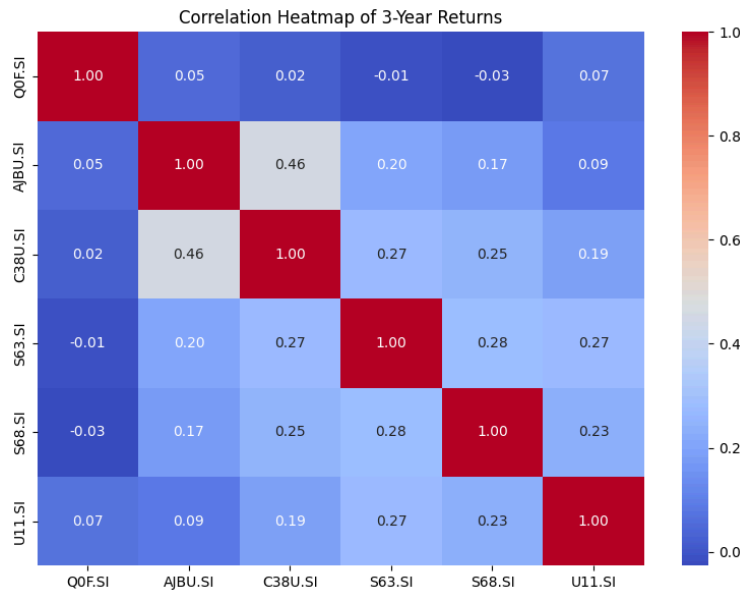


Figure 5.9 Correlation matrix of selected stocks

This portfolio strikes a thoughtful balance between growth and stability by combining both cyclical and non-cyclical stocks. With names like UOB, ST Engineering, SGX, and CapitaLand Integrated Commercial Trust, it's positioned to benefit from economic growth and market momentum. At the same time, defensive picks like IHH Healthcare and Keppel DC REIT add a layer of stability, helping cushion the portfolio during market downturns. This combination of growth-oriented and stable assets, supported by fundamental screening, technical analysis, and reinforcement learning-based prioritization, enhances diversification and optimizes the risk-return profile of the portfolio.

5.5 IFDTNN Implementation

This section provides a detailed, step-by-step explanation of the implementation process for the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN).

5.5.1 Data Preprocessing

Data preprocessing is a foundational step in building reliable and effective machine learning models. It encompasses a range of techniques such as data cleaning, normalization, and feature transformation to ensure the input data is well-suited for training. This step is especially critical when working with deep learning architectures like LSTM, BiLSTM, MLP, and Transformer networks, where the quality and consistency of input features directly impact the model's ability to learn temporal patterns and complex relationships. Normalization is particularly important for models trained using gradient descent, as unscaled features can slow convergence and introduce bias in weight updates [78][79].

In the context of stock price prediction, where the same feature (e.g., closing price) may vary greatly across time and stocks, normalization helps maintain a consistent learning context. For example, a stock that historically traded between \$0.50 and \$2.00 may later rise to a range of \$20 to \$80. Without normalization, the model may misinterpret this change in scale as a new pattern, leading to poor generalization and increased prediction error. To mitigate this, price data is scaled prior to training, allowing the models to learn based on relative fluctuations rather than absolute values. Once predictions are generated, the output values are denormalized and defuzzified (in the case of fuzzy models) to recover the final predicted closing prices in their original scale.

5.5.1.1 Feature Engineering

To enhance the predictive capacity of the model, additional features were derived from the raw stock market data, which includes daily closing prices and trading volume. One of the key engineered features is momentum, represented by the rate of change (ROC) of the closing price. This metric helps capture short-term price movement trends and was computed using the formula shown in Equation (5.1).

The core feature set constructed for model training includes:

- (1) the adjusted closing price (denoted as P),
- (2) the daily trading volume (denoted as V), and
- (3) the momentum or rate of change of the closing price (denoted as R).

$$ROC = \frac{P_t - P_{t-k}}{t - k}$$

where:

- P_t and P_{t-k} represent the stock prices at two different time points
- t and k are the respective indices of those time points
- ROC quantifies the relative speed and direction of price movements

Since models like LSTM, BiLSTM, and Transformers learn from sequences of past events, a sliding time window of historical data is used to construct the input features. This window captures the temporal patterns in P , V , and R over a fixed number of previous time steps. Correspondingly, another sliding window is formed to define the target outputs, which consist of a sequence of future closing prices.

This transformation allows the model to learn from a structured time series format, mapping recent market behavior to expected price movements in the near future. The full process of organizing input and output sequences is visually represented in Figure 5.2.

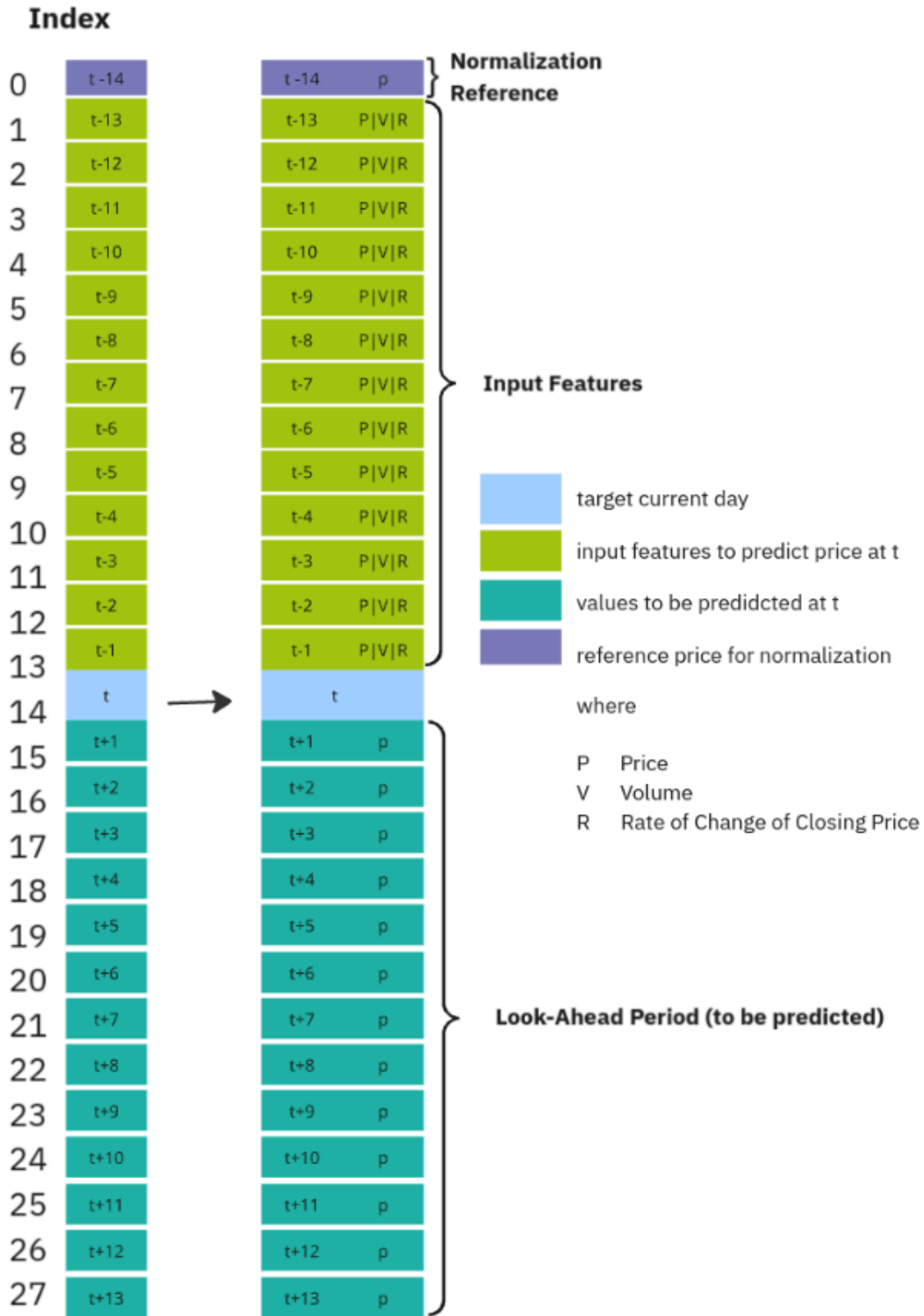


Figure 5.10 Feature Engineering

For this project, a historical window comprising the most recent 13 days along with their associated basic indicators is used as the input feature set to predict future closing prices across a 13-day look-ahead horizon. The decision to use a 13-day forecast period was guided by its compatibility with a 26-day Simple Moving Average (SMA) strategy, which is commonly applied in trend-following technical analysis. By predicting 13 future data points, the model enables the estimation of a forward-looking SMA(26), combining both actual and forecasted values to anticipate potential trend reversals more effectively.

5.5.1.2 Handling Data Shifts and Drifts: Sliding Window Normalization

To maintain the model's ability to generalize across different time periods, especially in the presence of evolving market conditions the input data is normalized using a sliding window approach. This ensures that the model continues to recognize patterns effectively, even when absolute price levels shift significantly over time. By normalizing data within a moving window, the model avoids being biased toward a particular price scale and can better adapt to new, unseen data.

For the price features, normalization is performed using the closing price at a reference point, $t-14$, as a baseline. This transforms each price value into a cumulative percentage change relative to that base, as shown in Equation (5.2):

$$N(P_{t+i}) = \frac{P_{t+i} - P_{t-14}}{P_{t-14}}, \quad \text{where } i \in \{-13, \dots, -1, 1, \dots, 13\}$$

This approach allows the model to focus on relative price movements rather than absolute values, which is particularly useful when stock prices vary drastically across different periods or instruments.

For the volume features, a different method is used. Instead of referencing a fixed point in time, min-max normalization is applied within each sliding window to account for large variability in trading volume over time. The formula used is given in Equation (5.3):

$$N(V_{t+i}) = \frac{V_{t+i} - V_{\min,sw}}{V_{\max,sw} - V_{\min,sw}}, \quad \text{where } i \in \{-13, \dots, -1\}$$

Here, $V_{max,sw}$ and $V_{min,sw}$ represent the maximum and minimum volumes, respectively, within the same input window. This ensures that volume data is scaled consistently with price data, improving the network's ability to interpret combined signals.

Notably, the ROC feature is left unnormalized, as it inherently falls within a bounded range and maintains consistent interpretability without additional scaling. By applying these tailored normalization techniques, the data becomes well-suited for training sequence-based neural networks and helps mitigate the effects of data drift and distributional shifts, enabling the model to retain relevance even as market dynamics evolve over time.

5.5.2 K-Means Clustering for Fuzzification

To enable the transformation of crisp input and output features into fuzzy representations, clustering was applied to each individual feature in the training dataset. The goal of this step is to group similar data points together, so that Gaussian membership functions can be defined for each cluster. These functions are later used to fuzzify the crisp features turning precise numerical values into fuzzy descriptors suitable for a fuzzy inference system. For this purpose, the K-Means algorithm was employed to segment the one-dimensional feature values into meaningful clusters.

5.5.2.1 Defining the Number of Clusters

Before clustering, all input and output features were normalized to a consistent scale. Price features were scaled to approximately $[-1, 1]$, volume features to $[0, 1]$, and the rate of change (ROC) already followed a constrained range. An initial number of 15 clusters was chosen to provide detailed coverage of the feature space—equivalent to a resolution of one cluster for roughly every 0.133 units. This ensures high granularity during the early stage of clustering.

5.5.2.2 Centroid Initialization

To promote faster convergence and broad coverage of the input space, initial centroids were evenly distributed across the feature range. The first $N-1$ centroids were spaced equidistantly between the feature's minimum and maximum values, with the last centroid fixed at the maximum value. This initialization process is captured in Equation (5.4):

$$C_i = F_{\min} + \frac{F_{\max} - F_{\min}}{N - 2} \times i, \quad C_{N-1} = F_{\max}$$

where:

- C_i is the initialized centroid for cluster i
- N is the total number of clusters
- $F_{\min}$ and $F_{\max}$ are the minimum and maximum values of the feature

This approach ensures that all clusters start with coverage across the full data range. As the algorithm iterates, centroids are updated to reflect the actual center of data within each cluster, allowing the final cluster statistics to accurately represent local distributions.

5.5.2.3 Cluster Merging for Generalization

Although 15 clusters provide detailed coverage, such high granularity can lead to overfitting in the fuzzy system. To encourage generalization, similar clusters are iteratively merged based on their proximity. The merging is guided by the distance between centroids—closer clusters are considered more alike and are merged first. For each merged cluster, updated statistics are calculated using Equation (5.5):

$$\mu_i = \frac{1}{L} \sum_{j=1}^L x_{ij}, \quad \sigma_i = \sqrt{\frac{1}{L} \sum_{j=1}^L (x_{ij} - \mu_i)^2}$$

where:

- μ_i is the mean (centroid) of cluster i
- σ_i is the standard deviation within the cluster
- x_{ij} are the data points in cluster i
- L is the number of points in the cluster

The merging process continues until the clusters become sufficiently distinct and non-overlapping. The stopping criteria are:

1. A final number of clusters between 6 and 8, and
2. A minimum inter-cluster distance of 1.75 standard deviations.

5.5.2.4 Fuzzification Outcome

Through this iterative process, each cluster's statistics are used to define a Gaussian membership function that best captures the local distribution of the data. As a result, each feature is fuzzified based on the actual shape and spread of its values, ensuring the fuzzy representation is data-driven and adaptive. The number of clusters and their spread will differ across features depending on how the original data is distributed, leading to flexible and accurate fuzzy mappings. These final membership values are then used as inputs to the fuzzy-enhanced transformer model.

5.5.3 Feature Selection Using Monte Carlo Evaluative Selection (MCES)

The initial phase of feature engineering produces a total of 39 input features, derived from the three core indicators—closing price, trading volume, and rate of change (ROC) across a 13-day historical window. However, once these crisp features are transformed through fuzzification, the total number of fuzzy features expands significantly, ranging from 180 to 240, depending on the number of clusters generated per feature. This variation stems from the fuzzification process, where each crisp input may be associated with between 6 to 8 clusters, as determined by the clustering termination criteria.

To manage this increase in dimensionality and ensure the model remains efficient and generalizable, Monte Carlo Evaluative Selection (MCES) described earlier in chapter 2. was applied as a method for feature selection. Given that the project adopts a multi-model forecasting approach, where a separate model is trained to predict each day within the 13-day look-ahead horizon, MCES was run independently for each forecasting model targeting the prediction of $t+i$, where $i \in \{1, 2, \dots, 13\}$.

In practice, a lightweight Multi-Layer Perceptron (MLP) model is used as a reference learner to evaluate feature importance. This MLP is trained for 20 epochs on the unnormalized prediction target, and the performance of each feature is assessed using root mean squared error (RMSE).

Through repeated sampling and evaluation, MCES provides cumulative feedback scores for each input feature where a higher score indicates stronger predictive relevance.

To ensure fairness across models and features, the feedback is aggregated by how often each input is selected across the different forecasting horizons. For each model, the top six crisp features with the highest MCES feedback are retained. These selected features are then converted into fuzzy representations using the previously derived membership functions, and only these are passed into the transformer model for training. This approach not only reduces the risk of overfitting but also streamlines the model, focusing it on the most informative patterns in the data.

5.5.4 Hyperparameter Tuning

To enhance the forecasting accuracy of the transformer model, a targeted hyperparameter tuning process was carried out using a limited genetic algorithm. Rather than performing a brute-force grid search, this evolutionary approach intelligently explored the configuration space by generating initial candidates at random and evolving them iteratively based on performance.

The tuning process involved a mix of fixed and adjustable parameters:

5.5.4.1 Parameter

Fixed Parameters

Learning Rate: Held constant to maintain training stability

Number of Attention Heads: Fixed at 10, the model's upper limit

Tuned Parameters

Encoder Layers: 1 or 2

Encoder Dimensions: 128 or 256

Hidden Layers: Between 1 and 6

Hidden Layer Size: 128, 256, or 512

Activation Function: ReLU , GeLU or Sigmoid

Batch Size: 32, 64, or 128

Optimizer: Adam or SGD

Each candidate configuration was evaluated based on its Root Mean Squared Error (RMSE) and R^2 score on both validation and test datasets. The fitness of each configuration during the genetic search was primarily judged by its validation RMSE, with lower values indicating better performance.

5.5.4.2 Results and Insights

Throughout the tuning process, a wide range of transformer architectures was tested. While validation R^2 scores varied considerably, some even negative, the test performance remained consistently strong among top configurations. The best-performing model achieved a test R^2 score of 0.91, demonstrating exceptional ability to generalize and make accurate predictions on unseen data.

This result underscores the success of the genetic algorithm in identifying well-balanced architectures. Notably, even in cases where validation metrics appeared less promising, the combination of optimal batch sizing and thoughtful architectural design helped maintain robust generalization on the test set.

Ultimately, the final transformer model was selected based on its strong predictive accuracy and stable performance, validating the effectiveness of this evolutionary tuning approach in navigating complex hyperparameter spaces.

5.5.5 Evaluation IFDTNN Prediction

The IFDTNN is designed to forecast normalized price values for future timesteps $t+i$, using historical data up to a reference point at $t-14$. Following the prediction and defuzzification processes, the output consists of normalized price values relative to the base day $t-14$.

To convert these predictions back to actual price levels, a sliding window denormalization is applied. This transformation, described in Equation (5.6), re-scales the predicted normalized values to actual close prices:

$$P_{t+i} = P_{t-14} \times (1 + \text{pred}_{ci})$$

where:

P_{t+i} : Predicted close price at future timestep $t+i$

P_{t-14} : Known close price at reference timestep

pred_{ci} : Predicted normalized price relative to the base day

This allows for the accurate reconstruction of predicted close prices, maintaining consistency with the original price scale.

The performance of the IFDTNN model is then evaluated by comparing the predicted close prices against the actual values. Evaluation metrics include R^2 (coefficient of determination) and RMSE, as outlined earlier in chapter 2. These metrics provide insight into the model's ability to fit and generalize across unseen data.

5.5.6 Benchmarking the IFDTNN Architecture

To evaluate the performance of the IFDTNN architecture, multiple benchmarking approaches were conducted for comparison.

5.5.6.1 Crisp Feature Training as a Baseline

To rigorously assess the contribution of the Mamdani-type fuzzy logic system integrated within the IFDTNN, a comparative benchmarking strategy was adopted. Specifically, a baseline transformer model, a LSTM and a BiLSTM was trained on the same historical market data but using only crisp features, without any fuzzification.

This benchmark allows us to isolate the effects of fuzzy logic integration and examine whether the fuzzified features and interpretable structure contribute to performance improvements. By comparing the prediction accuracy and error metrics of the baseline transformer and the full IFDTNN, we can evaluate the added value of incorporating fuzzy inference into a deep learning framework.

Chapter 6

Experimental Results and Benchmarking

Chapter 6 presents the experimental setup, including dataset preparation, training pipelines and feature engineering.

6.1 MCES Feature Selection Results

To determine which input features had the greatest influence on prediction performance, the MCES method was employed. This algorithm works by assessing the impact of each individual feature on the model's accuracy, essentially measuring how much the prediction quality drops when a particular feature is excluded. Features that caused the largest degradation in performance when removed were considered the most critical.

For each of the 13 prediction horizons (from $t+1$ to $t+13$), MCES was applied independently, allowing for a more tailored feature selection that reflects the unique dynamics and patterns associated with each forecast window. Rather than using a fixed set of features for all time steps, this approach helped uncover temporal dependencies, where the importance of features changes depending on how far into the future the model is attempting to predict.

The top six features, ranked by their MCES feedback scores, were retained for each model. These scores were then aggregated across all 13 models, enabling an overall analysis of feature importance trends. This cumulative feedback was especially useful in identifying features that consistently contributed to predictive power across different look-ahead periods. Notably, certain features such as price or signals from the most recent days tended to appear more frequently, suggesting their stronger correlation with short-term price behavior.

C38U.SI Model	Feature 1	Feedback 1
t + 1	t-3 Price	0.002985
t + 2	t-2 ROC	0.080832
t + 3	t-4 Price	0.011445
t + 4	t-1 ROC	0.011037
t + 5	t-1 ROC	0.024790
t + 6	t-5 Price	0.020918
t + 7	t-5 ROC	0.049377
t + 8	t-12 Price	0.028004
t + 9	t-4 Vol	0.028476
t + 10	t-1 Price	0.032595
t + 11	t-9 Vol	0.027458
t + 12	t-12 Vol	0.017140
t + 13	t-12 Vol	0.016584

Table 6.1 Top features selected Using MCES for C38U.SI Look-Ahead Models

The application of the MCES (Monte Carlo Evaluation Strategy) algorithm allowed for the precise identification of the most impactful features driving predictive performance across the 13 forecasting models (t+1 to t+13).

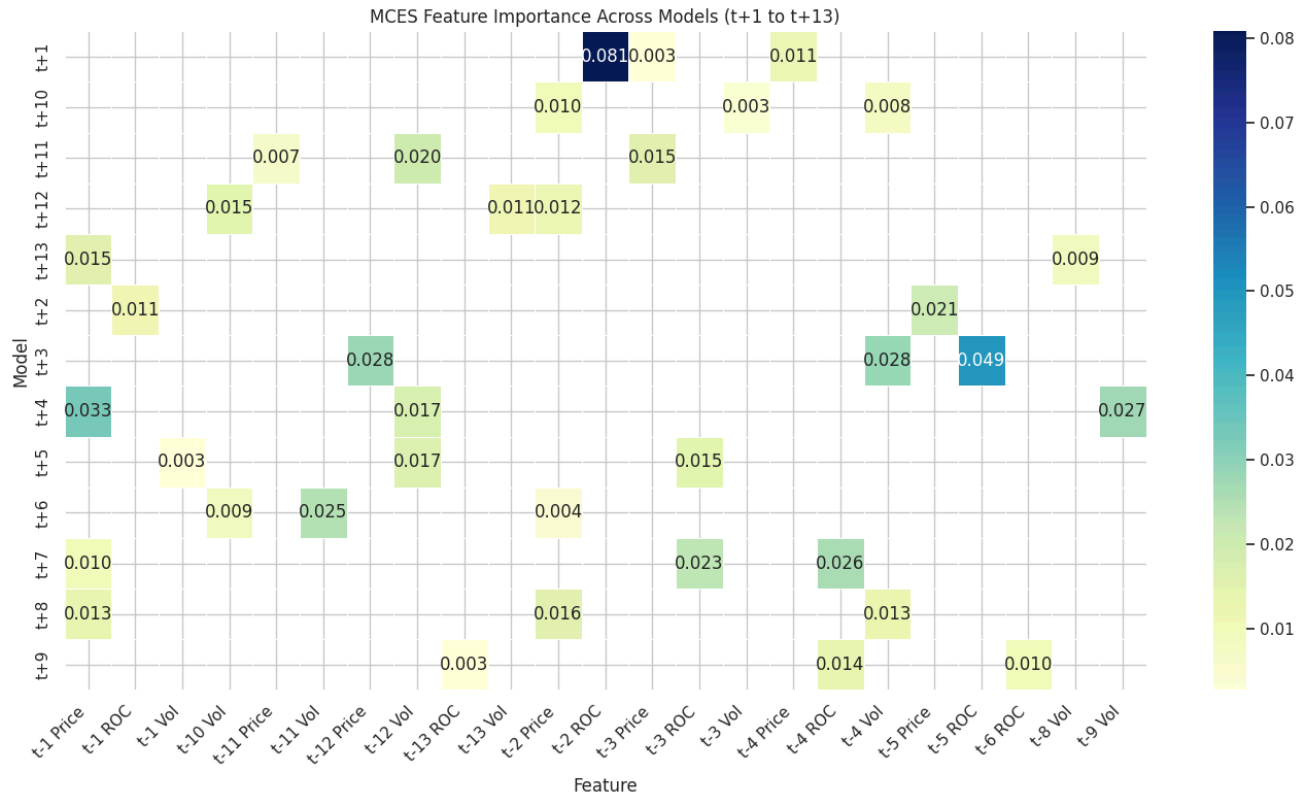


Figure 6.1 MCES Feature Importance across (t+1 to t+13) of C38U.SI

The second and third most selected features, t-2 Price and t-4 Volume, highlight the algorithm's sensitivity to recent market dynamics while still leveraging slightly longer historical patterns. The presence of volume-based features like t-4 Vol among the top selections suggests that MCES effectively captures demand-side pressure and momentum factors often overlooked in more traditional approaches. A key strength of MCES is its ability to derive feature importance purely from performance feedback, without relying on prior assumptions or domain bias. As shown in the heatmaps and frequency charts, the model adapts to what works best for each time horizon in a transparent and interpretable way. This reinforces the robustness of MCES as a data-driven and explainable approach to feature selection in time series forecasting.

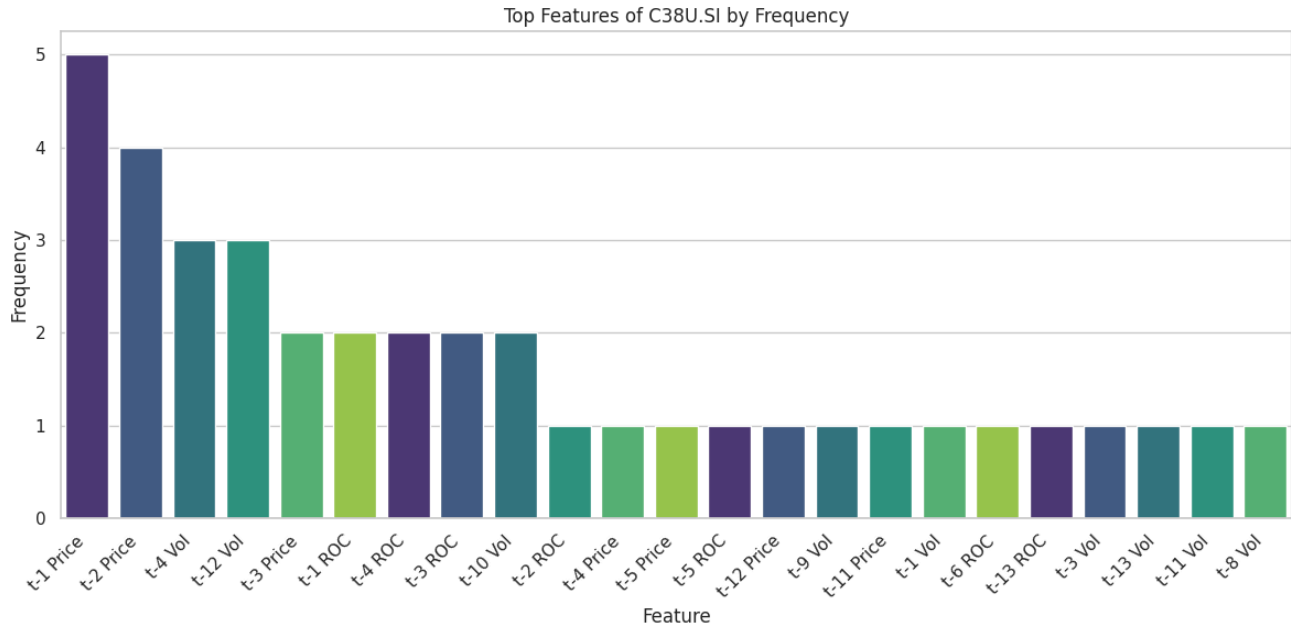


Figure 6.2 Frequency Statistics of the features of MCES

As seen from Table 6.1, t-1 Price is one of the most commonly valued features as it is the feature that is temporally closest to the set of the look-ahead values to be predicted. The count instance of each feature being selected is not consistent. This provides evidence that the multiple model approach is more suitable for predicting price in the look-ahead period given that different models require different sets of features for prediction. It is also interesting to see that t-1 price, t-2 Price, t-4 Vol, t-12 Vol were consistently within the top three features for all 13 models for C38U.SI. The count statistics of the remaining stocks in the cyclic portfolio are detailed in Appendix A1 MCES Feature Selection Outcome.

6.2 Evaluation of K-Means Clustering-Based Defuzzified Predictions

The main purpose of applying K-Means clustering was to better capture the underlying patterns of the stock price data. By organizing data points into clusters, each described clearly by a Gaussian function (defined by its mean and standard deviation), the fuzzy membership values accurately reflect the data distribution. This process reduces errors when converting fuzzy predictions back into precise (defuzzified) stock prices.

Table below summarizes the clustering performance based on the R^2 scores and RMSE values of the defuzzified predicted stock price changes for the test dataset:

Ticker	R^2 Score	RMSE Score
	0.8326	0.0147
	0.8687	0.0177
	0.8628	0.0222
	0.8775	0.0370
	0.8850	0.0279
Metrics		

Table 6.2 Cluster-based Prediction Performance (Test Dataset)

Above 0.80, indicating that the clustering method was effective in capturing and representing the price patterns using fuzzy Gaussian memberships. AJBU.SI exhibited slightly lower performance (R^2 of 0.8326), suggesting greater complexity or volatility in its price movements, possibly due to factors like liquidity or sensitivity to economic changes. Additionally, the RMSE values generally indicate low prediction errors, affirming that the clustering method successfully represented the actual price variations. These results highlight the overall effectiveness of using K-Means clustering coupled with Gaussian fuzzy membership functions in stock price prediction tasks.

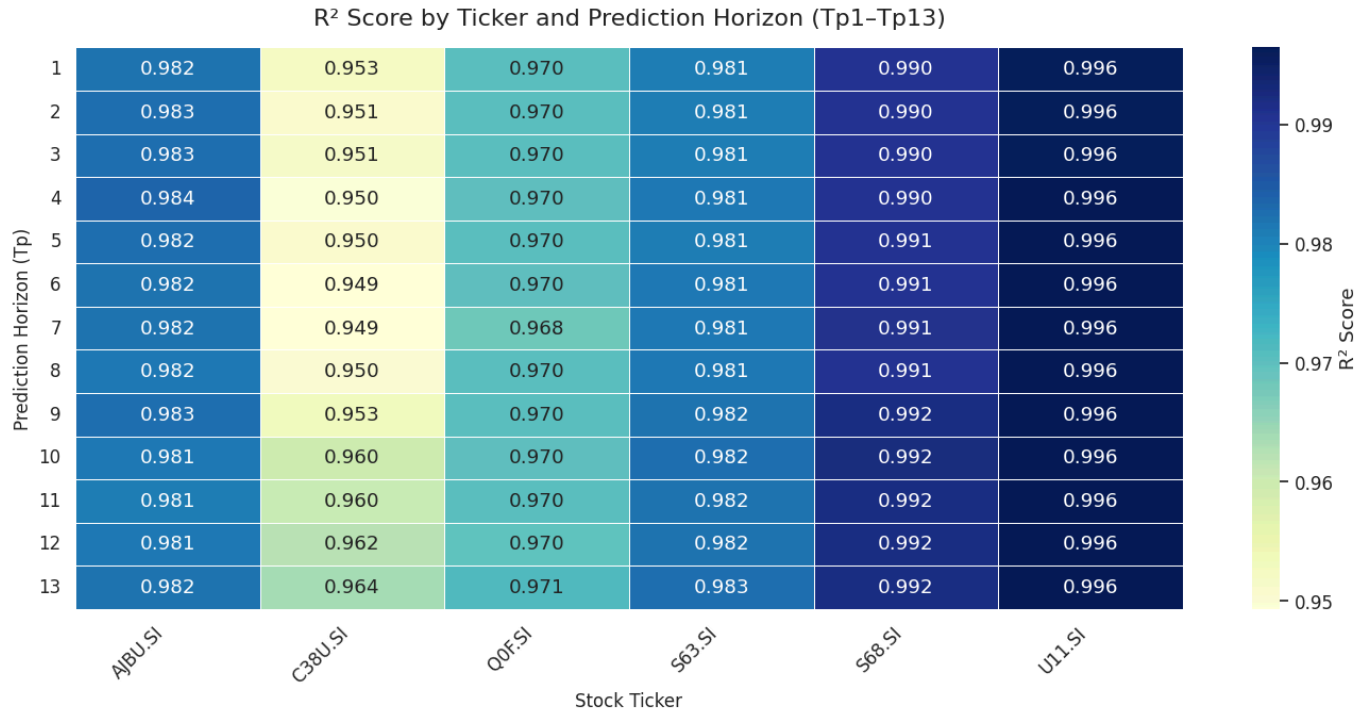


Table 6.3 R² model performance of all stocks

The performance of the ensemble-based IFDTNN model across the selected SGX stocks demonstrates remarkable predictive accuracy and consistency over multiple future time points (Tp1–Tp13). As shown in the heatmap of R² scores, the model achieves values consistently above 0.95, with several stocks such as U11.SI, S68.SI, and S63.SI reaching R² scores of 0.98 and beyond across all horizons. This indicates a strong ability of the model to generalize and maintain precision, even as the prediction window extends. Furthermore, the use of multiple evaluation metrics including RMSE provides a comprehensive validation of the model's low error rates and stable forecast behavior.

IFDTNN Prediction Performance Across Tp1-Tp13

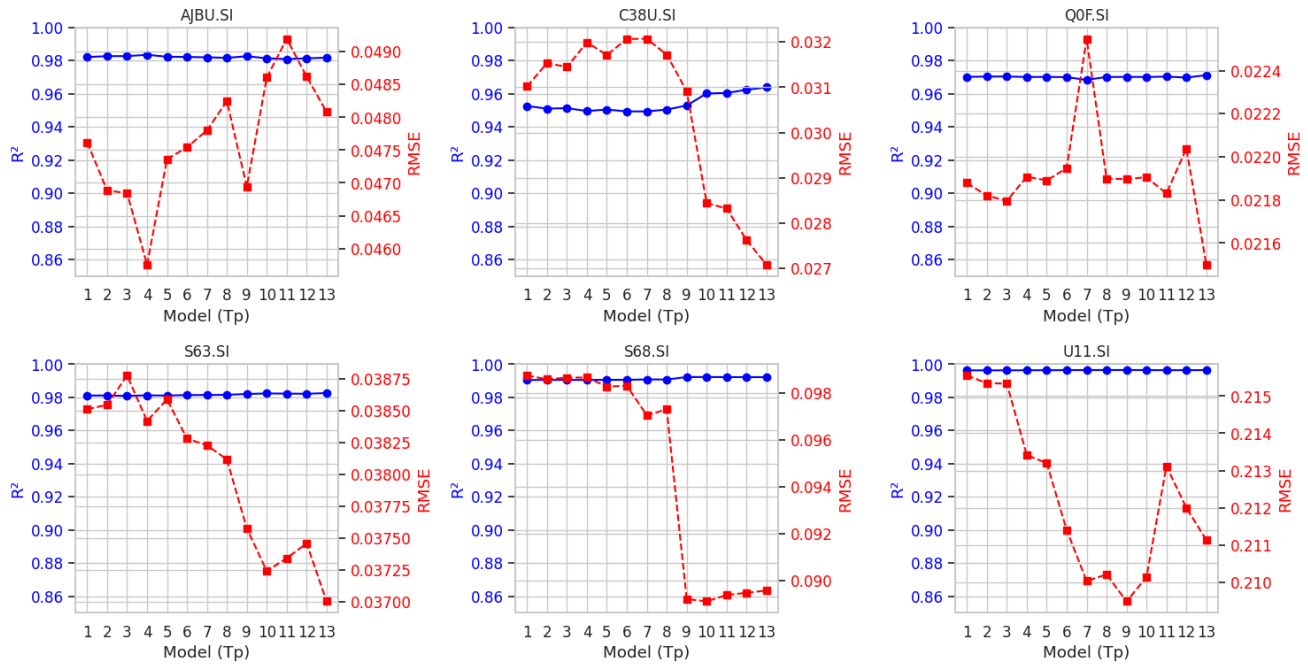


Figure 6.3 Trends of IFDTNN Prediction Performance

In addition to strong tabular performance, the plotted dual-axis graphs illustrate clear and interpretable trends in both R^2 and RMSE values for each stock. Most tickers exhibit flat or steadily improving R^2 trends, while RMSE remains low or decreases as prediction length increases. This reinforces the robustness of the IFDTNN framework in handling both short- and long-term financial forecasting. Notably, stocks like C38U.SI and AJBU.SI show some minor fluctuations, yet overall performance remains reliable. The combined visual and statistical evidence supports the conclusion that this hybrid fuzzy-deep learning approach is well-suited for high-accuracy time series modeling in the financial domain.

Actual vs Predicted Close Prices (T+5 and T+10)

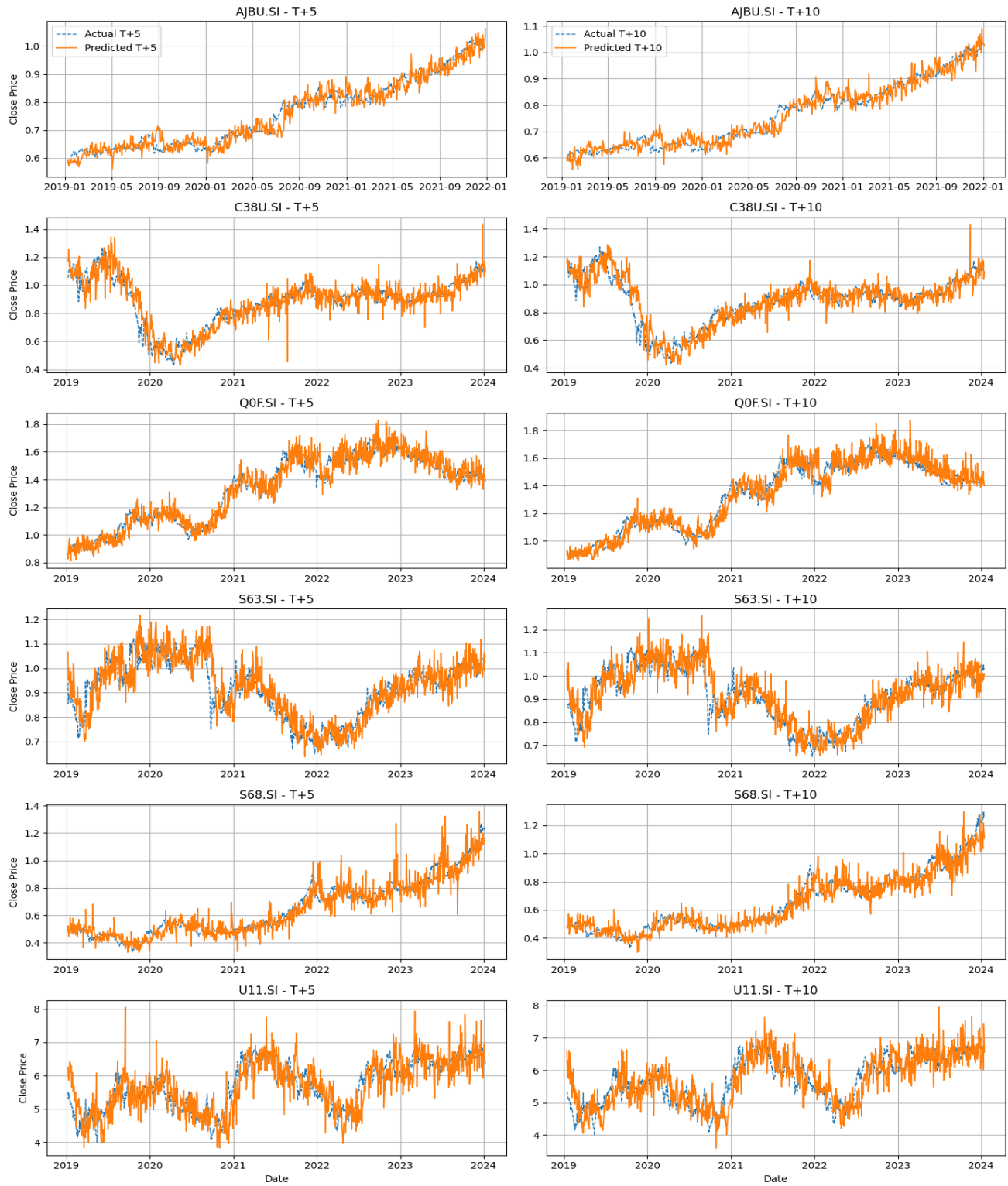


Figure 6.4 Consolidation of stocks Prediction vs Actual Trace T+5 and T+10

6.3.2 Hyperparameter Optimization

The hyperparameter tuning process revealed several key insights into the relationship between architectural choices and model performance. Through iterative tuning of 98 configurations, several trends emerged from the five top-performing Transformer configurations. Most models favored an encoder dimension of 256, suggesting it provided a strong balance between model capacity and overfitting risk. The best result came from a setup using GELU activation, 2 hidden layers, and a batch size of 64, which achieved the lowest validation RMSE (0.0436) and highest test R^2 (0.909). This indicates that moderate depth and smoother activations can be particularly effective for modeling complex stock price movements.

Hyperparameter	1	2	3	4	5
Hidden Layer Activation function	gelu	relu	relu	gelu	relu
Encoder Dimension	256	256	256	128	256
Hidden_layers	2	6	6	2	2
Batch Size	64	64	64	32	64
Loss function	HuberLoss	MCELoss	MCELoss	HuberLoss	MCELoss
Validation set RMSE	0.0436	0.0466	0.0466	0.0431	0.0487
Test R^2	0.909	0.901	0.901	0.898	0.893
Test set RMSE	0.106	0.112	0.112	0.113	0.116

Table 6.5 C38U.SI (t+13) Hyperparameter Tuning Outcome

Furthermore, while ReLU appeared in multiple strong-performing trials, it was slightly outperformed by GELU in both validation and test metrics. Batch size also played a critical role, with most top trials using a batch size of 64, though one trial (Trial 4) with a batch size of 32 still achieved competitive results, indicating that smaller batches may improve learning granularity at the cost of minor performance tradeoffs. In terms of loss functions, HuberLoss and MCELoss emerged as viable alternatives to standard MSE, with HuberLoss yielding the best validation and test performance,

likely due to its robustness to outliers. These results collectively highlight that careful calibration of activation functions, loss formulation, and batch size can yield meaningful improvements in time series forecasting accuracy.

6.3.3 Fuzzy Rule Base

To enhance the interpretability of the model's predictions, a Hebbian-inspired learning mechanism was employed to associate fuzzy inputs with their corresponding fuzzy output clusters. Historical data from 2019 to 2024, taken from the training set, was used to track which fuzzy rules were activated during this period. The relative importance of each rule was quantified using Hebbian weights, which effectively capture how frequently and how recently a given rule has been triggered. Higher weights suggest more prominent or influential rules in the model's reasoning process. Table 2.1 highlights the most influential rule for each model, based on these Hebbian weights (e.g., $t-1_Price_c4$ refers to timestep $t-1$ price in cluster 4).

Model	Rank	Rule	
		Antecedents	Consequent
AJBU.SI	1	● $t+1_Price_c3$ ● $t+4_Price_c3$ ● $t+7_Vol_c0$ ● $t+6_Vol_c6$ ● $t+2_Price_c2$ ● $t+9_ROC_c3$	$t+13_Price_c3$
C38U.SI	1	● $t+12_Vol_c0$ ● $t+4_ROC_c3$ ● $t+1_Price_c3$ ● $t+3_Price_c2$ ● $t+1_ROC_c3$ ● $t+5_Vol_c0$	$t+13_Price_c3$
Q0F.SI	1	● $t+3_Vol_c0$ ● $t+12_Vol_c0$ ● $t+4_Price_c1$ ● $t+13_Vol_c0$ ● $t+9_Price_c3$ ● $t+11_Price_c3$	$t+13_Price_c3$
S63.SI	1	● $t+2_Price_c3$	$t+13_Price_c3$

		● $t+10_ROC_c2$ ● $t+4_Vol_c0$ ● $t+12_Price_c2$ ● $t+6_Vol_c0$ ● $t+6_ROC_c2$	
S68.SI	1	● $t+3_Vol_c0$ ● $t+3_ROC_c3$ ● $t+6_Price_c3$ ● $t+7_ROC_c3$ ● $t+11_ROC_c3$ ● $t+1_Vol_c0$	$t+13_Price_c3$
U11.SI	1	● $t+1_ROC_c2$ ● $t+11_ROC_c2$ ● $t+2_Vol_c1$ ● $t+12_Vol_c0$ ● $t+2_Price_c2$ ● $t+5_Price_c2$	$t+13_Price_c3$

Table 6.6 Top Fuzzy Rules of $t+13$ Models

By constructing this fuzzy rule base, the inner workings of the predictive model become more transparent. Stakeholders and analysts can now trace the logic behind predictions by observing the input–output fuzzy cluster relationships and their associated Hebbian strength. This structure offers a meaningful lens into how the model arrives at its conclusions, bridging the gap between opaque deep learning mechanisms and human-understandable reasoning. As a result, the framework supports greater confidence and explainability when communicating model decisions to decision-makers or regulatory bodies.

6.3.4 IFDTNN Architecture Benchmark Results

6.3.4.1 Benchmarking IFDTNN Architecture on Crisp Feature Training

To benchmark the IFDTNN architecture, a set of baseline models including Transformer, LSTM, and BiLSTM were trained using the full set of crisp input features. These models underwent standard preprocessing steps such as feature engineering and sliding window normalization. Importantly, no MCES-based feature selection was applied during this process, meaning the models had access to all raw input features without any filtering or dimensionality reduction. This setup allowed for a fair

comparison, testing whether IFDTNN's fuzzy logic and selective feature processing could outperform models relying purely on raw data and conventional deep learning architectures.

C38U.SI	(IFDTNN)		Transformer		LSTM		Bi LSTM	
Model	R2	RMSE	R2	RMSE	R2	RMSE	R2	RMSE
t+1	0.9526	0.0310	0.9553	0.1532	0.9295	0.0379	0.8343	0.0817
t+2	0.9510	0.0315	0.9560	0.2517	0.8374	0.0575	0.8204	0.1188
t+3	0.9512	0.0314	0.9546	0.1620	0.7990	0.0640	0.6607	0.0946
t+4	0.9495	0.0319	0.9344	0.1256	0.8221	0.0752	0.7628	0.0695
t+5	0.9504	0.0317	0.9322	0.1391	0.8651	0.0523	0.7913	0.0793
t+6	0.9493	0.0320	0.9341	0.1324	0.8606	0.6312	0.9038	0.0442
t+7	0.9493	0.0317	0.9333	0.1267	0.8104	0.54571	0.8632	0.0528
t+8	0.9504	0.0309	0.9378	0.1324	0.8537	0.3541	0.7126	0.0765
t+9	0.9529	0.0284	0.9421	0.1388	0.9384	0.6057	0.8649	0.0524
t+10	0.9601	0.0283	0.9564	0.1370	0.8197	0.3666	0.7043	0.0776
t+11	0.9605	0.0282	0.9451	0.1391	0.9339	0.4247	0.9369	0.0811
t+12	0.9623	0.0276	0.9358	0.1461	0.9114	0.0577	0.8284	0.0881
t+13	0.9638	0.270	0.9541	0.1463	0.8361	0,6212	0.7945	0.1016

Table 6.7 IFDTNN Benchmark on C38U.SI Crisp Feature Training Performance Comparison

The results clearly show that IFDTNN is the most reliable and consistent model for forecasting C38U.SI stock prices over a 13-day horizon. It consistently delivers high R^2 values (mostly above 0.95) and very low RMSE values, showing not just strong accuracy but also stability over time. Even as the prediction horizon extends, IFDTNN remains steady, unlike LSTM and BiLSTM, which become more erratic, especially LSTM, which shows unusually high errors at t+6 and t+9. The Transformer model performs reasonably well in the short term (t+1 and t+2), but its accuracy

gradually drops off with longer forecasts. IFDTNN stands out not only for its prediction strength but also for how consistently it handles the volatility in financial time series data.

It's also worth highlighting that the benchmarked Transformer models were trained using the full set of crisp input features, without any MCES feature selection or fuzzy logic. In contrast, IFDTNN used only a selected subset of features identified through MCES, yet still outperformed the Transformer across every time step. This makes a strong case for the power of MCES in filtering out noise and boosting signal quality, as well as the value of fuzzy logic in improving both performance and interpretability. Despite having access to more raw data, the traditional Transformer couldn't match IFDTNN's clarity and precision, showing that smarter, more meaningful inputs often beat sheer data volume.

Chapter 7

Chapter 7 analyzes and compares the performance of buy and hold strategy with Genetic Algorithms

Trend Reversals and Algorithmic Finance

In Chapter 7, the focus shifts to leveraging IFDTNN's price forecasts to refine trading strategies by incorporating these predicted future closing prices into technical indicators. This approach aims to transform traditionally lagging indicators into more proactive tools capable of anticipating trend reversals earlier. By doing so, traders can respond in real time based on anticipated shifts rather than waiting for confirmation. The effectiveness of IFDTNN's predictions is then assessed by analyzing how well these enhanced indicators perform in actual trading scenarios. Additionally, various technical indicators and their modified versions are examined to further sharpen the accuracy of trend reversal detection.

7.1 Buy and Hold Strategy: Baseline Benchmark

To provide a baseline for evaluation, we implemented a Buy and Hold strategy across the same stock basket. The algorithm allocates capital to purchase as many shares as possible on the first trading day and liquidates the portfolio on the final day. This method captures the natural appreciation (or

depreciation) of each security over the full period with a transaction cost of 0.125% applied to both entry and exit. And there is no trading logic involved one entry and one exit only.

This strategy offers insight into the upper-bound of passive investment returns, serving as a point of comparison for active trading methods. ROI (Return on Investment) is a measure of how much profit or loss you made relative to your initial investment.

Formula:

$$\text{ROI} = \frac{\text{Final Portfolio Value} - \text{Initial Investment}}{\text{Initial Investment}} \times 100$$

7.2 Technical Indicators and Trend Analysis

To complement the fundamental analysis, a set of technical indicators was applied to each stock's historical price data. These indicators provide insight into price momentum, trend direction, and volatility, helping identify favorable entry and exit points. The goal is to detect trend reversals, sustained movements, and overbought/oversold conditions as part of the cyclic and momentum strategy.

7.2.1 Simple Moving Average(SMA)

Indicator	Description	Interpretation
SMA 12	The 12-day simple moving average	Captures the short-to medium-term trend of a stock. It responds more quickly to recent price changes.
SMA 26	The 26-day simple moving	Represents the long-term trend and acts as a

	average	signal of sustained bullish or bearish momentum.
--	---------	--

Table 7.1 Simple Moving Average

These two averages are often used together for trend confirmation:

- When the SMA 12 crosses above the SMA 26, it may indicate a bullish trend reversal, commonly referred to as a golden cross.
- When the SMA 12 crosses below the SMA 26, it may signal a bearish reversal, known as a death cross.

7.2.2 Volatility Penalty

The excessive price fluctuation can indicate instability or noise. The standard deviation of daily returns was used to compute a volatility penalty, where higher volatility reduced a stock's trend score. This ensures the model favors consistent momentum over random swings.

7.2.3 MACD

The standard MACD uses a 26/12/9 setup—26 for the long-term EMA, 12 for the short-term EMA, and 9 for the signal line. This configuration helps detect trend direction and momentum shifts. Traders may adjust these values to suit different timeframes or asset behaviors.

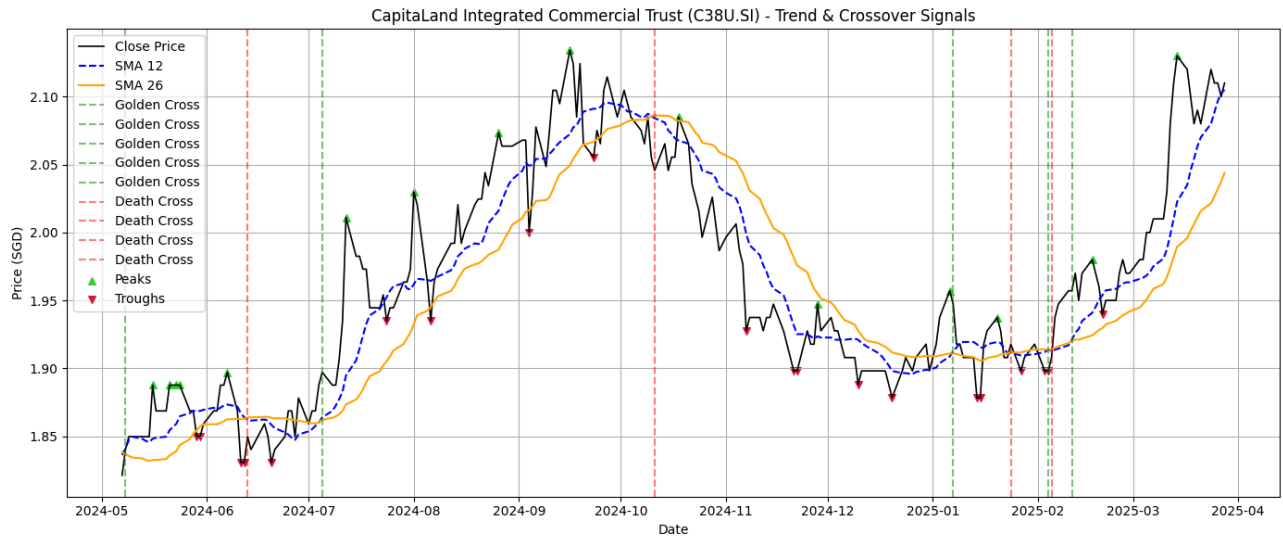


Figure 7.1 Trend and Crossover Signals of CapitaLand Integrated Commercial Trust (C38.SI)

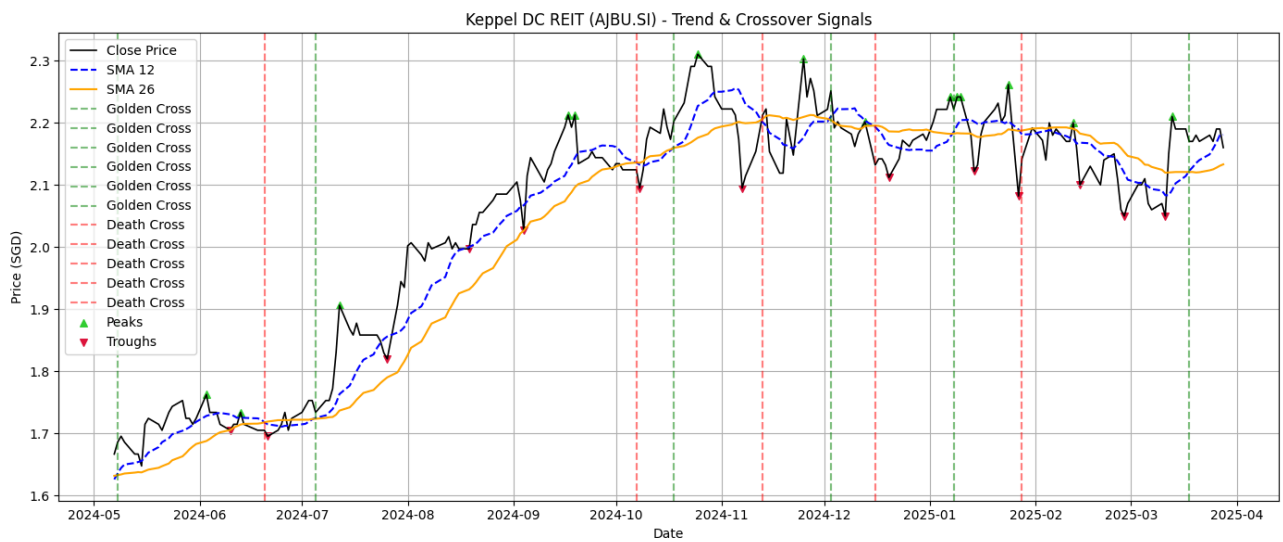


Figure 7.2 Trend and Crossover Signals of Keppel DC REIT (AJBU.SI)

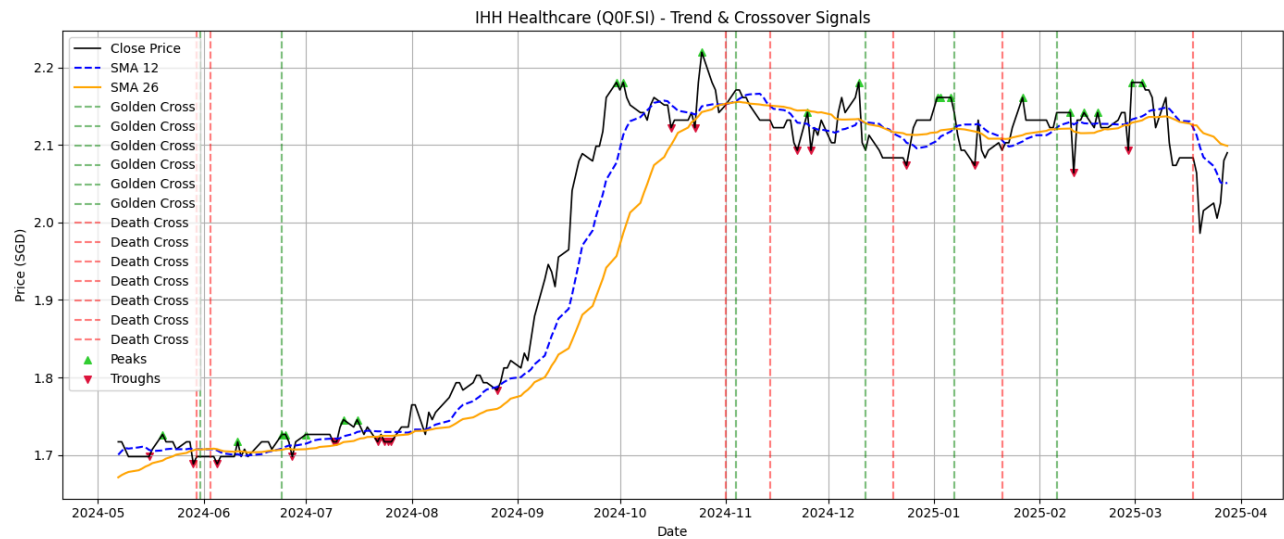


Figure 7.3 Trend and Crossover Signals of IHH Healthcare (Q0F.SI)

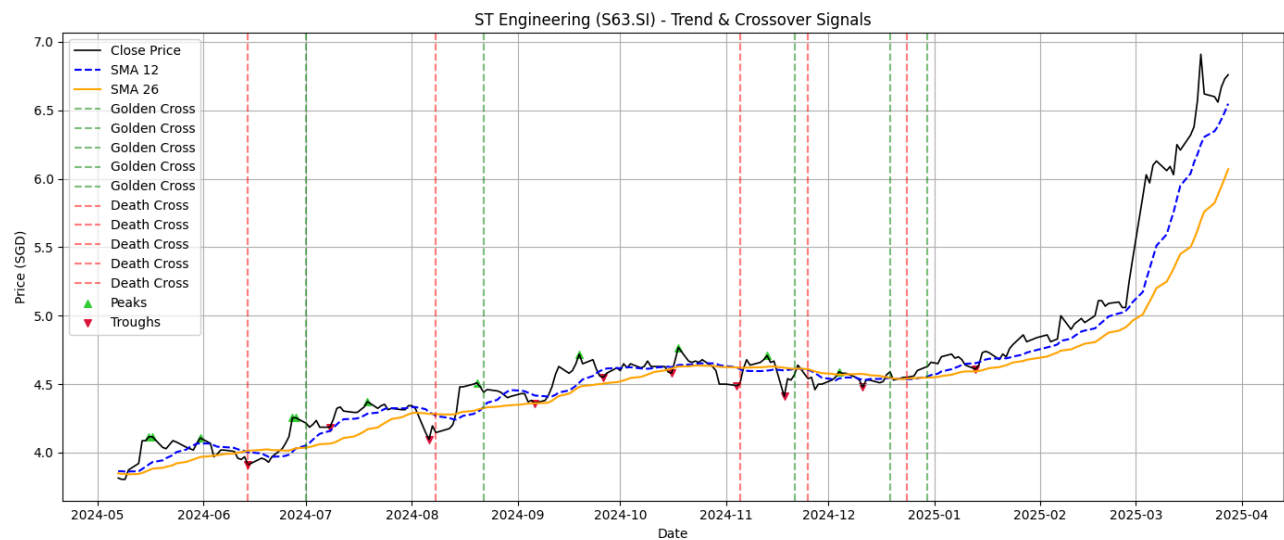


Figure 7.4 Trend and Crossover Signals of ST Engineering (S63.SI)

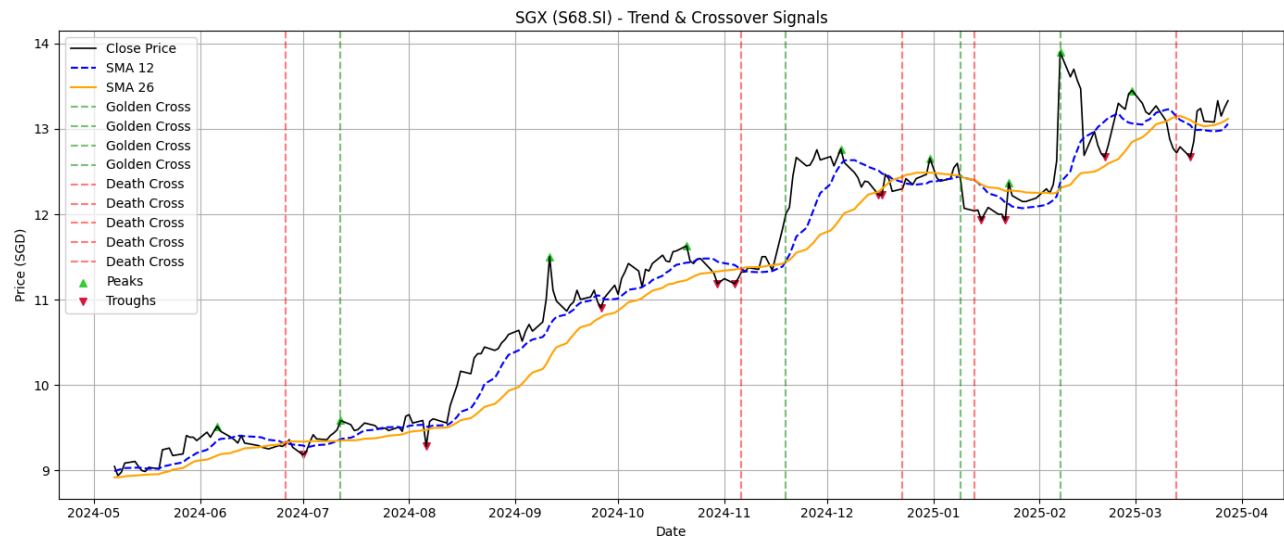


Figure 7.5 Trend and Crossover Signals of SGX (S68.SI)

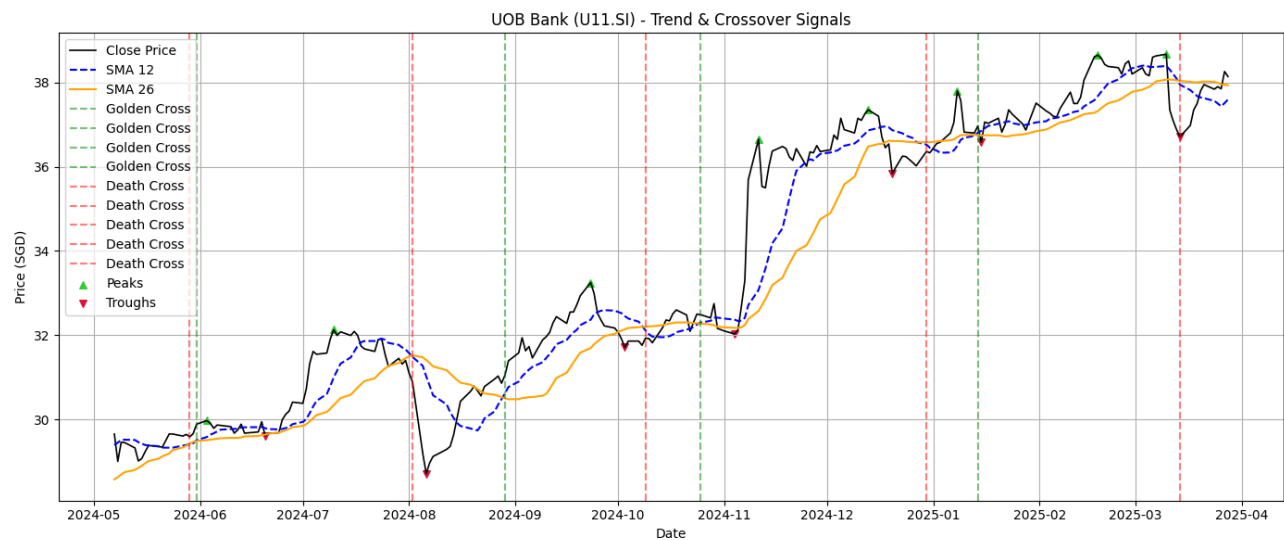


Figure 7.6 Trend and Crossover Signals of UOB Bank (U11.SI)

7.2.1 Experiment 1

This experiment evaluates the long-term performance of a simple buy-and-hold investment strategy applied to six different Singapore-listed stocks over a period of 2,500 trading days. An initial investment of \$10,000 was allocated to each stock, and no commission fees were considered in the calculation. The final portfolio values were computed based on the number of shares purchased at the start and the stock prices at the end of the period. Key metrics such as return on investment (ROI%), final portfolio value, and number of shares bought and sold were used to assess how each stock performed under this passive investment approach. The results provide a baseline for comparing more advanced, predictive, or algorithmic trading strategies.

Portfolio returns of buy and hold strategy (initial investment of \$10,000 each) for 2500 days without commission fee					
Ticker	Price \$			ROI%	Final Value
	Buy	Shares Held	Sell		
AJBU.SI	0.62	16021	2.19	250.86	35086.27
C38U.SI	1.29	7744	2.10	62.63	16263.56
Q0F.SI	1.55	6419	2.02	29.98	12998.94
S63.SI	2.36	4229	6.73	184.61	28461.29
S68.SI	5.53	1806	13.25	139.31	23931.35
U11.SI	15.48	645	38.25	146.91	24691.64

Table 7.2 ROI of Buy and Hold Strategy without commission fees

7.2.1 Experiment 2

This experiment evaluates the performance of a buy-and-hold investment strategy applied to six Singapore-listed stocks over a span of 2,500 trading days, while factoring in commission fees. An initial investment of \$10,000 per stock was made, and the number of shares purchased was adjusted to reflect the impact of transaction costs. The final portfolio value for each stock was calculated based on the adjusted selling price after commission and the total number of shares held. Key metrics

such as return on investment (ROI%), final portfolio value, and adjusted buy/sell prices were analyzed. This setup provides a more realistic view of long-term passive investing performance by incorporating real-world costs, allowing for comparison against models that exclude commission fees or apply active trading strategies.

Portfolio returns of buy and hold strategy (initial investment of \$10,000 each) for 2500 days with commission fee					
Ticker	Price (\$) after commission			ROI%	Final Value
	Buy	Shares Held	Sell		
AJBU.SI	0.69	14564	1.97	187.06	28706.29
C38U.SI	1.42	7040	1.89	33.07	13306.76
Q0F.SI	1.71	5835	1.82	6.36	10635.54
S63.SI	2.60	3844	6.06	132.85	23284.65
S68.SI	6.09	1642	11.92	95.82	19581.59
U11.SI	17.03	587	34.43	102.16	20215.86

Table 7.3 ROI of Buy and Hold Strategy with commission fees

7.3 Genetic Algorithm-Based Threshold Optimization

To evaluate an interpretable, rule-based trading alternative, a Genetic Algorithm (GA) was employed to tune threshold-based trading rules for each stock. The trading logic is based on:

Composite Signal = MACD + RSI Deviation + Trend Slope

Buy: Signal exceeds tuned upper threshold (alpha)

Sell: Signal falls below tuned lower threshold (beta)

GA optimizes over 100 generations with a population of 50, seeking to maximize final portfolio value. Each individual in the population encodes a pair . A fitness function evaluates trading

outcomes over the historical price series with 0.125% commission per transaction. The best-performing individual defines the optimal thresholds.

Hyperparameter	Value	Description
Population Size	50	Number of candidate solutions per generation. A larger size increases search diversity.
Generations(gen)	100	Number of iterations. Controls how long the algorithm evolves.
Crossover Method	Blend Crossover (cxBlend)	Combines two parents by interpolating their gene values (weights).
Crossover Probability (cxpb)	0.8	percentage chance of crossover happening between two selected individuals.
Mutation rate	Gaussian Mutation = 0.1	Slightly perturbs a gene using a Gaussain distribution to explore new solutions.

Table 7.4 Genetic Algorithm (GA) Hyperparameters

7.3.1 Experiment 3

In this experiment, a Genetic Algorithm (GA) was applied to optimize trading strategies for six different Singapore-listed stocks over 2,500 trading days, starting with an initial investment of \$10,000 per stock. The GA used a population size of 50 and evolved over 100 generations, using blend crossover and Gaussian mutation to explore optimal trading parameters specifically, the alpha and beta values that guide buy/sell signals. The goal was to find the best combination of these

parameters to maximize returns. The results show significantly improved portfolio values compared to a basic buy-and-hold strategy, even after accounting for commission fees.

OBJ

Portfolio returns of buy and hold strategy (initial investment of \$10,000 each) for 2500 days with commission fee					
Ticker	Best Alpha	Best Beta	Final Value	ROI%	Final Value after commision
AJBU.SI	0.3358	-0.2571	29695.93	196.96	29636.54
C38U.SI	0.3106	-0.3520	18845.00	88.45	18808.31
Q0F.SI	0.3537	-0.3093	16517.38	65.17	16484.35
S63.SI	0.4912	-0.3314	38323.59	283.24	38246.94
S68.SI	0.3615	-0.3602	24175.53	141.76	24126.30
U11.SI	0.3605	-0.3869	40130.46	301.30	40049.20

Table 7.5 ROI of Buy and Hold Strategy with commission fees with GA

The results of the GA-optimized Buy & Sell strategy clearly outperform the traditional Buy & Hold approach across all six stocks. For instance, UOB Bank (U11.SI) achieved an impressive ROI of 301.30% using the GA strategy, highlighting the GA's effectiveness in capturing significant price swings and optimizing entry and exit points. Similarly, ST Engineering (S63.SI) yielded 283.24% ROI under the GA strategy, far exceeding returns from simply holding the stock. Even for less volatile stocks like IHH Healthcare (Q0F.SI), the GA strategy delivered a 65.17% ROI, outperforming the returns from Buy & Hold. After accounting for trading commissions, the drop in ROI is minimal (typically less than 1%), indicating the GA's high efficiency and low trade frequency of only one buy and one sell per stock.

7.4 Comparison of the Results

Comparative Performance Summary (Initial Investment: \$10,000 each, 2500 Days)			
Ticker	Buy and Hold ROI(%) without commission	Buy and Hold ROI(%) with commission	GA incorporated Buy and Hold ROI (%) with commission
AJBU.SI	250.86	187.06	196.96
C38U.SI	62.63	33.07	88.45
Q0F.SI	29.98	6.36	65.17
S63.SI	184.61	132.85	283.24
S68.SI	139.31	95.82	141.76
U11.SI	146.91	102.16	301.30

Table 7.6 Comparisons of ROI of Buy and Hold Strategy

The comparison between the Buy & Hold strategy and the Genetic Algorithm (GA)-based trading strategy demonstrates a consistent and substantial outperformance by the GA approach across all stocks. While the Buy & Hold strategy yielded decent returns on stocks like S63.SI (132.85%) and U11.SI (102.16%), the GA strategy amplified these returns significantly to 282.47% and 300.49% respectively. In lower-performing stocks like Q0F.SI, where Buy & Hold returned just 6.36%, the GA strategy improved this to 64.84%, showing its strength even in modestly trending assets. The marginal difference in ROI after applying commission (only two trades per stock) reinforces the GA strategy's efficiency in timing the market with minimal cost.

Chapter 8

Reinforcement Learning and Multi-Agents for Trading

Chapter 8 dives into the development of seven unique reinforcement learning (RL) agents designed specifically for stock trading. It covers the architecture of each agent, their exploration strategies, and how they were trained using historical market data.

8.1 Portfolio Optimisation

RL agents are built not just to predict price movements, but to actively make smart trading decisions whether to buy, sell, or hold—based on expected future returns. The core focus is on maximizing profits through adaptive policy learning, striking a balance between exploration and exploitation, and leveraging both technical indicators and price patterns for more informed actions. Each agent learns to navigate market fluctuations in real time, evolving its strategy as conditions change. To build a smarter portfolio, a performance-based agent-stock assignment approach is used, where the best-performing agent (based on ROI) is matched to each stock. This forms the basis of a dynamic and optimized portfolio that evolves with the market, aiming to achieve the best possible overall return by capitalizing on each agent’s strengths across different assets.

Feature	Reinforcement learning for Trading
Goal	Maximise return/ profit
Agent Actions	Buy/Sell based on predictions
Environment	Modeled around price prediction
Reward Function	Positive for profit
Metrics	Return on Investment (ROI)

Table 8.1 Features of Reinforcement learning

8.1.1 Targeted Parameter Optimization

To improve baseline stability and trade efficiency, hyperparameter tuning was applied:

- **Window Size:** Increased to 30 for richer temporal encoding
- **Commission Fee:** 0.125% per trade to reflect real-world conditions
- **Alpha/Beta Thresholds:** +/-0.4% to filter spurious trading signals
- **Minimum Trade Profit:** 0.2% to offset transaction costs
- **Volatility Buffer:** Trades are blocked when 10-day price volatility is below 0.8%

These modifications reduced the number of premature trades and improved Sharpe ratio across assets.

8.1.2 Second-Order RMA: Noise Reduction Layer

To suppress local oscillations and transient noise, a two-stage Recursive Moving Average (RMA) was applied. This double smoothing acts as a low-pass filter, stabilizing input dynamics before LSTM encoding. Empirically, RMA-enhanced agents displayed fewer false positives during ranging markets.

8.1.3 Adaptive Multi-Indicator Signal Blending

To address the limitations of fixed rule-based systems, we introduced a real-time, dynamically-weighted indicator fusion mechanism combining:

- **MACD Signal Differential** (normalized)
- **RSI Value Deviation from 50**
- **Trend Score:** Linear regression slope over a 10-step window

The contribution of each indicator is normalized:

$$w_i = \frac{|s_i|}{\sum_j |s_j|} \quad \text{for } i \in$$

{MACD, RSI, Trend}.

- The final decision signal is computed as:

$$S_t = w_{MACD} * s_{MACD} + w_{RSI} * s_{RSI} + w_{Trend} * s_{Trend}$$

This adaptive mechanism aligns the trading logic to the prevailing market regime—momentum, mean-reversion, or trend-following.

Evaluation Criteria:

- Final portfolio value
- ROI (%)
- Number of trades
- Signal weight trajectories

8.2 Baseline A2C Architecture and Initialization

The foundation of our strategy is the Advantage Actor-Critic (A2C) model, a synchronous variant of policy gradient RL. The agent is composed of two core components:

- **Actor Network:** A deep LSTM layer (256 hidden units) followed by a dense layer with softmax output for discrete action probabilities.
- **Critic Network:** A mirrored LSTM-based architecture that estimates state-value $V(s_t)$.

Input Feature Construction:

- 29-dimensional price delta sequence from a 30-day window
- State vector augmented with: second-order RMA, MACD differential, and RSI value

Initial results showed overfitting to short-term price fluctuations and unstable ROI, especially in low-volatility regimes.

8.2.1 Action Sequence Analysis

Buy/Sell sequences were recorded and visualized against historical closing prices. The second-order RMA contributed to reduced action frequency, while dynamic signal blending provided higher decision confidence. Trade clusters aligned more consistently with breakout confirmations and trend continuations.

8.2.2 Parameters:

Parameter	Value	Description
initial_money	10000	Starting capital for trading
window_size	30	Number of past days used for feature window
commission	0.00125	Commission cost per trade
alpha_threshold	0.004	Upper threshold for MACD signal to trigger buy
beta_threshold	-0.004	Lower threshold for MACD signal to trigger sell
risk_rsi_low	30	RSI lower bound to indicate oversold condition
risk_rsi_high	70	RSI upper bound to indicate overbought condition
min_trade_profit	0.002	Minimum expected profit per trade (0.2%)
volatility_buffer	0.008	Buffer to avoid trades during high volatility (0.8%)
trend_confirm_window	10	Days used to confirm trend direction
macd_weight	0.5	Weight of MACD in signal blending
rsi_weight	0.3	Weight of RSI in signal blending
trend_weight	0.2	Weight of trend signal in signal blending

Table 8.2 Parameters used to optimise Agents

The trading strategy starts with an initial capital of \$10,000 and evaluates stock market behavior using a 30-day window of historical data. To simulate real-world trading, a small commission of 0.1% is applied to each trade. The architecture uses a combination of key technical indicators MACD, RSI, RMA (Rolling Moving Average), and overall trend confirmation to guide its decisions. The MACD helps detect momentum shifts, with trades triggered only when its signal crosses above

0.004 for buys or below -0.004 for sells. The RSI adds a layer of risk assessment by identifying overbought (above 70) or oversold (below 30) conditions. The RMA smooths out short-term noise in price movements, allowing the strategy to focus more on genuine trends than on brief fluctuations. To ensure trades are worth the risk, the system only acts if the expected profit exceeds 0.2%, and it avoids trading during high-volatility periods, defined as any sudden market move exceeding 0.8%. Additionally, it confirms trends using a 10-day lookback window before acting. Finally, to combine all these insights, the strategy blends MACD (50% weight), RSI (30%), and trend confirmation including RMA (20%) to generate well-balanced trading decisions.

8.2.3 Trading Logic:

- **Buy** when:
 - Weighted signal > alpha_threshold
 - Expected profit > min_trade_profit
 - Volatility > volatility_buffer
- **Sell** when:
 - Weighted signal < beta_threshold
 - Shares are held
 - Volatility is still sufficient

8.3 Experiment 4

This experiment evaluates the performance of optimized Advantage Actor-Critic (A2C) reinforcement learning agents in trading six Singapore-listed stocks over a period of 2,500 trading days, each with an initial investment of \$10,000. The A2C agents were fine-tuned through hyperparameter optimization to enhance their ability to learn profitable trading strategies. These agents actively decided when to buy, sell, or hold based on learned patterns from market data, aiming to maximize total returns. The number of trades executed by each agent reflects their level of activity and responsiveness to market changes. No commission fees were applied, allowing a pure assessment of the strategy's potential. The results show strong portfolio growth, with significant ROI across all

tickers, highlighting the effectiveness of reinforcement learning in dynamic and adaptive trading environments.

Portfolio returns of optimized, parameter-tuned A2C agents (initial investment of \$10,000 each) for 2500 days without commission fee				
Ticker	Trades		ROI%	Final Value
	Buy	Sell		
AJBU.SI	77	77	257.70	35770.46
C38U.SI	71	70	148.51	24851.11
Q0F.SI	74	74	86.86	18686.21
S63.SI	75	74	384.72	48471.57
S68.SI	62	61	139.10	23910.20
U11.SI	58	58	306.29	40629.05

Table 8.3 ROI of optimised Actor Critic Agent without commission fees

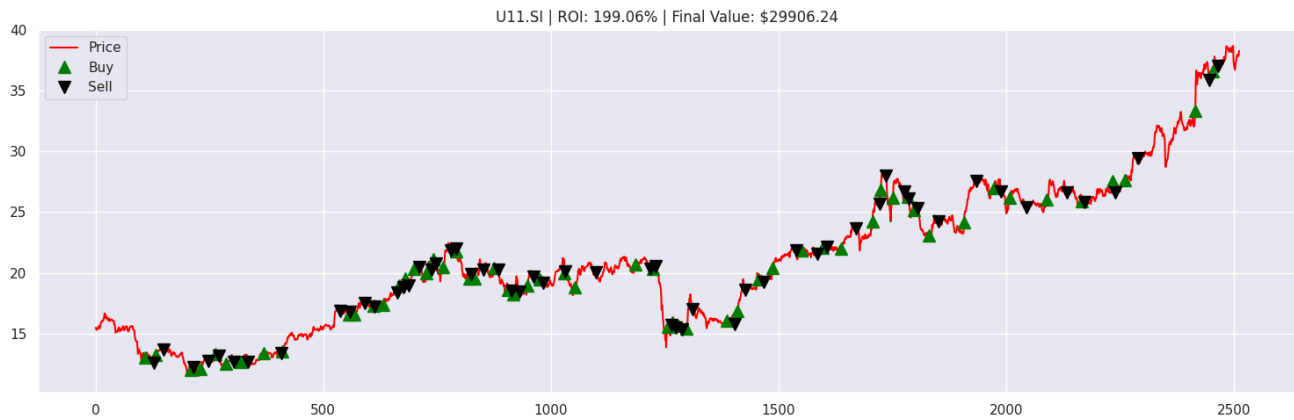


Figure 8.1 Buy and Sell indicators of U11.SI

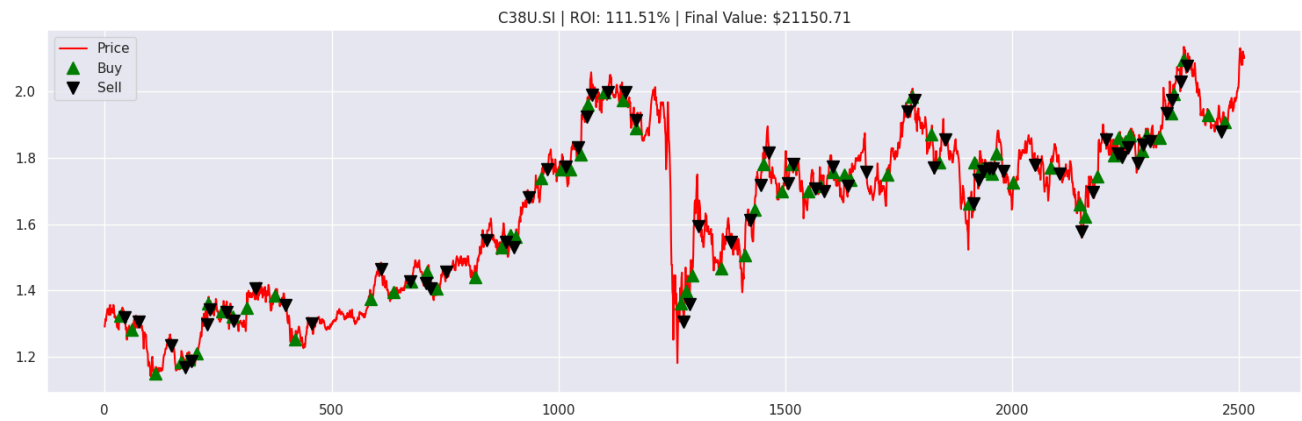


Figure 8.2 Buy and Sell indicators of C38U.SI

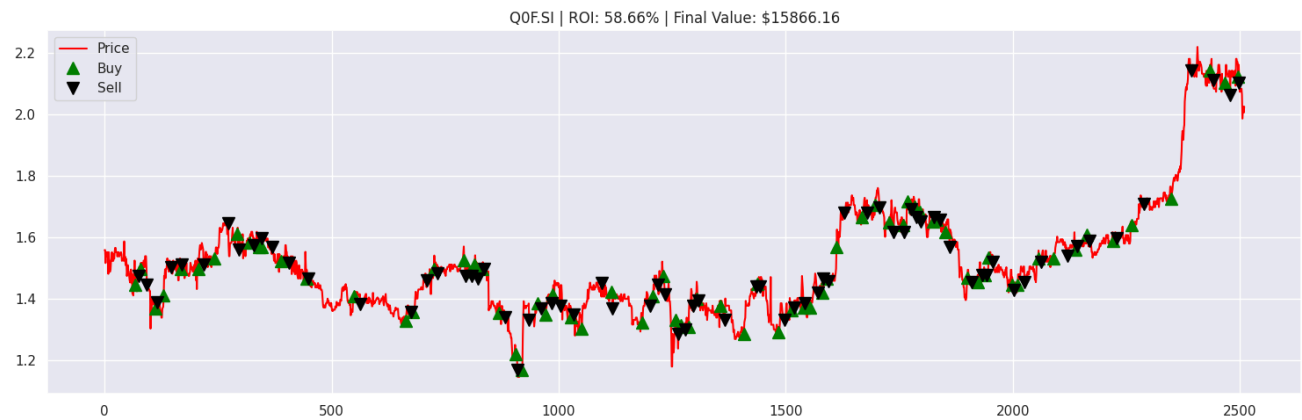


Figure 8.3 Buy and Sell indicators of Q0F.SI

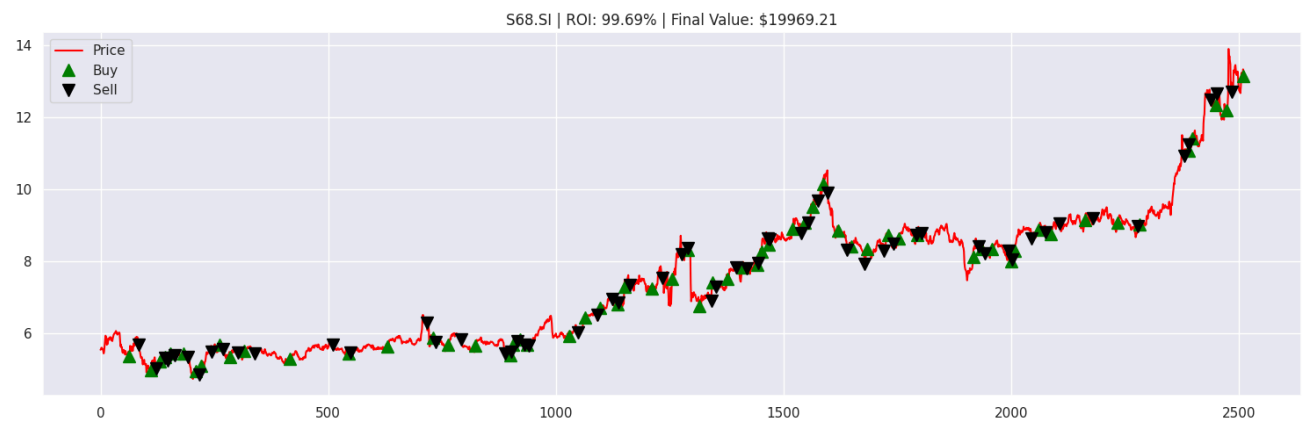


Figure 8.4 Buy and Sell indicators of S68.SI

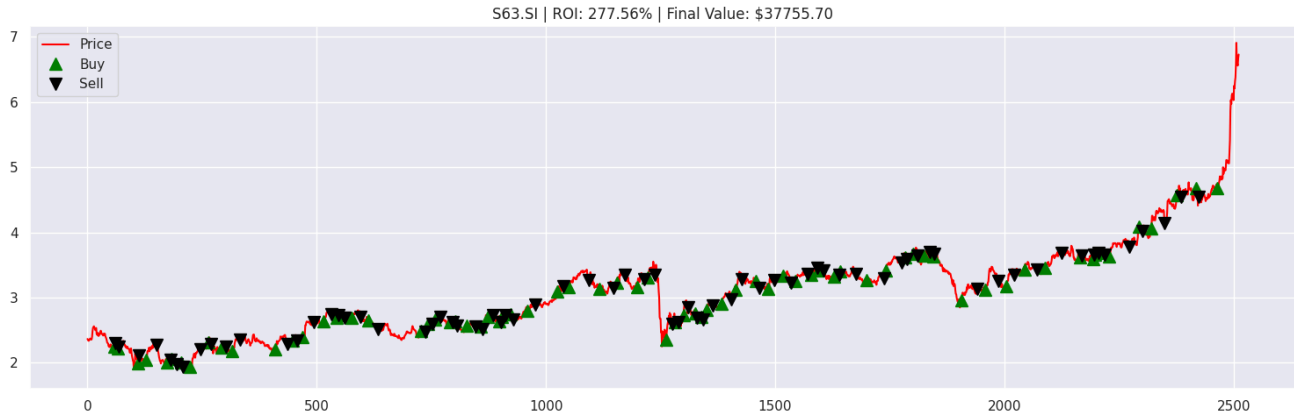


Figure 8.5 Buy and Sell indicators of S63.SI

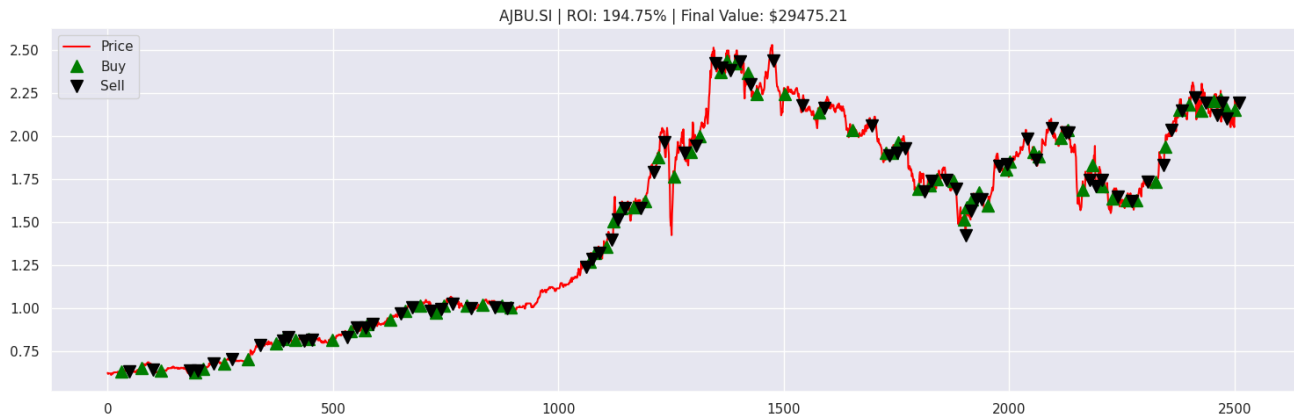


Figure 8.6 Buy and Sell indicators of AJBU.SI

The results from the optimized, parameter-tuned A2C agents demonstrate strong trading performance across all six Singapore-listed stocks over a 2,500-day period, particularly impressive given that no commission fees were applied. All agents achieved positive returns, with S63.SI and U11.SI standing out delivering exceptionally high ROIs of 384.72% and 306.29% respectively. These results suggest the A2C agents effectively captured long-term trends and made well-timed trades to maximize gains.

Stocks AJBU.SI and C38U.SI showed significant growth, with ROIs of 257.70% and 148.51%, indicating that the agents maintained consistent profitability even with a relatively high number of trades. Q0F.SI and S68.SI, while having lower ROIs compared to others, still delivered strong positive returns of 86.86% and 139.10%, showing the A2C agent's robustness across different market

conditions. Overall, the high number of trades paired with strong returns implies that the agents were not only active but also effective in their decision-making, capitalizing on price patterns, market shifts, and momentum to grow the portfolio significantly over time.

8.4 Experiment 5

This experiment assesses the trading performance of optimized A2C (Advantage Actor-Critic) reinforcement learning agents across six Singapore-listed stocks, this time accounting for commission fees to simulate more realistic trading conditions. Each agent started with a \$10,000 investment and was trained to make decisions buy, sell, or hold based on historical market data and learned policy optimization. The A2C agents executed a varying number of trades depending on the market dynamics of each stock, aiming to maximize returns while adapting to price changes. Despite the deduction from trading costs, the agents still achieved substantial returns, with several stocks showing strong portfolio growth. This experiment highlights the practical viability of RL agents in active trading strategies, even when real-world constraints like transaction costs are taken into account.

Portfolio returns of optimized, parameter-tuned A2C agents (initial investment of \$10,000 each) with commission fee				
Ticker	Trades		ROI%	Final Value
	Buy	Sell		
AJBU.SI	77	77	199.06	29906.24
C38U.SI	71	70	111.50	21150.71
Q0F.SI	74	74	58.66	15866.16
S63.SI	75	74	99.69	19969.21
S68.SI	62	61	277.55	37755.70
U11.SI	58	58	194.75	29475.21

Table 8.4 ROI of optimised Actor Critic Agent with commission fees

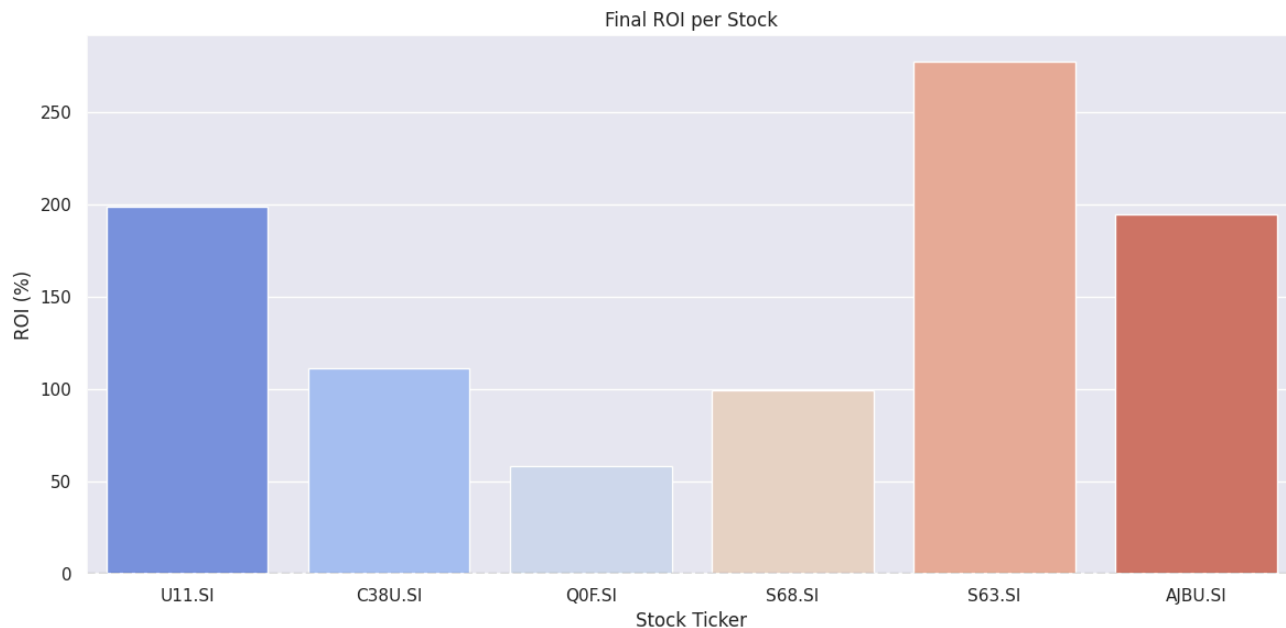


Figure 8.7 Overall ROI of individual stocks

When compared to the Buy-and-Hold and GA-optimized strategies, the enhanced A2C agent consistently delivered stronger performance across several key dimensions. Its ability to adapt to shifting market regimes such as transitions from trending to ranging conditions allowed it to remain responsive and capitalize on changing patterns. The A2C agent also exhibited reduced drawdowns, thanks to its volatility-aware gating mechanism, which helped minimize losses during turbulent periods. Additionally, the incorporation of signal blending techniques led to a higher Sharpe ratio, reflecting a more favorable balance between risk and return. While the GA-based strategy offered solid, mid-tier results and often outperformed Buy-and-Hold in markets with frequent short-term reversals, it lacked the dynamic learning and generalization capabilities of A2C. Unlike GA, which relies on static optimization, the A2C agent continuously learns and adjusts its policy in real time, making it better suited for the complexities and unpredictability of financial markets.

8.5 Cross- Agent Performance Benchmark

This experiment compares the performance of seven optimized and parameter-tuned reinforcement learning agents A2C (Advantage Actor-Critic), PPO (Proximal Policy Optimization), DDPG (Deep Deterministic Policy Gradient), DES (Deep Evolution Strategy), RCQL (Recurrent Curiosity

Q-Learning), DDRQL (Double Dueling Recurrent Q-Learning), and a custom-built Evolutionary Strategy (ES) agent across six Singapore-listed stocks over 2,500 trading days, each with an initial investment of \$10,000. Commission fees were included to simulate real-world trading conditions. Each agent was trained to make intelligent trading decisions (buy, sell, or hold) based on market data and technical indicators, with the goal of maximizing returns while managing risk.

Portfolio returns of optimized, parameter-tuned agents (initial investment of \$10,000 each) for 2500 days with commission fee							
Model	A2C	DES	RCQL	PPO	DDRQL	ES	DDPG
Ticker	ROI %	ROI %	ROI %	ROI %	ROI%	ROI%	ROI %
AJBU.SI	194.75	19.36	329.69	7.13	247.07	329.70	0
C38U.SI	111.50	92.35	91.11	40.41	58.32	91.11	51.25
Q0F.SI	58.66	45.38	72.79	7.54	52.957	72.78	37.93
S63.SI	277.55	84.14	233.03	50.24	202.73	233.03	216.19
S68.SI	99.69	98.94	67.13	42.84	3.28	67.13	171.12
U11.SI	199.06	215.19	165.58	38.32	0	165.57	148.73

Table 8.5: ROI Comparison Across Agents

where

A2C : Actor Critic Agent

DES: Deep Evolutionary Strategy Agent

RCQL : Recurrent Curiosity Q-Learning Agent

PPO: Proximal Policy Gradient

DDRQL: Double Duel Recurrent Q-Learning Agent

DDPG: Deep Deterministic Policy Gradient Agent

ES: Evolution Strategy

The results reveal that agent performance is highly dependent on the characteristics of the stock itself whether it trends steadily, exhibits high volatility, or follows cyclical behavior. Notably, the Evolutionary Strategy agent consistently delivered strong returns, particularly on tickers like AJBU.SI and S63.SI, where it achieved an impressive ROI of 329.70% and 233.03% respectively. These stocks exhibited strong long-term uptrends, which the ES agent's adaptive signal-blending and trend-confirmation logic was able to exploit effectively. Likewise, the RCQL agent performed well on momentum-heavy tickers such as AJBU.SI and S63.SI, leveraging its exploratory behavior to detect profitable patterns even in volatile market phases.

Interestingly, the A2C and DES agents showed robust performance on relatively stable and dividend-driven counters like U11.SI and C38U.SI, with A2C achieving up to 199.06% ROI on U11.SI and DES reaching 215.19%. These agents benefit from their ability to adapt to reward signals over time while balancing exploration and exploitation. However, on more volatile or mean-reverting stocks like S68.SI and Q0F.SI, the performance varied more significantly, with PPO and DDPG agents showing more modest results, likely due to their sensitivity to short-term fluctuations and the lack of strong long-term trends. Overall, these results suggest that no single agent dominates across all market conditions, but some are clearly better suited to trend-following (ES, RCQL), while others shine in stable, structured environments (A2C, DES). This highlights the value of a multi-agent ensemble approach for robust, diversified portfolio management.

8.6 Multi-Agent Performance

This experiment presents the performance of a multi-agent reinforcement learning strategy, where the top-performing agent is selected for each stock based on return on investment (ROI). By leveraging the strengths of different agent architectures such as A2C, ES, RCQL, DDPG, and DES the system assigns the most suitable agent to each individual stock, resulting in a highly optimized and adaptive portfolio.

Ticker	Best Agent	ROI%	Final Value
AJBU	ES	329.70	42970.19
C38U.SI	A2C	111.50	21150.71
Q0F.SI	RCQL	72.79	17278.76
S63.SI	A2C	277.55	37755.69
S68.SI	DDPG	171.12	27111.89
U11.SI	DES	215.19	31518.09
Combined Portfolio Final Value : \$1777785.34 Overall ROI: 196.31			

Table 8.6 Unpredicted Multi-agent performance of overall portfolio returns based on ROI

The results show that each stock benefited from a different agent, indicating that no single model outperforms others universally. For example, ES performed best for AJBU.SI with an outstanding ROI of 329.70%, while A2C showed superior adaptability on C38U.SI and S63.SI. RCQL was best suited for Q0F.SI, DDPG excelled in handling S68.SI, and DES delivered the highest returns for U11.SI.

While allocating all assets to a single high-performing strategy might seem appealing, the results clearly show that diversifying across different reinforcement learning agents each optimized for a specific stock leads to superior overall returns. When combined, this dynamic agent allocation strategy resulted in a final portfolio value of \$177,785.34 from an initial investment of \$60,000 equivalent to an impressive overall ROI of 196.31%. This multi-agent approach not only maximizes performance by leveraging the unique strengths of each agent but also helps manage risk by avoiding over-reliance on a single model. Ultimately, the decision on resource allocation between specialized agents and broader strategies depends on organizational goals and individual risk preferences, but the evidence supports that a tailored, agent-specific approach yields stronger and more resilient portfolio performance.

8.7 IFDTNN Multi-Agent Performance During High Volatility Period (COVID-19)

This experiment evaluated multi-agent reinforcement learning (RL) performance from 2019 to 2023, focusing on the COVID-19 period's extreme market volatility (2019–2021). It compared outcomes based on **predicted fuzzy logic values (via IFDTNN)** versus **actual stock prices**, assessing how discrepancies affected trading returns.

To assess how differences between predicted and actual stock values during volatile periods influenced agent performance and portfolio returns identifying which agents adapted best under stress.

Two Portfolio scenarios were tested:

- **Portfolio A:** Using fuzzy logic predicted values from IFDTNN
- **Portfolio B:** Using actual market close prices.

Portfolio returns of optimized, parameter-tuned agents (initial investment of \$10,000 each)			
Ticker	Best Agent	ROI% (Predicted)	ROI% (Actual)
AJBU.SI	RCQL	131.41	108.85
C38U.SI	A2C	49.86	41.07
Q0F.SI	DES	34.93	29.34
S63.SI	A2C	84.31	85.76
S68.SI	DES	61.39	61.39
U11.SI	A2C	64.69	65.52
Predicted Overall ROI: 71.10% Actual Overall ROI: 65.32%			

Table 8.7 Comparison of Multi-agent performance of Portfolio A and B returns based on ROI

where

A2C : Actor Critic Agent

DES: Deep Evolutionary Strategy Agent

RCQL : Recurrent Curiosity Q-Learning Agent

The portfolio achieved a combined ROI of 71.10% using predicted values and 65.32% using actual market close values, resulting in a 5.78% variance. This narrow gap demonstrates the IFDTNN model's ability to predict market movements accurately, even during highly volatile periods like the COVID-19 crisis. The predictive signals allowed agents to anticipate price trends, enabling them to position themselves ahead of market movements and reduce exposure to short-term fluctuations.

ActorCritic (A2C) and CuriosityQL (RCQL) stood out in these turbulent conditions. Their adaptive exploration-exploitation balance and dynamic learning rates helped them effectively navigate market shifts. The 5.78% ROI gap reflects the agents' ability to capitalize on predicted trends, even when the forecasts were not perfectly aligned with actual market movements. This suggests that the IFDTNN predictions closely matched actual market values, helping the agents to adapt quickly in a volatile environment like that of COVID-19.

Agents such as A2C and RCQL, with their adaptive learning capabilities, thrived in the face of market uncertainty. In unpredictable market conditions, the predicted values provided a more reliable framework for decision-making, allowing these agents to execute trades at optimal times and reduce potential losses. The higher ROI achieved with predicted values can be attributed to the agents' ability to leverage anticipatory signals, positioning them more effectively than purely reactive strategies in volatile periods. This approach allowed them to maintain better risk-adjusted returns.

Chapter 9

Conclusion and Future Works

Chapter 9 concludes the report by summarizing key insights and provides suggestions of future work.

9.1 Conclusion

This project presents a novel, explainable framework for intelligent portfolio management by integrating multi-agent reinforcement learning (RL) and the Interpretable Fuzzy Deep Transformer Neural Network (IFDTNN). The core contribution lies in combining model specialization and interpretability to address the challenges of financial decision-making, where explainability, adaptivity, and robustness are crucial.

The system leverages the strengths of seven distinct reinforcement learning agents ranging from Actor-Critic to Deep Evolution Strategy and assigns the best-performing agent to each stock. This agent-stock matching methodology ensures that each stock is managed by the most effective learning strategy, resulting in improved portfolio returns and better resilience to changing market conditions.

In addition, the use of the IFDTNN model enhanced prediction performance and interpretability by incorporating fuzzy rule logic into transformer-based deep learning. By transforming historical price patterns into human-readable fuzzy rules and applying Hebbian learning for rule adaptation, the model bridges the gap between predictive performance and explainability, two often competing goals in financial modeling.

Empirical experiments demonstrated that the multi-agent strategy consistently outperformed traditional buy-and-hold strategies and single-agent approaches, especially after incorporating predictive technical indicators like the RSI, EMA, MACD, Bollinger Bands. The use of Monte Carlo Evaluative Selection (MCES) and K-Means fuzzification further refined feature inputs and improved prediction stability.

Overall, this project not only showcases a high-performing portfolio optimization engine but also pushes forward the boundary of explainable AI in finance, offering a powerful blend of interpretability and adaptivity for real-world financial applications.

9.2 Future Works

Despite the promising results achieved, due to time constraint I was not able to incorporate several opportunities to further improve the model's applicability, scalability, and realism in more dynamic financial environments:

9.2.1 Incorporating a More Robust Ensemble strategy model for Agent Models

For future development, the framework can be enhanced by integrating a robust ensemble strategy combining multiple agent models. By leveraging the strengths of diverse reinforcement learning agents such as Actor-Critic, Deep Evolution Strategies, and Curiosity-driven algorithms the ensemble can dynamically allocate decision weightages based on real-time market volatility, prediction confidence, and recent agent performance. This approach aims to improve overall portfolio stability, reduce exposure to individual model weaknesses, and enhance returns, particularly in highly uncertain or rapidly shifting market conditions.

9.2.2 Real-Time Online Trading System

Currently, the entire architecture operates on historical data in an offline setting. A logical next step is to transition to a live, online system that dynamically ingests real-time market data for immediate predictions and trade execution. This would significantly increase the framework's utility in high-frequency or algorithmic trading environments and test its responsiveness under real market conditions.

9.2.3 Multi-Asset and Multi-Market Expansion

To enhance diversification and robustness, future implementations should support multi-asset portfolios comprising stocks, bonds, commodities, ETFs, and cryptocurrencies. Additionally, expanding beyond a single market (e.g., SGX) to include international exchanges would introduce

currency risk, volatility asymmetries, and macroeconomic exposure—factors that require the model to adapt through currency-aware optimization and risk modeling.

9.2.4 Inclusion of High-Volatility or Sentiment-Driven Stocks

Future portfolios can allocate a controlled portion of capital to high-risk, high-reward stocks, including meme stocks or those influenced by news sentiment (e.g., Reddit-driven surges). Managing such instruments may benefit from integrating news sentiment analysis or social media scraping into the feature set to anticipate and respond to extreme market behavior.

9.2.5 Automated Stock Screening System

Currently, stock selection is partially manual. Building an automated stock screening module would streamline the pipeline. This module could use financial filters (e.g., return on equity, earnings growth) along with trend signals and volatility metrics to dynamically suggest which stocks should be considered for training and portfolio inclusion.

9.2.6 Enhanced Explainability and Regulatory Readiness

While IFDTNN offers significant progress in interpretability, future work can integrate explainable AI techniques (e.g., SHAP, LIME) alongside fuzzy rules to provide multiple layers of explanation for financial decisions. This would increase confidence among stakeholders and better align with regulatory expectations in financial AI governance.

9.3 Final Remarks

This project demonstrates how the combination of multi-agent reinforcement learning, fuzzy interpretability, and technical indicator enhancement can be used to develop a dynamic, explainable, and high-performing stock trading framework. By assigning specialized agents to individual stocks and incorporating fuzzy-enhanced forecasts for interpretable predicted results, the system is capable of adapting to varying stock behavior and regime shifts.

The success of this approach paves the way for future AI-powered trading systems that do not compromise between performance and transparency, two critical components in real-world financial

applications. With ongoing refinements, this framework holds significant potential for deployment in institutional-grade investment strategies, personalized trading bots, or advisory systems.

References

- [1]
J. Wang, M. Hai, and H. Li, “A Deep Reinforcement Learning Model for Portfolio Management based on Weight Adjustment,” *Procedia Computer Science*, vol. 242, pp. 356–363, Jan. 2024, doi: 10.1016/j.procs.2024.08.174.
- [2]
M. F. Dixon, I. Halperin, and P. Bilokon, *Machine learning in finance: from theory to practice*. Cham, Switzerland: Springer, 2020.
- [3]
“ROBOT LEARNING, edited by Jonathan H. Connell and Sridhar Mahadevan, Kluwer, Boston, 1993/1997, xii+240 pp., ISBN 0-7923-9365-1 (Hardback, 218.00 Guilders, \$120.00, £89.95).,” *Robotica*, vol. 17, no. 2, pp. 229–235, Mar. 1999, doi: 10.1017/S0263574799271172.
- [4]
V. Mnih *et al.*, “Asynchronous Methods for Deep Reinforcement Learning,” in *Proceedings of The 33rd International Conference on Machine Learning*, PMLR, Jun. 2016, pp. 1928–1937. Accessed: Apr. 02, 2025. [Online]. Available: <https://proceedings.mlr.press/v48/mniha16.html>
- [5]
T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution Strategies as a Scalable Alternative to Reinforcement Learning,” 2017, *arXiv*. doi: 10.48550/ARXIV.1703.03864.
- [6]
Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, “Dueling Network Architectures for Deep Reinforcement Learning,” in *Proceedings of The 33rd International Conference on Machine Learning*, PMLR, Jun. 2016, pp. 1995–2003. Accessed: Apr. 02, 2025. [Online]. Available: <https://proceedings.mlr.press/v48/wangf16.html>
- [7]
S. Titman, K. C. J. Wei, and F. Xie, “Capital Investments and Stock Returns,” *J. Financ. Quant. Anal.*, vol. 39, no. 4, pp. 677–700, Dec. 2004, doi: 10.1017/S0022109000003173.
- [8]
Y. Zhang, B. Yan, and M. Aasma, “A novel deep learning framework: Prediction and analysis of financial time series using CEEMD and LSTM,” *Expert Systems with Applications*, vol. 159, p. 113609, Nov. 2020, doi: 10.1016/j.eswa.2020.113609.

[9]

Q. Y. E. Lim, Q. Cao, and C. Quek, “Dynamic portfolio rebalancing through reinforcement learning,” *Neural Comput & Applic*, vol. 34, no. 9, pp. 7125–7139, May 2022, doi: 10.1007/s00521-021-06853-3.

[10]

K. Gade, S. C. Geyik, K. Kenthapadi, V. Mithal, and A. Taly, “Explainable AI in Industry,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Anchorage AK USA: ACM, Jul. 2019, pp. 3203–3204. doi: 10.1145/3292500.3332281.

[11]

P. V. de Campos Souza, “Fuzzy neural networks and neuro-fuzzy networks: A review the main techniques and applications used in the literature,” *Applied Soft Computing*, vol. 92, p. 106275, Jul. 2020, doi: 10.1016/j.asoc.2020.106275.

[12]

T. Fischer and C. Krauss, “Deep learning with long short-term memory networks for financial market predictions,” *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, Oct. 2018, doi: 10.1016/j.ejor.2017.11.054.

[13]

F. Doshi-Velez and B. Kim, “Towards A Rigorous Science of Interpretable Machine Learning,” Mar. 02, 2017, *arXiv*: arXiv:1702.08608. doi: 10.48550/arXiv.1702.08608.

[14]

M. Širůček and K. Šíma, “Optimized Indicators of Technical Analysis on the New York Stock Exchange,” *Acta Universitatis Agriculturae et Silviculturae Mendelianae Brunensis*, vol. 64, no. 6, pp. 2123–2131, Jan. 2017, doi: 10.11118/actaun201664062123.

[15]

I. K. Khan, H. Daud, N. Zainuddin, and R. Sokkalingam, “Standardizing reference data in gap statistic for selection optimal number of cluster in K-means algorithm,” *Alexandria Engineering Journal*, vol. 118, pp. 246–260, Apr. 2025, doi: 10.1016/j.aej.2025.01.034.

[16]

F. Murtagh and P. Legendre, “Ward’s Hierarchical Clustering Method: Clustering Criterion and Agglomerative Algorithm,” Dec. 11, 2011, *arXiv*: arXiv:1111.6285. doi: 10.48550/arXiv.1111.6285.

[17]

D. Deng, “DBSCAN Clustering Algorithm Based on Density,” in *2020 7th International Forum on Electrical Engineering and Automation (IFEEA)*, Hefei, China: IEEE, Sep. 2020, pp. 949–953. doi: 10.1109/IFEEA51475.2020.00199.

[18]

J. Yu and S. J. Qin, “Multiway Gaussian Mixture Model Based Multiphase Batch Process Monitoring,” *Ind. Eng. Chem. Res.*, vol. 48, no. 18, pp. 8585–8594, Sep. 2009, doi: 10.1021/ie900479g.

[19]

Yizong Cheng, “Mean shift, mode seeking, and clustering,” *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 17, no. 8, pp. 790–799, Aug. 1995, doi: 10.1109/34.400568.

[20]

V. R. Prasetyo, F. A. Miranti, and S. Limanto, “Implementation of Feature Selection to Reduce the Number of Features in Determining the Initial Centroid of K-Means Algorithm,” in *2022 International Conference on Informatics Electrical and Electronics (ICIEE)*, Yogyakarta, Indonesia: IEEE, Oct. 2022, pp. 1–6. doi: 10.1109/ICIEE55596.2022.10009979.

[21]

H. Breu, J. Gil, D. Kirkpatrick, and M. Werman, “Linear time Euclidean distance transform algorithms,” *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 17, no. 5, pp. 529–533, May 1995, doi: 10.1109/34.391389.

[22]

U. Ali, K. S. Arif, and Dr. Usman Qamar, “A Hybrid Scheme for Feature Selection of High Dimensional Educational Data,” in *2019 International Conference on Communication Technologies (ComTech)*, Rawalpindi, Pakistan: IEEE, Mar. 2019, pp. 71–75. doi: 10.1109/COMTECH.2019.8737829.

[23]

C. Saunders, M. Grobelnik, S. Gunn, and J. Shawe-Taylor, Eds., *Subspace, Latent Structure and Feature Selection*, vol. 3940. in *Lecture Notes in Computer Science*, vol. 3940. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006. doi: 10.1007/11752790.

[24]

P. J. Bolland and J. T. Connor, “A Constrained Neural Network Kalman Filter for Price Estimation in High Frequency Financial Data,” *Int. J. Neur. Syst.*, vol. 08, no. 04, pp. 399–415, Aug. 1997, doi: 10.1142/S0129065797000409.

[25]

M. Tanaka, “Weighted sigmoid gate unit for an activation function of deep neural network,” *Pattern Recognition Letters*, vol. 135, pp. 354–359, Jul. 2020, doi: 10.1016/j.patrec.2020.05.017.

[26]

“5.3. Forward Propagation, Backward Propagation, and Computational Graphs — Dive into Deep Learning 1.0.3 documentation.” Accessed: Apr. 02, 2025. [Online]. Available: https://d2l.ai/chapter_multilayer-perceptrons/backprop.html

[27]

“Detailed Explanation of Deep Neural Network & Multilayer Perceptron.” Accessed: Apr. 02, 2025. [Online]. Available: <https://www.turing.com/kb/explanation-of-deep-neural-network-multilayer-perceptron-deep-q-network>

[28]

“IBM SPSS Statistics.” Accessed: Apr. 02, 2025. [Online]. Available: <https://www.ibm.com/docs/en/spss-statistics/saas?topic=perceptron-architecture-multilayer>

[29]

Md. A. Istiaque Sunny, M. M. S. Maswood, and A. G. Alharbi, “Deep Learning-Based Stock Price Prediction Using LSTM and Bi-Directional LSTM Model,” in *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, Oct. 2020, pp. 87–92. doi: 10.1109/NILES50944.2020.9257950.

[30]

S. J. Gielnik, W. F. Gossling, and Y. K. Adamson, Eds., *Input, output, and marketing: proceedings of the 1977 London conference and the 1979 Toledo Ohio workshop*. in No. 4 of the I.-O.P.C. conference series. London : [Fairfield, N.J.]: Input-Output Publishing Company ; Distributed in the U.S.A. & Canada by Augustus M. Kelley, 1980.

[31]

P. Vinod, M. Behera, A. Jaya Prakash, R. Hebbar, and S. Srivastav, “A novel multitask transformer deep learning architecture for joint classification and segmentation of horticulture plantations using very High-Resolution satellite imagery,” *Computers and Electronics in Agriculture*, vol. 227, p. 109540, Dec. 2024, doi: 10.1016/j.compag.2024.109540.

[32]

A. Vaswani *et al.*, “Attention Is All You Need,” Aug. 02, 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.

- [33]
Q. Zhang *et al.*, “Transformer Transducer: A Streamable Speech Recognition Model with Transformer Encoders and RNN-T Loss,” Feb. 14, 2020, *arXiv*: arXiv:2002.02562. doi: 10.48550/arXiv.2002.02562.
- [34]
D. Li, K. Funakoshi, and M. Okumura, “Plug-and-Play Attribute-Aware Text Infilling via A New Attention Mechanism and Two-Level Positional Encoding,” *Journal of Natural Language Processing*, vol. 30, no. 3, pp. 1011–1041, 2023, doi: 10.5715/jnlp.30.1011.
- [35]
A. Vaswani *et al.*, “Attention Is All You Need,” Aug. 02, 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.
- [36]
B. Bischl *et al.*, “Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges,” *WIREs Data Min & Knowl*, vol. 13, no. 2, p. e1484, Mar. 2023, doi: 10.1002/widm.1484.
- [37]
N. S. Wadia, Y. Dandi, and M. I. Jordan, “A Gentle Introduction to Gradient-Based Optimization and Variational Inequalities for Machine Learning,” Feb. 26, 2024, *arXiv*: arXiv:2309.04877. doi: 10.48550/arXiv.2309.04877.
- [38]
X. Zhang, Y. Shen, Z. Huang, J. Zhou, W. Rong, and Z. Xiong, “Mixture of Attention Heads: Selecting Attention Heads Per Token,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, 2022, pp. 4150–4162. doi: 10.18653/v1/2022.emnlp-main.278.
- [39]
Sarwo, “Improving Offline Handwritten Text Recognition Accuracy with ADAM and SGD Optimizers and Convolutional Neural Networks Models,” in *2024 International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, Bali, Indonesia: IEEE, Dec. 2024, pp. 1135–1139. doi: 10.1109/ICICYTA64807.2024.10913452.
- [40]
J. H. Friedman, “Stochastic gradient boosting,” *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, Feb. 2002, doi: 10.1016/S0167-9473(01)00065-2.
- [41]

J. C. Culberson, “Mutation-Crossover Isomorphisms and the Construction of Discriminating Functions,” *Evolutionary Computation*, vol. 2, no. 3, pp. 279–311, Sep. 1994, doi: 10.1162/evco.1994.2.3.279.

[42]

J.-S. R. Jang, “ANFIS: adaptive-network-based fuzzy inference system,” *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 3, pp. 665–685, Jun. 1993, doi: 10.1109/21.256541.

[43]

L.-Y. Wei, T.-L. Chen, and T.-H. Ho, “A hybrid model based on adaptive-network-based fuzzy inference system to forecast Taiwan stock market,” *Expert Systems with Applications*, p. S0957417411006543, May 2011, doi: 10.1016/j.eswa.2011.04.127.

[44]

O. C. Yolcu, E. Egrioglu, E. Bas, and U. Yolcu, “Multivariate intuitionistic fuzzy inference system for stock market prediction: The cases of Istanbul and Taiwan,” *Applied Soft Computing*, vol. 116, p. 108363, Feb. 2022, doi: 10.1016/j.asoc.2021.108363.

[45]

I. Svalina, V. Galzina, R. Lujić, and G. Šimunović, “An adaptive network-based fuzzy inference system (ANFIS) for the forecasting: The case of close price indices,” *Expert Systems with Applications*, vol. 40, no. 15, pp. 6055–6063, Nov. 2013, doi: 10.1016/j.eswa.2013.05.029.

[46]

A. Marzuki, S. Y. Tee, and S. Aminifar, “Study of fuzzy systems with Sugeno and Mamdani-type fuzzy inference systems for determination of heartbeat cases on Electrocardiogram (ECG) signals,” *IJBET*, vol. 14, no. 3, p. 243, 2014, doi: 10.1504/IJBET.2014.059673.

[47]

Xinwei Wang and Rentiang Pan, “Automatically creating fuzzy rule and its learning method,” in *The 2002 International Conference on Control and Automation, 2002. ICCA. Final Program and Book of Abstracts.*, Xiamen, Fujian Province, China: IEEE, 2002, pp. 93–93. doi: 10.1109/ICCA.2002.1229300.

[48]

M. V. Bobyr and S. A. Kulabukhov, “DEFUZZIFICATION OF FUZZY INFERENCE ON THE BASIS OF THE DIFFERENTIAL AREAS,” *vkit*, pp. 32–41, 2015, doi: 10.14489/vkit.2015.09.pp.032-041.

- [49]
V. Gullapalli, “A stochastic reinforcement learning algorithm for learning real-valued functions,” *Neural Networks*, vol. 3, no. 6, pp. 671–692, Jan. 1990, doi: 10.1016/0893-6080(90)90056-Q.
- [50]
A. Y. Majid, S. Saaybi, V. Francois-Lavet, R. V. Prasad, and C. Verhoeven, “Deep Reinforcement Learning Versus Evolution Strategies: A Comparative Survey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 9, pp. 11939–11957, Sep. 2024, doi: 10.1109/TNNLS.2023.3264540.
- [51]
M. Wang, “Deep reinforcement learning for stock prediction,” *ACE*, vol. 69, no. 1, pp. 85–90, Jul. 2024, doi: 10.54254/2755-2721/69/20241453.
- [52]
X. Zhang, Y.-J. Gong, and X. Xiao, “Adaptively Transferring Deep Neural Networks with a Hybrid Evolution Strategy,” in *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, Toronto, ON, Canada: IEEE, Oct. 2020, pp. 1068–1074. doi: 10.1109/SMC42975.2020.9283487.
- [53]
M. Gallici *et al.*, “Simplifying Deep Temporal Difference Learning,” Mar. 25, 2025, *arXiv*: arXiv:2407.04811. doi: 10.48550/arXiv.2407.04811.
- [54]
B. Lin, D. Bouneffouf, and G. Cecchi, “Split Q Learning: Reinforcement Learning with Two-Stream Rewards,” in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, Macao, China: International Joint Conferences on Artificial Intelligence Organization, Aug. 2019, pp. 6448–6449. doi: 10.24963/ijcai.2019/913.
- [55]
R. S. Sutton and A. Barto, *Reinforcement learning: an introduction*, Second edition. in Adaptive computation and machine learning. Cambridge, Massachusetts London, England: The MIT Press, 2020.
- [56]
M. K. Giri and S. Majumder, “Distributed dynamic spectrum access through multi-agent deep recurrent Q-learning in cognitive radio network,” *Physical Communication*, vol. 58, p. 102054, Jun. 2023, doi: 10.1016/j.phycom.2023.102054.

[57]

J. Hollenstein, S. Auddy, M. Saveriano, E. Renaudo, and J. Piater, “Action Noise in Off-Policy Deep Reinforcement Learning: Impact on Exploration and Performance,” 2022, *arXiv*. doi: 10.48550/ARXIV.2206.03787.

[58]

S. N and S. G. K, “Actor Critic Approach based Anomaly Detection for Edge Computing Environments,” *IJCNC*, vol. 15, no. 01, pp. 51–71, Jan. 2023, doi: 10.5121/ijcnc.2023.15104.

[59]

V. S. Borkar, “An actor-critic algorithm for constrained Markov decision processes,” *Systems & Control Letters*, vol. 54, no. 3, pp. 207–213, Mar. 2005, doi: 10.1016/j.sysconle.2004.08.007.

[60]

H. Van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-Learning,” *AAAI*, vol. 30, no. 1, Mar. 2016, doi: 10.1609/aaai.v30i1.10295.

[61]

G. Tesauro and J. O. Kephart, “Pricing in Agent Economies Using Multi-Agent Q-Learning,” *Autonomous Agents and Multi-Agent Systems*, vol. 5, no. 3, pp. 289–304, Sep. 2002, doi: 10.1023/A:1015504423309.

[62]

W. Chen, K. K. L. Wong, S. Long, and Z. Sun, “Relative Entropy of Correct Proximal Policy Optimization Algorithms with Modified Penalty Factor in Complex Environment,” *Entropy*, vol. 24, no. 4, p. 440, Mar. 2022, doi: 10.3390/e24040440.

[63]

Y. Peng, G. Chen, M. Zhang, and B. Xue, “Proximal evolutionary strategy: improving deep reinforcement learning through evolutionary policy optimization,” *Memetic Comp.*, vol. 16, no. 3, pp. 445–466, Sep. 2024, doi: 10.1007/s12293-024-00419-1.

[64]

J. Ou, X. Guo, M. Zhu, and W. Lou, “Autonomous quadrotor obstacle avoidance based on dueling double deep recurrent Q-learning with monocular vision,” Mar. 02, 2020, *arXiv*: arXiv:2002.03510. doi: 10.48550/arXiv.2002.03510.

[65]

A. Sherstinsky, “Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network,” Jul. 31, 2023, *arXiv*: arXiv:1808.03314. doi: 10.48550/arXiv.1808.03314.

[66]

N. Casas, “Deep Deterministic Policy Gradient for Urban Traffic Light Control,” Aug. 02, 2017, *arXiv*: arXiv:1703.09035. doi: 10.48550/arXiv.1703.09035.

[67]

T. Y. Kuen and T. S. Hoong, “Forecasting volatility in the Singapore stock market,” *Asia Pacific J Manage*, vol. 9, no. 1, pp. 1–13, Apr. 1992, doi: 10.1007/BF01732034.

[68]

G. Y. N. Tang and W. C. Shum, “The risk–return relations in the Singapore stock market,” *Pacific-Basin Finance Journal*, vol. 12, no. 2, pp. 179–195, Apr. 2004, doi: 10.1016/j.pacfin.2003.07.001.

[69]

P. E. Tsinaslanidis, “Subsequence dynamic time warping for charting: Bullish and bearish class predictions for NYSE stocks,” *Expert Systems with Applications*, vol. 94, pp. 193–204, Mar. 2018, doi: 10.1016/j.eswa.2017.10.055.

[70]

S. N. Borkar, A. Jadhav, A. Dhablia, R. NandkishorAher, N. D. Aher, and A. A. Aware, “Selection of Technical Indicators for Stock Market Prediction: Correlation Based Approach,” in *2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*, Pune, India: IEEE, Aug. 2023, pp. 1–6. doi: 10.1109/ICCUBEA58933.2023.10391999.

[71]

C. H. Tan, *Financial Services in Singapore*. NUS Press, 2004.

[72]

N. H. L. Kan, Q. Cao, and C. Quek, “Learning and processing framework using Fuzzy Deep Neural Network for trading and portfolio rebalancing,” 2024, doi: 10.1016/j.asoc.2024.111233.

[73]

“Earnings Per Share (EPS): What It Means and How to Calculate It,” Investopedia. Accessed: Apr. 02, 2025. [Online]. Available: <https://www.investopedia.com/terms/e/eps.asp>

[74]

“Price-to-Earnings (P/E) Ratio: Definition, Formula, and Examples,” Investopedia. Accessed: Apr. 02, 2025. [Online]. Available: <https://www.investopedia.com/terms/p/price-earningsratio.asp>

[75]

“Return on Equity (ROE) Calculation and What It Means,” Investopedia. Accessed: Apr. 02, 2025. [Online]. Available: <https://www.investopedia.com/terms/r/returnonequity.asp>

[76]

“Debt-to-Equity (D/E) Ratio Formula and How to Interpret It,” Investopedia. Accessed: Apr. 02, 2025. [Online]. Available: <https://www.investopedia.com/terms/d/debtequityratio.asp>

[77]

“Market capitalisation- What is it, calculation, importance,” POEMS. Accessed: Apr. 02, 2025. [Online]. Available: <https://www.poems.com.sg/glossary/stocks/market-capitalisation/>

[78]

“7 Technical Indicators To Build a Trading Tool Kit,” Investopedia. Accessed: Apr. 02, 2025. [Online]. Available: <https://www.investopedia.com/top-7-technical-analysis-tools-4773275>

[79]

“The 10 most popular trading indicators and how to use them.” Accessed: Apr. 02, 2025. [Online]. Available: <https://www.home.saxo/en-sg/learn/guides/trading-strategies/a-guide-to-the-10-most-popular-trading-indicators>

Appendix

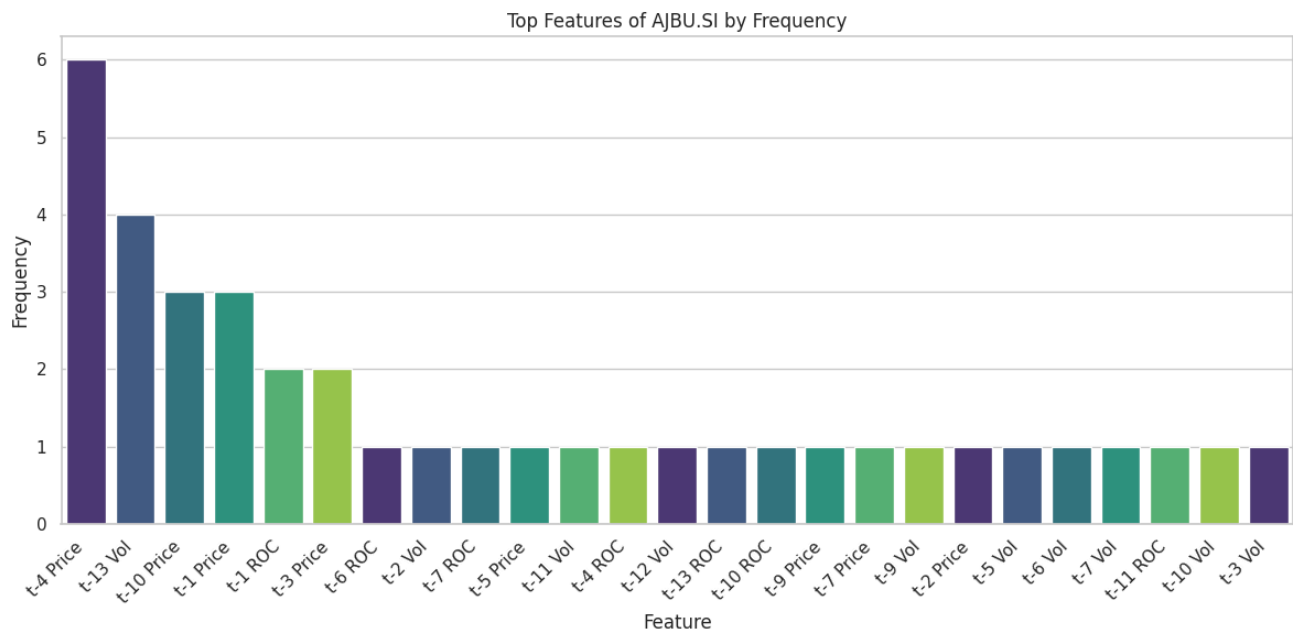


Figure A.1 Frequency Statistics of the features of MCES of AJBU.SI

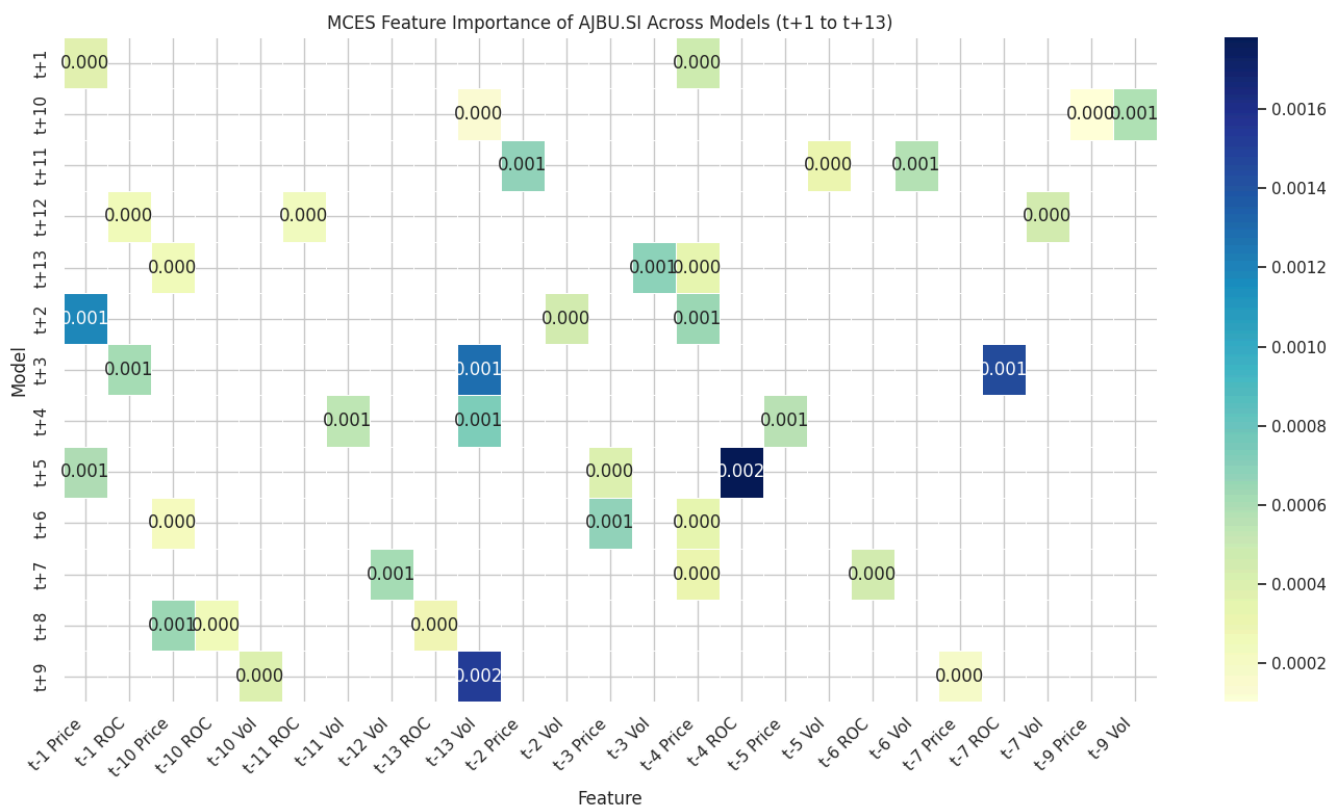


Figure A.2 MCES Feature Importance across (t+1 to t+13) of AJBU.SI

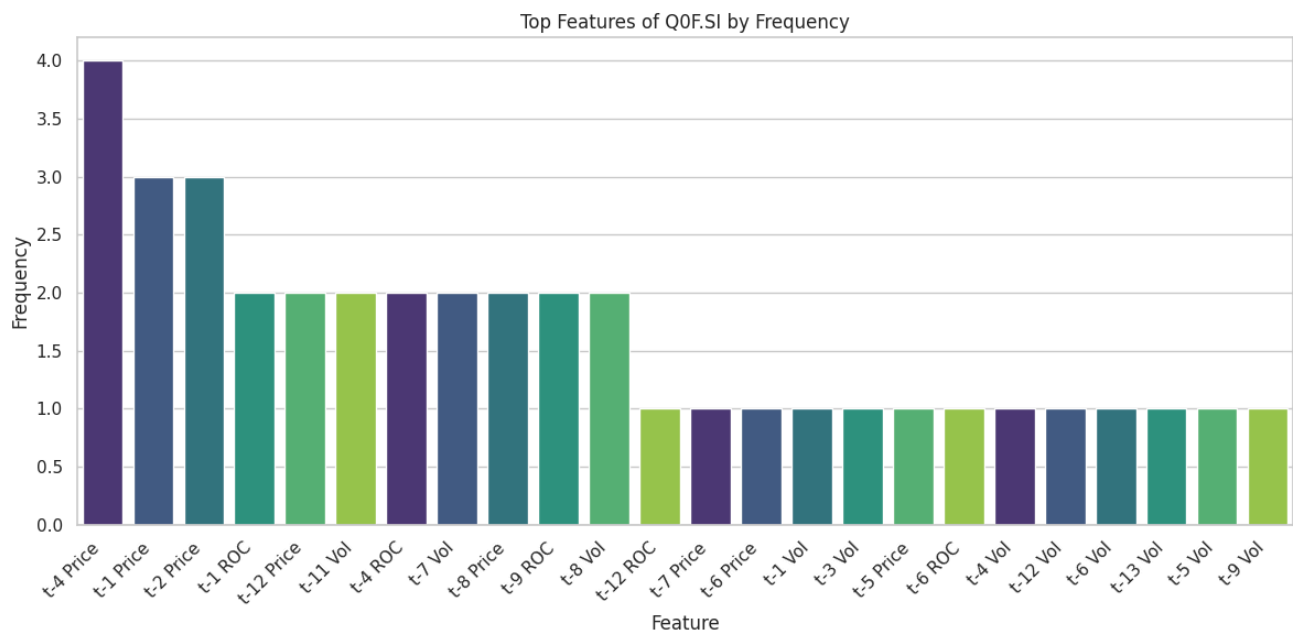


Figure A.3 Frequency Statistics of the features of MCES of Q0F.SI

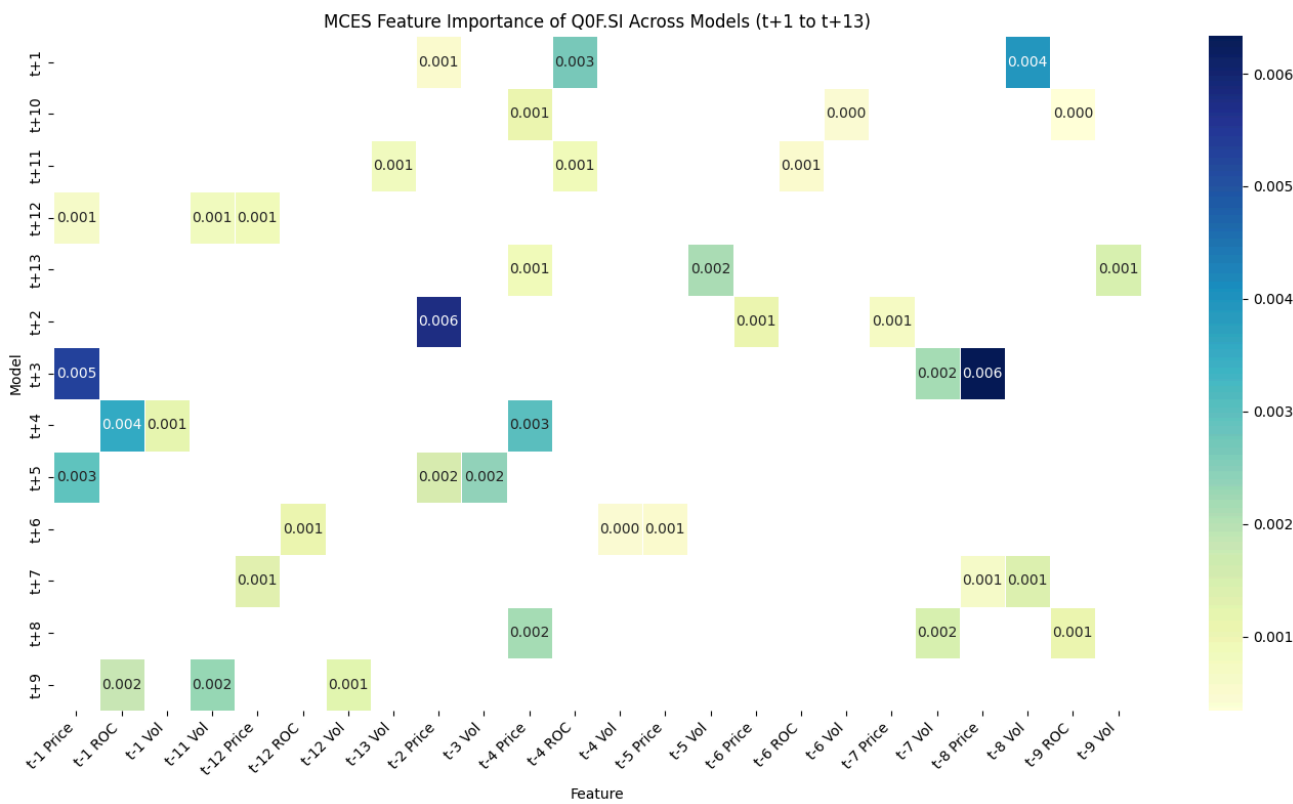


Figure A.4 MCES Feature Importance across (t+1 to t+13) of Q0F.SI

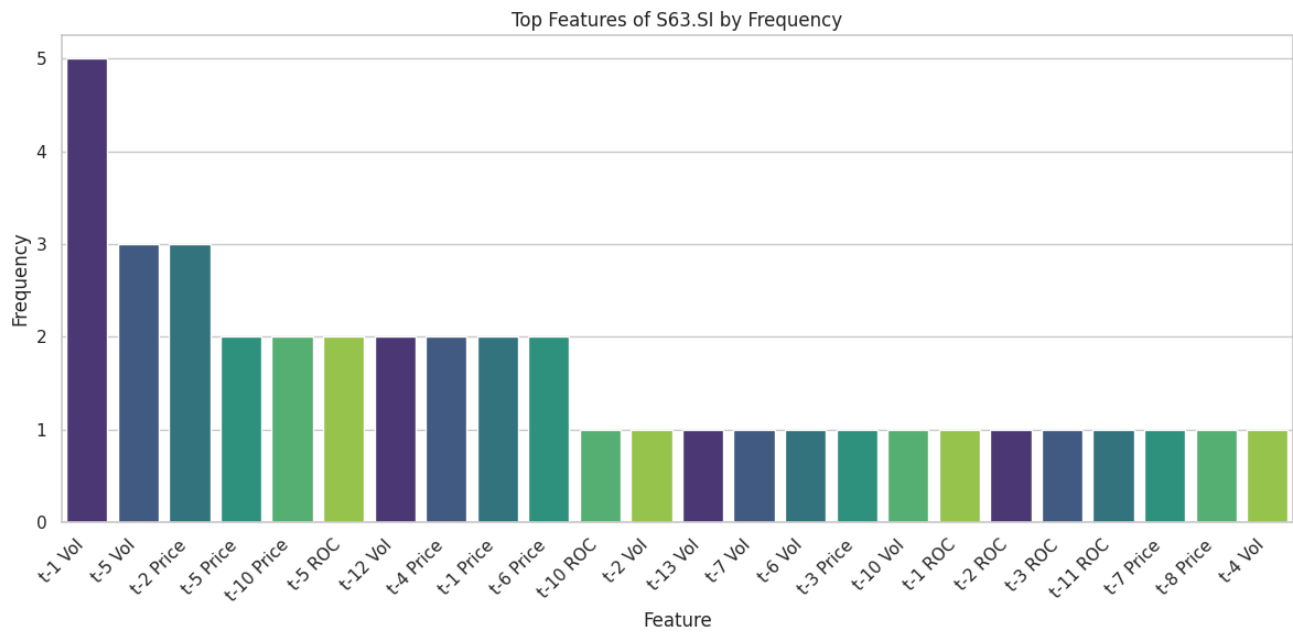


Figure A.5 Frequency Statistics of the features of MCES of S63.SI

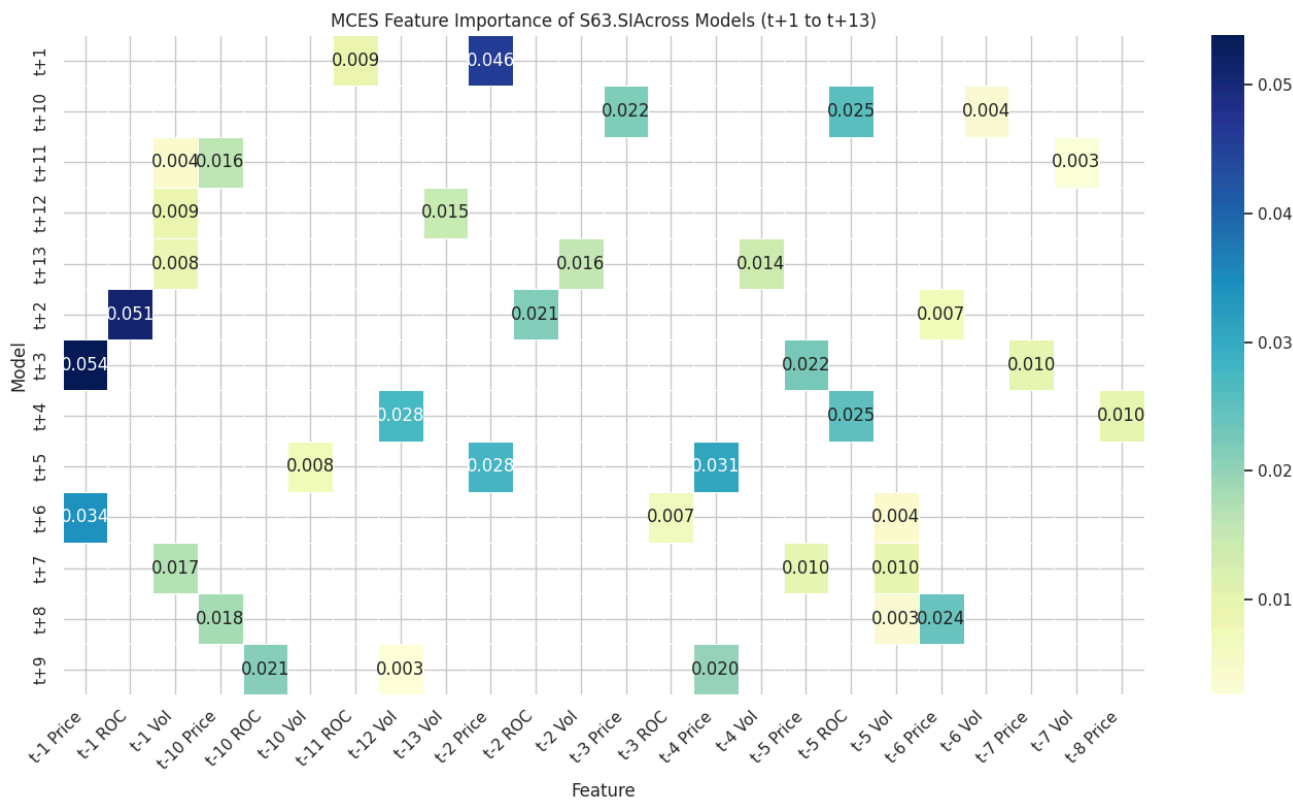


Figure A.6 MCES Feature Importance across (t+1 to t+13) of S63.SI

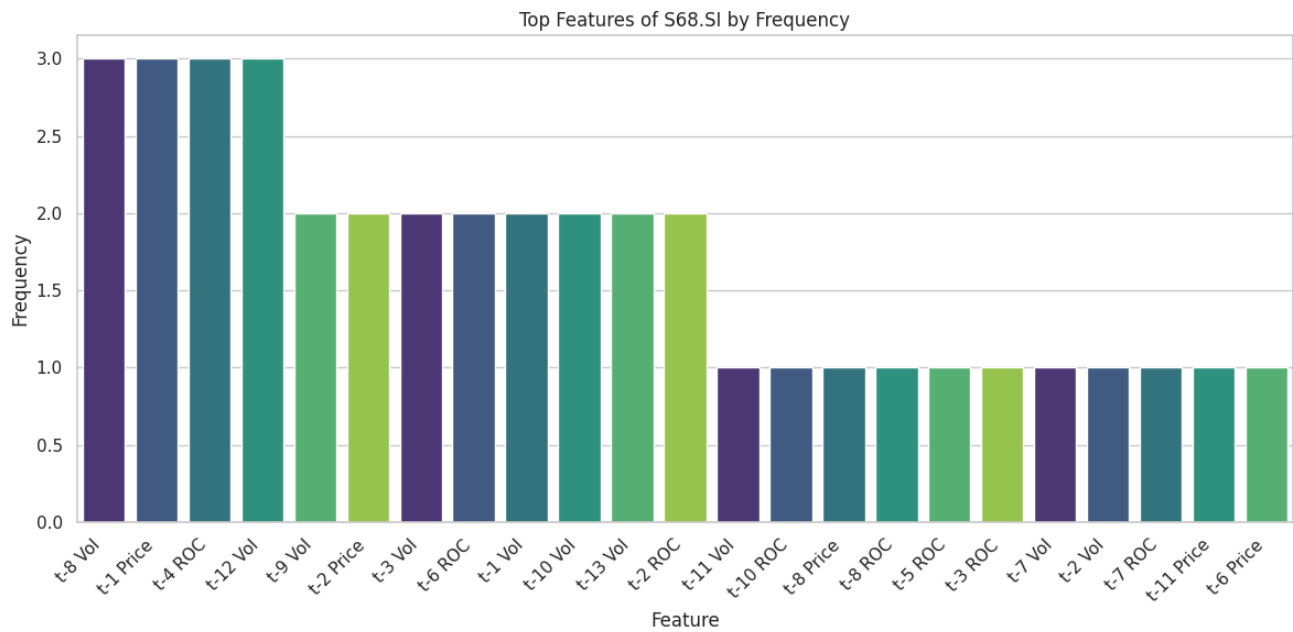


Figure A.7 Frequency Statistics of the features of MCES of S68.SI

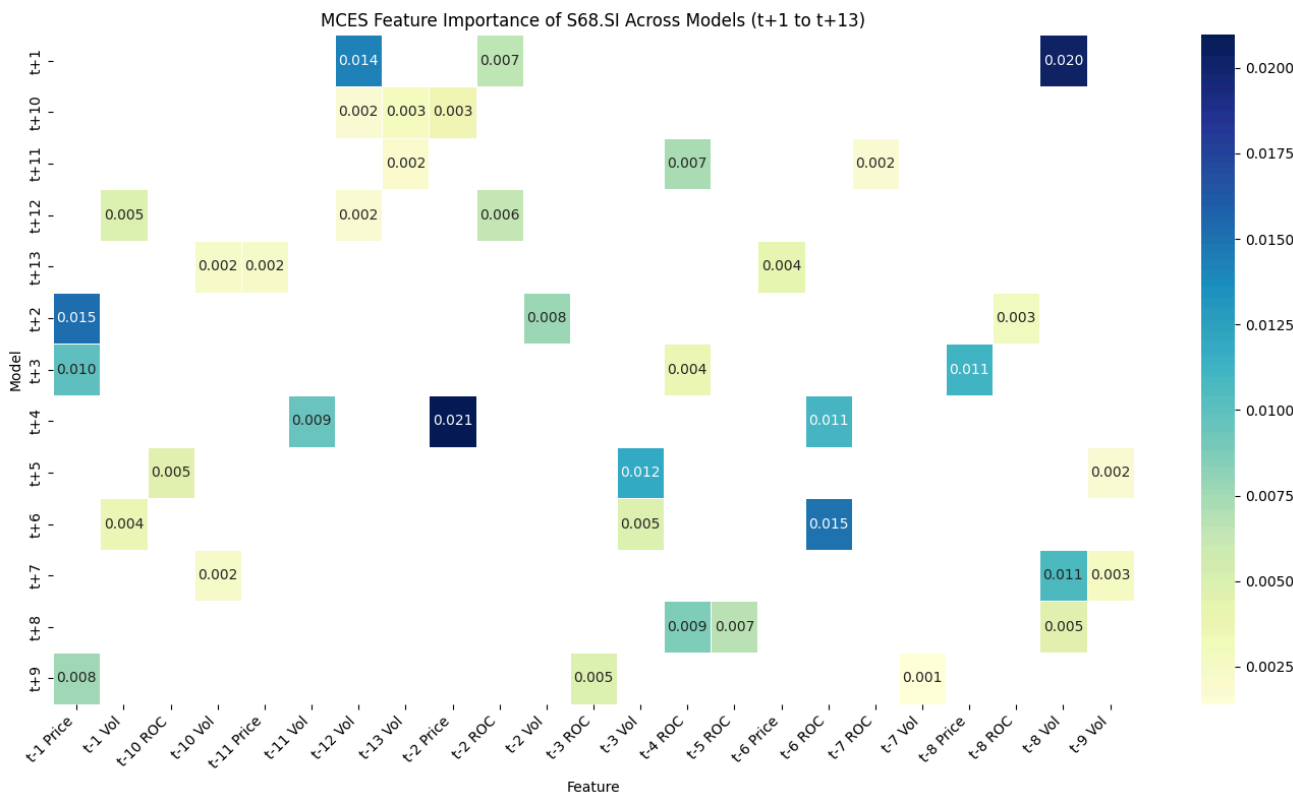


Figure A.8 MCES Feature Importance across (t+1 to t+13) of S68.SI

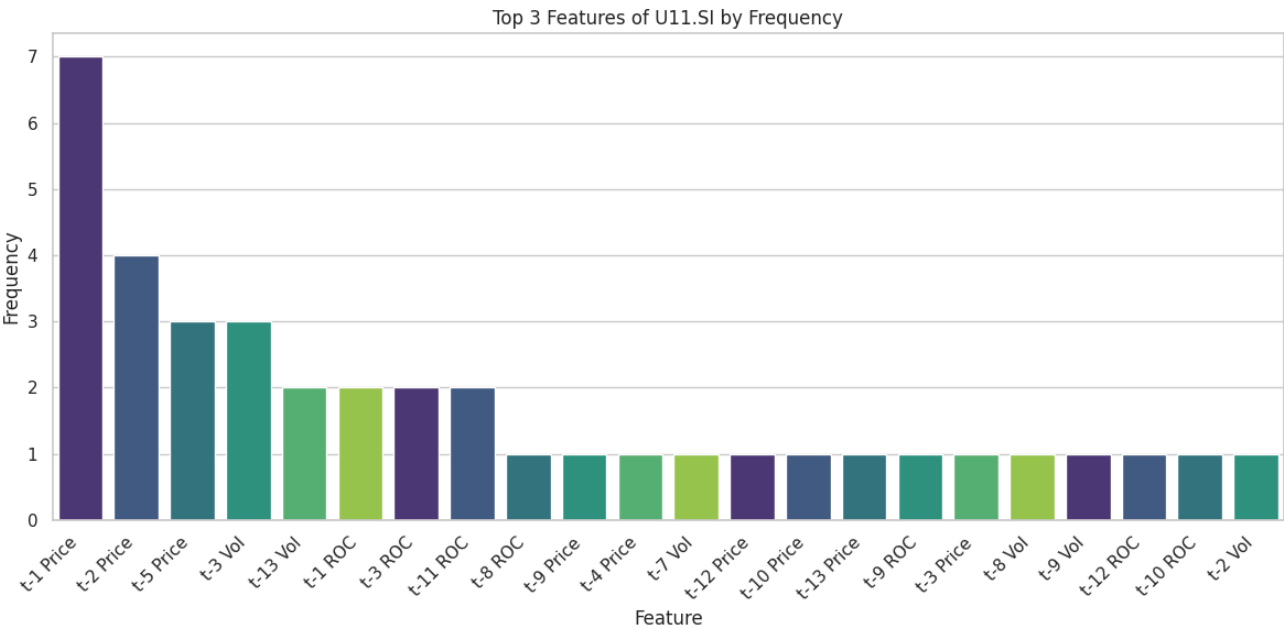


Figure A.9 Frequency Statistics of the features of MCES of U11.SI

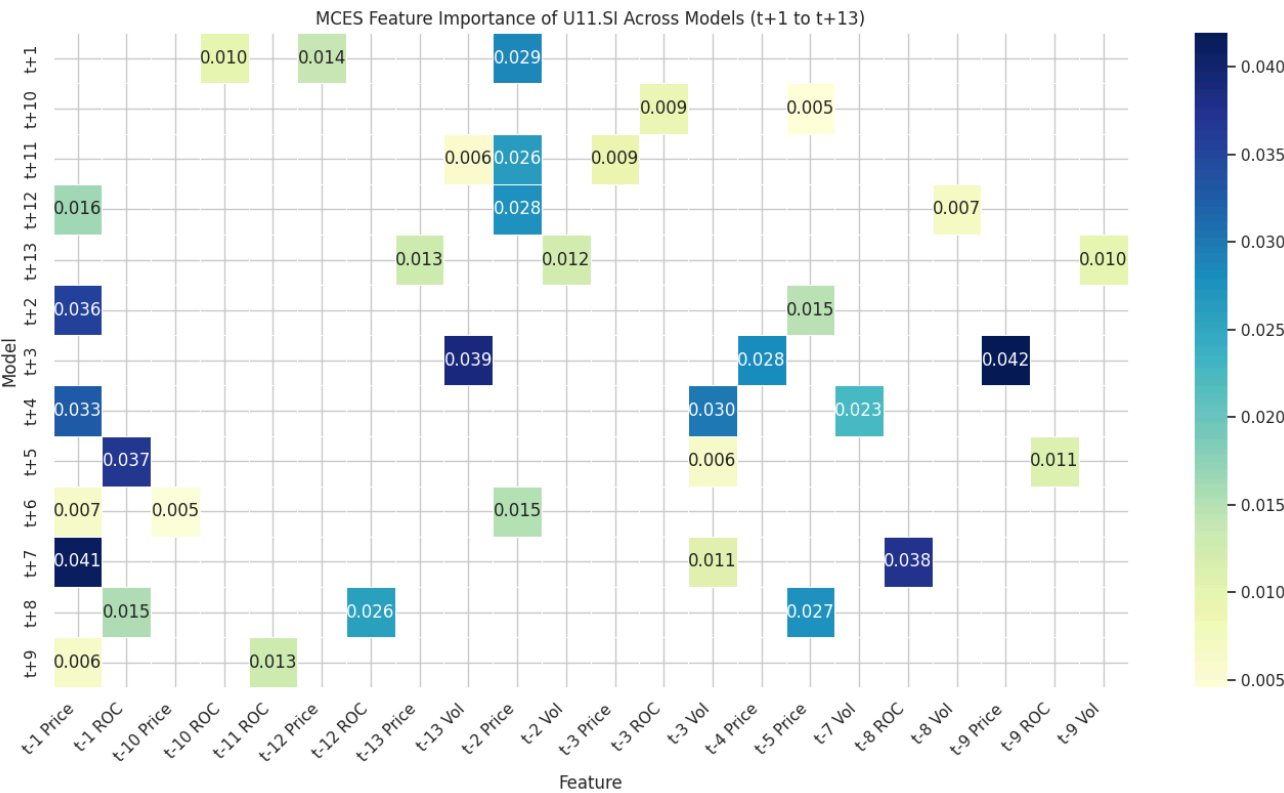


Figure A.10 MCES Feature Importance across (t+1 to t+13) of U11.SI

